I’m a RoboScientist

56 Arduino UNO 3D Product Project Manual

Connection Details • Circuit Diagram • Complete Code

Prepared BY ScienceUtsav RoboScientist

Student / Team	
School / Centre	
Batch / Grade	
Academic Year	

Standard Shield Configuration

These are the standard connections supplied for the Arduino UNO shield. Project-specific tables later in this manual override the generic pin choices whenever a project has a pin conflict.

Device	Standard connection
Arduino board	Arduino UNO
LCD 16x2 I2C	SDA A4, SCL A5, I2C address 0x27
Push buttons	D5, D6 and D7 where available
DIP LEDs	D2 to D7
RGB LED	Red D11, Green D10, Blue D13
Servo signal	D2, D3 or D4 where available
RC522 RFID	SS D10, SCK D13, MOSI D11, MISO D12, RST D8
RTC	SDA A4 and SCL A5
Stepper standalone	IN1 D10, IN2 D11, IN3 D12, IN4 D13
DFPlayer standalone	Arduino RX D8 and Arduino TX D9
MQ sensors	A0 to A3
Pulse sensor	Analog input, normally A0

Essential Hardware Safety

- **RC522 voltage:** Power the RC522 from the UNO 3.3 V pin. Do not directly apply a 5 V GPIO HIGH to its VCC pin. Proper level shifting is recommended for robust use with an UNO.
- **1 W LED / lamp:** D5 or another Arduino pin is only a control signal. Use a transistor or logic-level MOSFET, current limiting and an appropriate external supply.
- **Stepper motor:** Use a ULN2003, L293D or suitable driver. Do not power the motor windings from Arduino I/O pins. Connect the external supply ground to Arduino ground.
- **Servo motor:** A separate regulated 5 V supply is preferred for reliable movement. Connect the servo supply ground to Arduino ground.
- **Pulse sensor:** Connect the signal to an analog input. These activities are educational demonstrations and are not medical devices.
- **MQ sensors:** Allow warm-up time and calibrate the module. The code thresholds are starting values, not universal gas-concentration measurements.
- **DFPlayer Mini:** Connect DFPlayer TX to Arduino RX. Connect Arduino TX to DFPlayer RX through about a 1 kΩ resistor. Use a stable supply and common ground.

Project Index — Projects 1 to 40

No.	Project name	Main components
1	Static Text Display	LCD 16x2 I2C
2	Moving Text Display	LCD 16x2 I2C, 2 push buttons
3	Word Chain Starter	LCD 16x2 I2C, push button
4	Lucky Draw	LCD 16x2 I2C, push button
5	Game of Luck	LCD 16x2 I2C, 2 push buttons
6	Buzz Counter	4x4 keypad, buzzer
7	Sequence Memory Game	LCD 16x2 I2C, 4x4 keypad, start button
8	Simple Calculator	4x4 keypad, LCD 16x2 I2C
9	Password Check	LCD 16x2 I2C, 4x4 keypad
10	Key Lock Door	4x4 keypad, servo motor
11	3-Try Lock	4x4 keypad, servo motor, buzzer
12	Multi-User Access	LCD 16x2 I2C, 4x4 keypad, stepper motor with driver
13	Tag Detector	RC522 RFID reader, LED, buzzer
14	Person Identifier	RC522 RFID reader, LCD 16x2 I2C
15	RFID Door Unlock	RC522 RFID reader, servo motor, buzzer
16	Smart Equipment Log	RC522 RFID reader, LCD 16x2 I2C, servo motor
17	Attendance Gate	RC522 RFID reader, LCD 16x2 I2C
18	Multi-Safety Locker	RC522 RFID reader, LCD 16x2 I2C, 4x4 keypad, servo motor
19	Motion LED	Microwave radar sensor, LED
20	Intrusion Lock	Microwave radar sensor, servo motor, buzzer
21	Motion Lamp	Microwave radar sensor, 1 W LED/lamp with driver
22	Motion Alarm	Microwave radar sensor, buzzer
23	Auto Light + Alert	Microwave radar sensor, LED, buzzer
24	Automatic Gate System	Microwave radar sensor, stepper motor with driver
25	Guest Welcome Box	Microwave radar sensor, DFPlayer Mini, speaker, microSD card
26	Air Quality / Gas Reader	MQ135/MQ6/MQ2 sensor, LCD 16x2 I2C
27	Safe / Unsafe Level Display	MQ135 sensor, LCD 16x2 I2C
28	RFID player	RC522 RFID reader, DFPlayer Mini, speaker, microSD card
29	Air Quality Indicator	MQ135 sensor, 3 LEDs, buzzer
30	Clean-Air Safety Door	MQ135 sensor, LCD 16x2 I2C, servo motor
31	Multi-Safety Door	MQ2/MQ6 sensor, MQ135 sensor, LCD 16x2 I2C, servo motor
32	Gas Leakage Alert System	MQ sensor, DFPlayer Mini, speaker, microSD card
33	Musical Alarm System	DS3231 RTC, LCD 16x2 I2C, 4x4 keypad, push button, DFPlayer Mini, speaker
34	Heart Rate Display	Pulse sensor
35	Safe Range Check	Pulse sensor, LCD 16x2 I2C
36	Tag LED Controller	RC522 RFID reader, 3 LEDs, buzzer
37	Changing Text Display	LCD 16x2 I2C
38	Operator Wellness Alert	Pulse sensor, servo motor, buzzer
39	Clock with Heart Rate	DS3231 RTC, pulse sensor, LCD 16x2 I2C
40	Patient Reminder System	DS3231 RTC, pulse sensor, LCD 16x2 I2C, buzzer

Project Index — Projects 41 to 56

No.	Project name	Main components
41	Direction / Speed Control	Stepper motor with driver, 2 push buttons
42	Motor Positioning	LCD 16x2 I2C, 4x4 keypad, stepper motor with driver
43	Simple Password Door	4x4 keypad, servo motor
44	Vault Dial	4x4 keypad, stepper motor with driver
45	Time-Based Motor	DS3231 RTC, stepper motor with driver
46	RFID Conveyor Belt System	RC522 RFID reader, LCD 16x2 I2C, stepper motor with driver
47	Industry Machinery Setup	RC522 RFID reader, stepper motor with driver
48	Industrial Machinery Basic Secured	RC522 RFID reader, MQ2 sensor, LCD 16x2 I2C, stepper motor with driver
49	Full Smart Automated Industry Setup	RC522 RFID reader, MQ135 sensor, pulse sensor, DS3231 RTC, LCD 16x2 I2C, stepper motor with driver, override button
50	Time Display	DS3231/DS1307 RTC, LCD 16x2 I2C
51	Stopwatch	LCD 16x2 I2C, 2 push buttons
52	Alarm Clock	DS3231 RTC, LCD 16x2 I2C, 3 push buttons, buzzer
53	Time-Based Lock	DS3231 RTC, LCD 16x2 I2C, servo motor
54	Shuffle Playlist	DFPlayer Mini, speaker, microSD card
55	Song Controller	DFPlayer Mini, speaker, microSD card, 2 push buttons
56	Keypad Song Control	DFPlayer Mini, speaker, microSD card, 4x4 keypad

PROJECT 01

Static Text Display

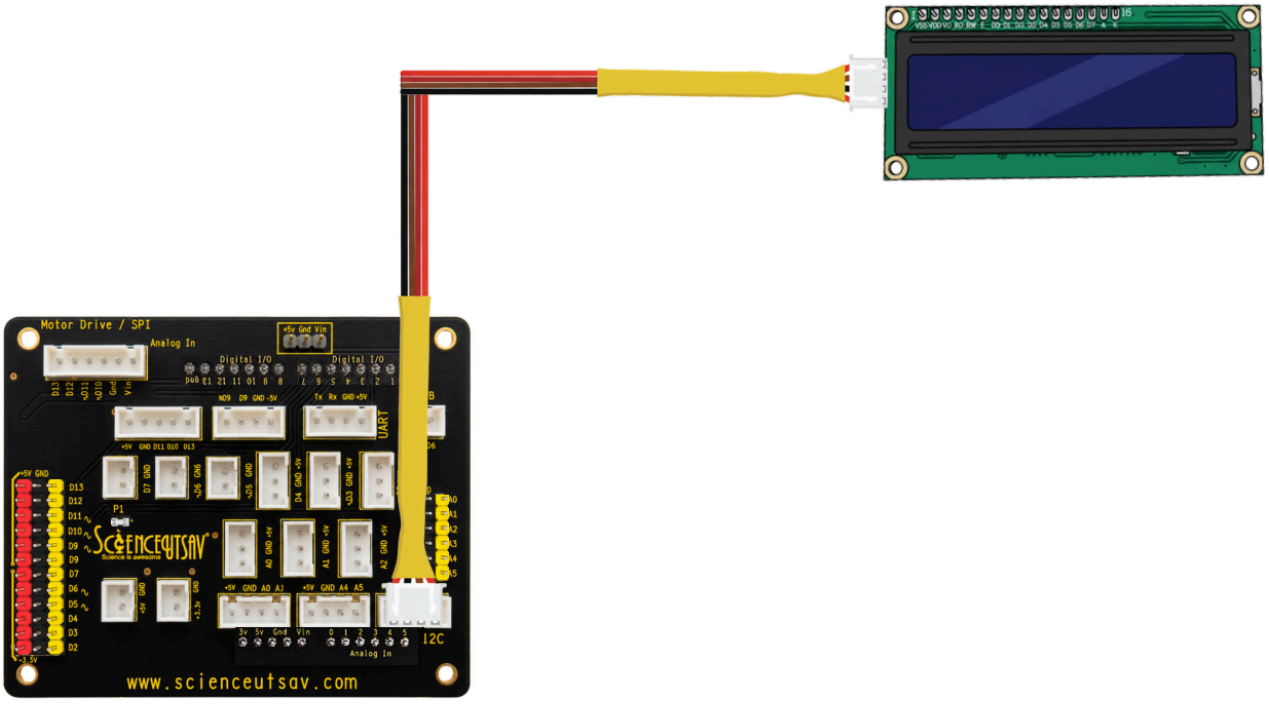
Display stationary text on the LCD screen.

Board	Arduino UNO
Components	LCD 16x2 I2C
Required libraries	Wire.h; LiquidCrystal_I2C.h
Sketch file	01_Static_Text_Display.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.

Project 01: Static Text Display — Circuit Diagram



Project 01: Static Text Display — Arduino Code

Sketch file: 01_Static_Text_Display.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Safety First");
  lcd.setCursor(0, 1);
  lcd.print("Stay Alert");
}
void loop() {}
```

PROJECT 02

Moving Text Display

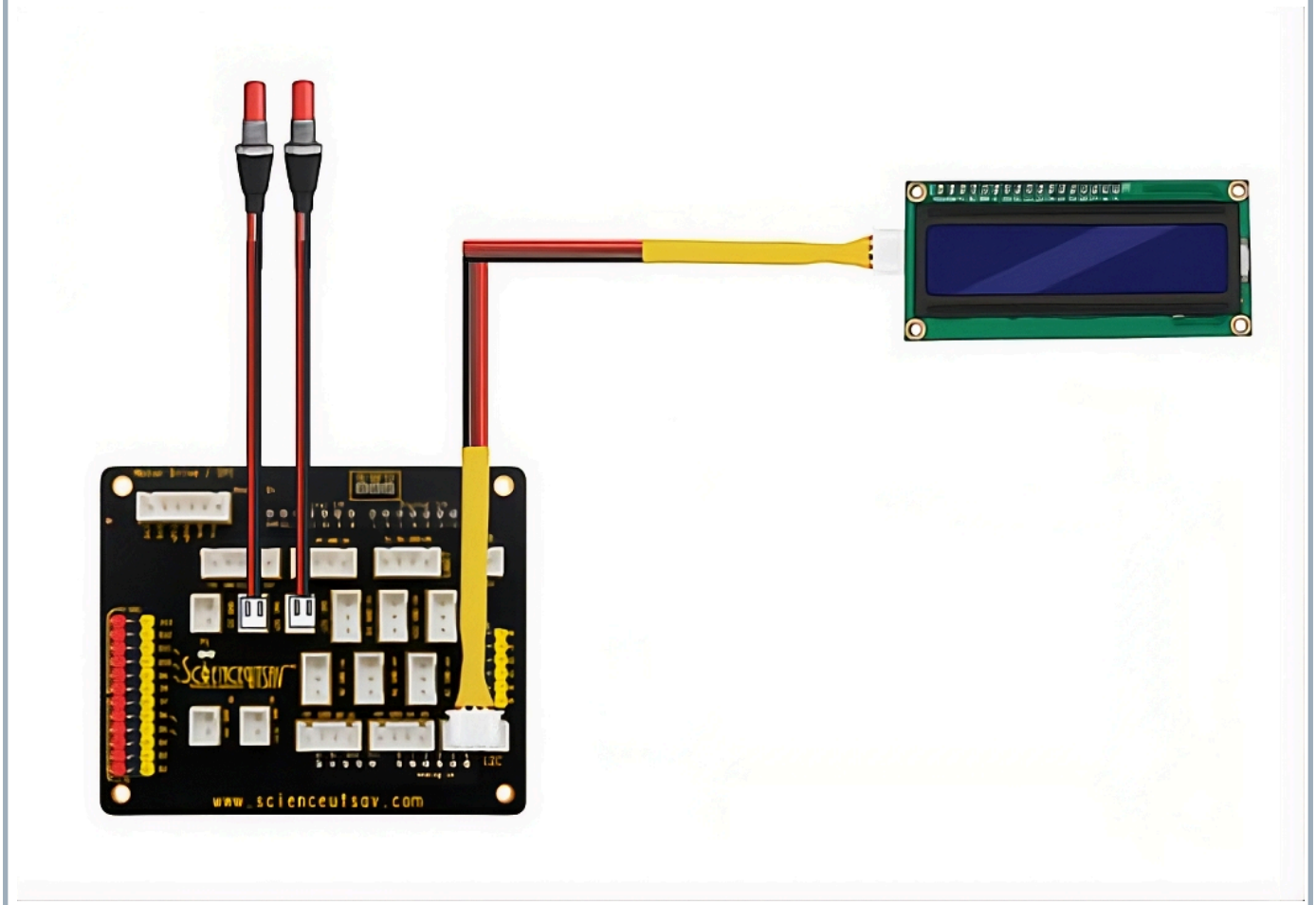
Scroll text and control the movement direction using two push buttons.

Board	Arduino UNO
Components	LCD 16x2 I2C, 2 push buttons
Required libraries	Wire.h; LiquidCrystal_I2C.h
Sketch file	02_Moving_Text_Display.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
Left Button	Signal terminal	5	Other terminal to GND; code uses INPUT_PULLUP unless noted.
Right Button	Signal terminal	6	Other terminal to GND; code uses INPUT_PULLUP unless noted.

Project 02: Moving Text Display — Circuit Diagram



Project 02: Moving Text Display — Arduino Code

Sketch file: 02_Moving_Text_Display.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);

const byte LEFT_BUTTON = 5;
const byte RIGHT_BUTTON = 6;
String message = "   EMBEDDED AI SAFETY SYSTEM   ";
int offset = 0;
int direction = 1;
unsigned long lastMove = 0;

void setup() {
  pinMode(LEFT_BUTTON, INPUT_PULLUP);
  pinMode(RIGHT_BUTTON, INPUT_PULLUP);
  lcd.init(); lcd.backlight();
}
void loop() {
  if (!digitalRead(LEFT_BUTTON)) direction = -1;
  if (!digitalRead(RIGHT_BUTTON)) direction = 1;

  if (millis() - lastMove >= 250) {
    lastMove = millis();
    offset += direction;
    if (offset < 0) offset = message.length() - 1;
    if (offset >= (int)message.length()) offset = 0;

    lcd.setCursor(0, 0);
    for (byte i = 0; i < 16; i++) {
      lcd.print(message[(offset + i) % message.length()]);
    }
    lcd.setCursor(0, 1);
    lcd.print(direction < 0 ? "Direction: LEFT " : "Direction: RIGHT");
  }
}
```

PROJECT 03

Word Chain Starter

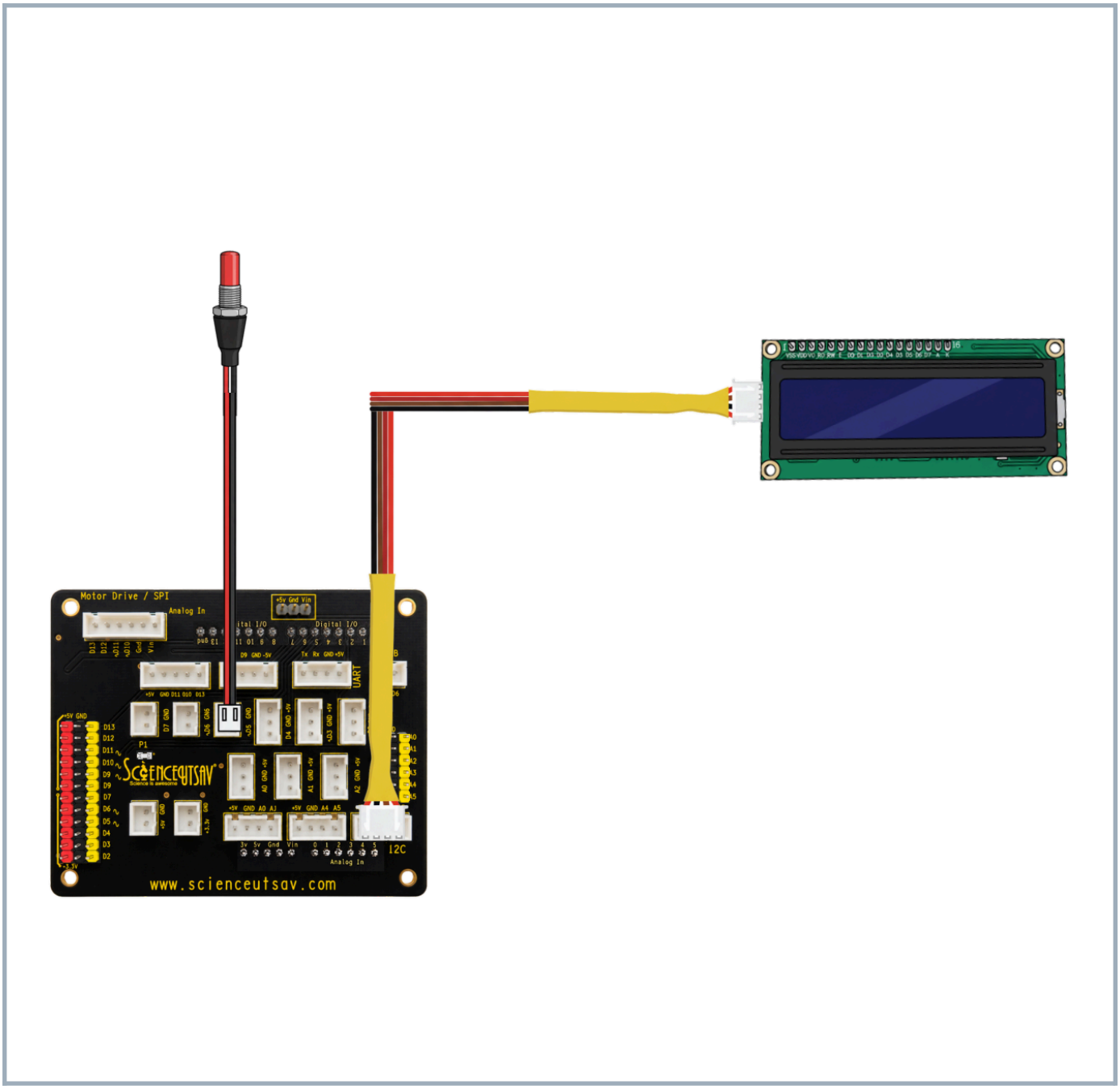
Randomly select and display a letter to start a word-chain game.

Board	Arduino UNO
Components	LCD 16x2 I2C, push button
Required libraries	Wire.h; LiquidCrystal_I2C.h
Sketch file	03_Word_Chain_Starter.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
Button	Signal terminal	5	Other terminal to GND; code uses INPUT_PULLUP unless noted.

Project 03: Word Chain Starter — Circuit Diagram



Project 03: Word Chain Starter — Arduino Code

Sketch file: 03_Word_Chain_Starter.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
const byte BUTTON = 5;
bool previous = HIGH;

void pickLetter() {
  char letter = 'A' + random(0, 26);
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print("Start with:");
  lcd.setCursor(6, 1); lcd.print(letter);
}
void setup() {
  pinMode(BUTTON, INPUT_PULLUP);
  randomSeed(analogRead(A3));
  lcd.init(); lcd.backlight();
  lcd.print("Press to start");
}
void loop() {
  bool now = digitalRead(BUTTON);
  if (previous == HIGH && now == LOW) pickLetter();
  previous = now;
  delay(15);
}
```

PROJECT 04

Lucky Draw

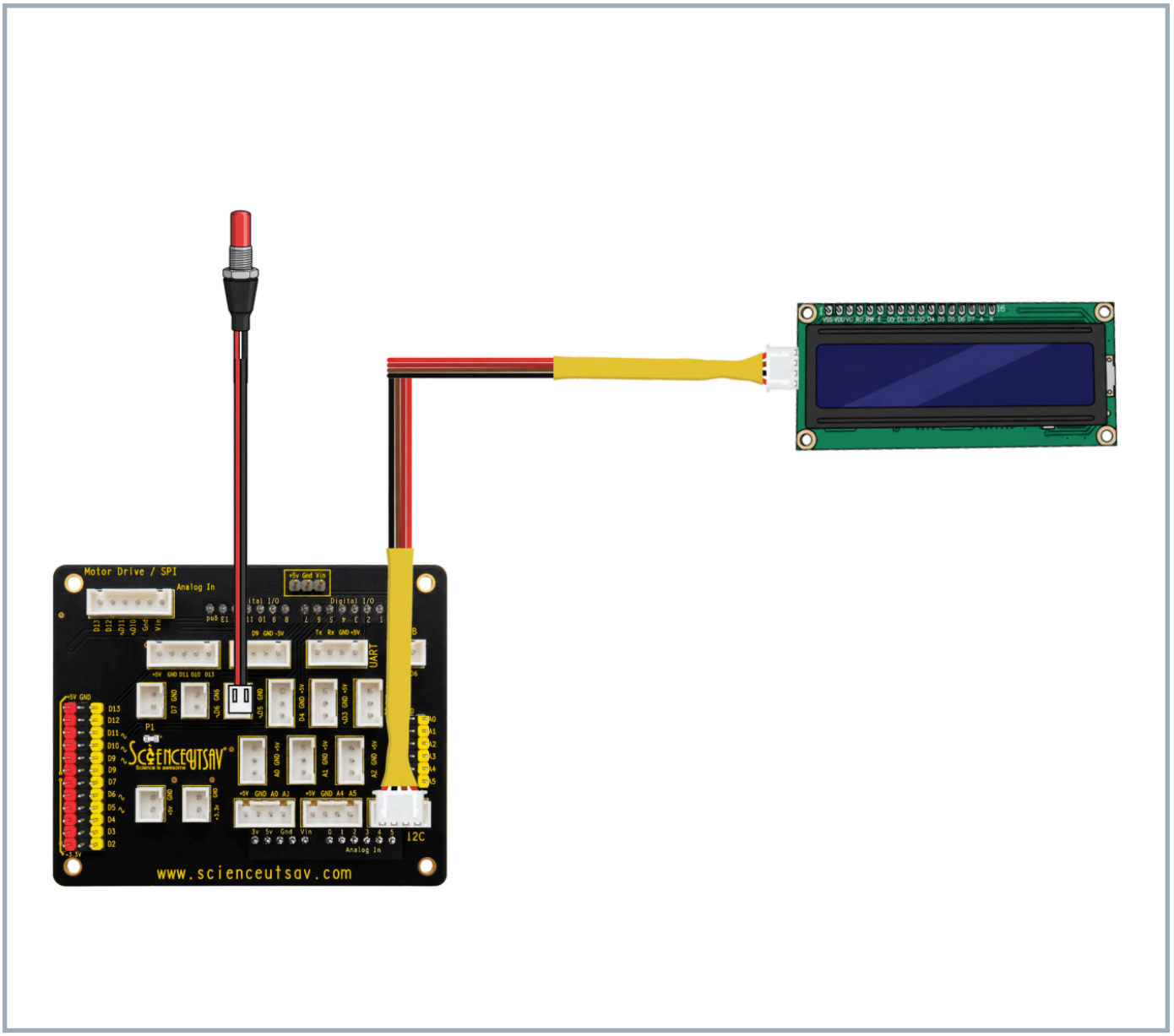
Randomly select and display one player name from a stored group.

Board	Arduino UNO
Components	LCD 16x2 I2C, push button
Required libraries	Wire.h; LiquidCrystal_I2C.h
Sketch file	04_Lucky_Draw.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
Button	Signal terminal	5	Other terminal to GND; code uses INPUT_PULLUP unless noted.

Project 04: Lucky Draw — Circuit Diagram



Project 04: Lucky Draw — Arduino Code

Sketch file: 04_Lucky_Draw.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
const byte BUTTON = 5;
const char* players[] = {"Aarav", "Diya", "Kabir", "Meera", "Rohan", "Sara"};
const byte PLAYER_COUNT = 6;
bool previous = HIGH;

void setup() {
  pinMode(BUTTON, INPUT_PULLUP);
  randomSeed(analogRead(A3));
  lcd.init(); lcd.backlight();
  lcd.print("Press Lucky Draw");
}
void loop() {
  bool now = digitalRead(BUTTON);
  if (previous == HIGH && now == LOW) {
    byte winner = random(PLAYER_COUNT);
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("Winner:");
    lcd.setCursor(0, 1); lcd.print(players[winner]);
  }
  previous = now;
  delay(15);
}
```

PROJECT 05

Game of Luck

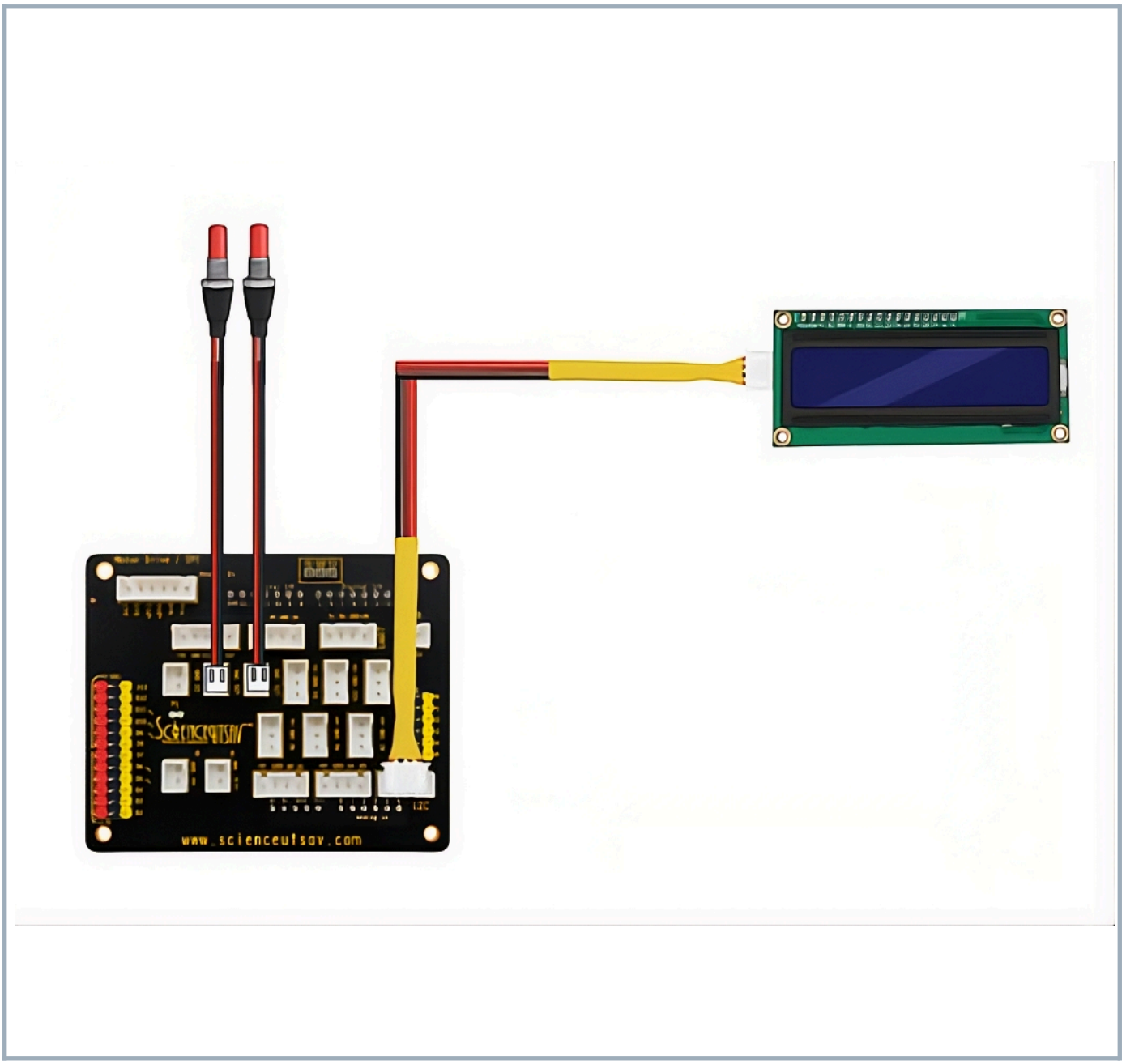
Two players press separate buttons for ten rounds; the hidden lucky button earns a point and the highest score wins.

Board	Arduino UNO
Components	LCD 16x2 I2C, 2 push buttons
Required libraries	Wire.h; LiquidCrystal_I2C.h
Sketch file	05_Game_of_Luck.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
Player1 Button	Signal terminal	5	Other terminal to GND; code uses INPUT_PULLUP unless noted.
Player2 Button	Signal terminal	6	Other terminal to GND; code uses INPUT_PULLUP unless noted.

Project 05: Game of Luck — Circuit Diagram



Project 05: Game of Luck — Arduino Code

Sketch file: 05_Game_of_Luck.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
const byte PLAYER1_BUTTON = 5;
const byte PLAYER2_BUTTON = 6;
byte score1 = 0, score2 = 0, roundNo = 0;
bool p1Prev = HIGH, p2Prev = HIGH;
bool roundOpen = true;
byte luckyButton;

void startRound() {
  luckyButton = random(1, 3);
  roundOpen = true;
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print("Round "); lcd.print(roundNo + 1);
  lcd.setCursor(0, 1); lcd.print("P1:"); lcd.print(score1);
  lcd.print(" P2:"); lcd.print(score2);
}
void finishGame() {
  lcd.clear();
  if (score1 > score2) lcd.print("Player 1 Wins");
  else if (score2 > score1) lcd.print("Player 2 Wins");
  else lcd.print("Game is a Draw");
  lcd.setCursor(0, 1);
  lcd.print(score1); lcd.print(" - "); lcd.print(score2);
}
void registerPress(byte player) {
  if (!roundOpen || roundNo >= 10) return;
  roundOpen = false;
  if (player == luckyButton) {
    if (player == 1) score1++; else score2++;
  }
  roundNo++;
  delay(600);
  if (roundNo >= 10) finishGame();
  else startRound();
}
void setup() {
  pinMode(PLAYER1_BUTTON, INPUT_PULLUP);
  pinMode(PLAYER2_BUTTON, INPUT_PULLUP);
  randomSeed(analogRead(A3));
  lcd.init(); lcd.backlight();
  startRound();
}
void loop() {
  bool p1 = digitalRead(PLAYER1_BUTTON);
  bool p2 = digitalRead(PLAYER2_BUTTON);
  if (p1Prev == HIGH && p1 == LOW) registerPress(1);
  if (p2Prev == HIGH && p2 == LOW) registerPress(2);
  p1Prev = p1; p2Prev = p2;
  delay(10);
}
```

PROJECT 06

Buzz Counter

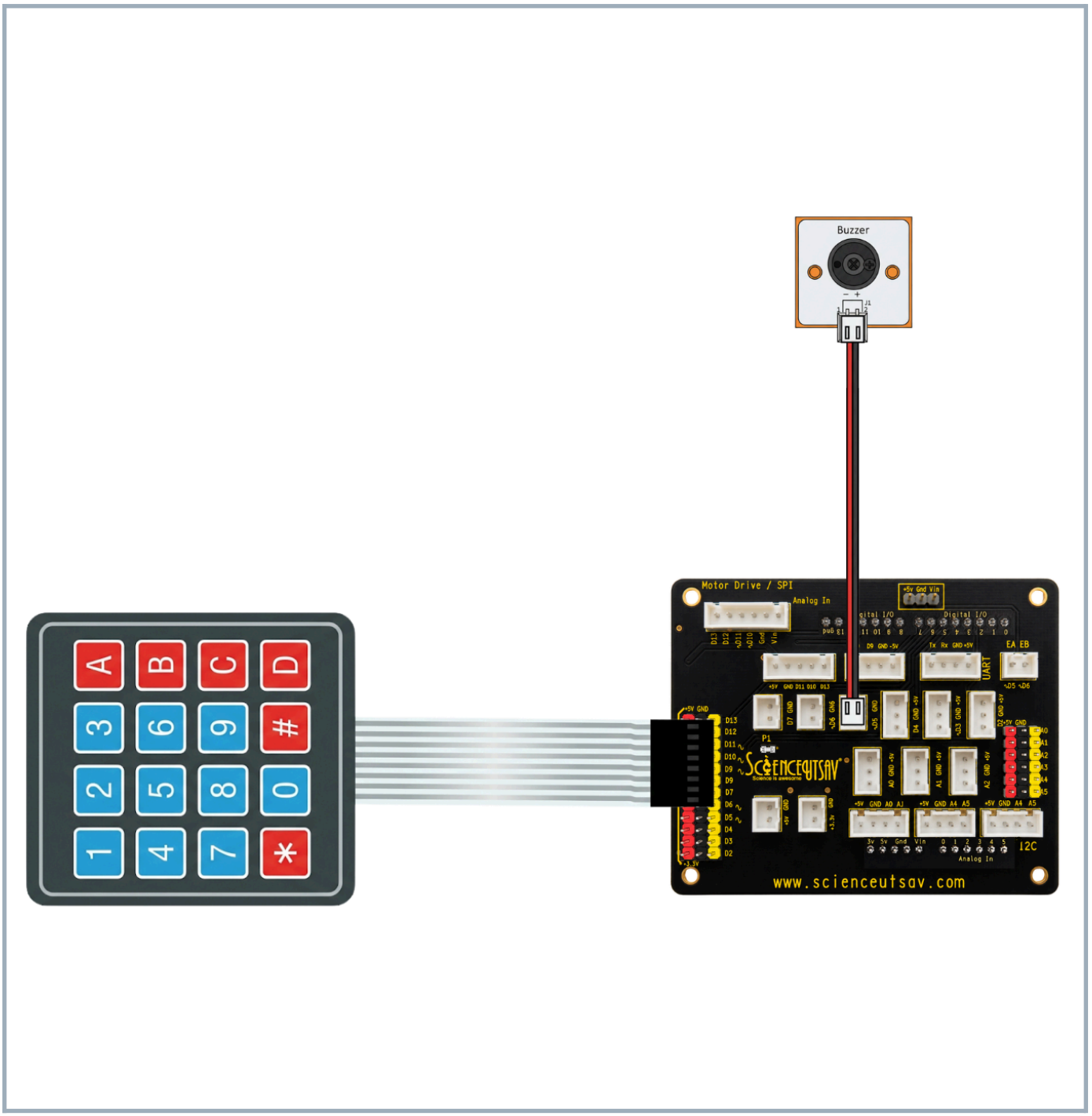
Make the buzzer beep the same number of times as the number key pressed.

Board	Arduino UNO
Components	4x4 keypad, buzzer
Required libraries	Keypad.h
Sketch file	06_Buzz_Counter.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
4x4 keypad	R1	6	Keypad row wire.
4x4 keypad	R2	7	Keypad row wire.
4x4 keypad	R3	8	Keypad row wire.
4x4 keypad	R4	9	Keypad row wire.
4x4 keypad	C1	10	Keypad column wire.
4x4 keypad	C2	11	Keypad column wire.
4x4 keypad	C3	12	Keypad column wire.
4x4 keypad	C4	13	Keypad column wire.
Buzzer	Positive / signal	5	Negative terminal to GND.

Project 06: Buzz Counter — Circuit Diagram



Project 06: Buzz Counter — Arduino Code

Sketch file: 06_Buzz_Counter.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Keypad.h>
const byte BUZZER = 5;
const byte ROWS = 4, COLS = 4;
char keys[ROWS][COLS] = {
  {'1','2','3','A'}, {'4','5','6','B'},
  {'7','8','9','C'}, {'*','0','#','D'}
};
byte rowPins[ROWS] = {6,7,8,9};
byte colPins[COLS] = {10,11,12,13};
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

void beepCount(byte count) {
  for (byte i = 0; i < count; i++) {
    tone(BUZZER, 1800, 150);
    delay(300);
  }
}
void setup() {
  pinMode(BUZZER, OUTPUT);
  Serial.begin(9600);
}
void loop() {
  char key = keypad.getKey();
  if (key >= '0' && key <= '9') {
    byte count = key - '0';
    Serial.print("Beeping "); Serial.print(count); Serial.println(" times");
    beepCount(count);
  }
}
```

PROJECT 07

Sequence Memory Game

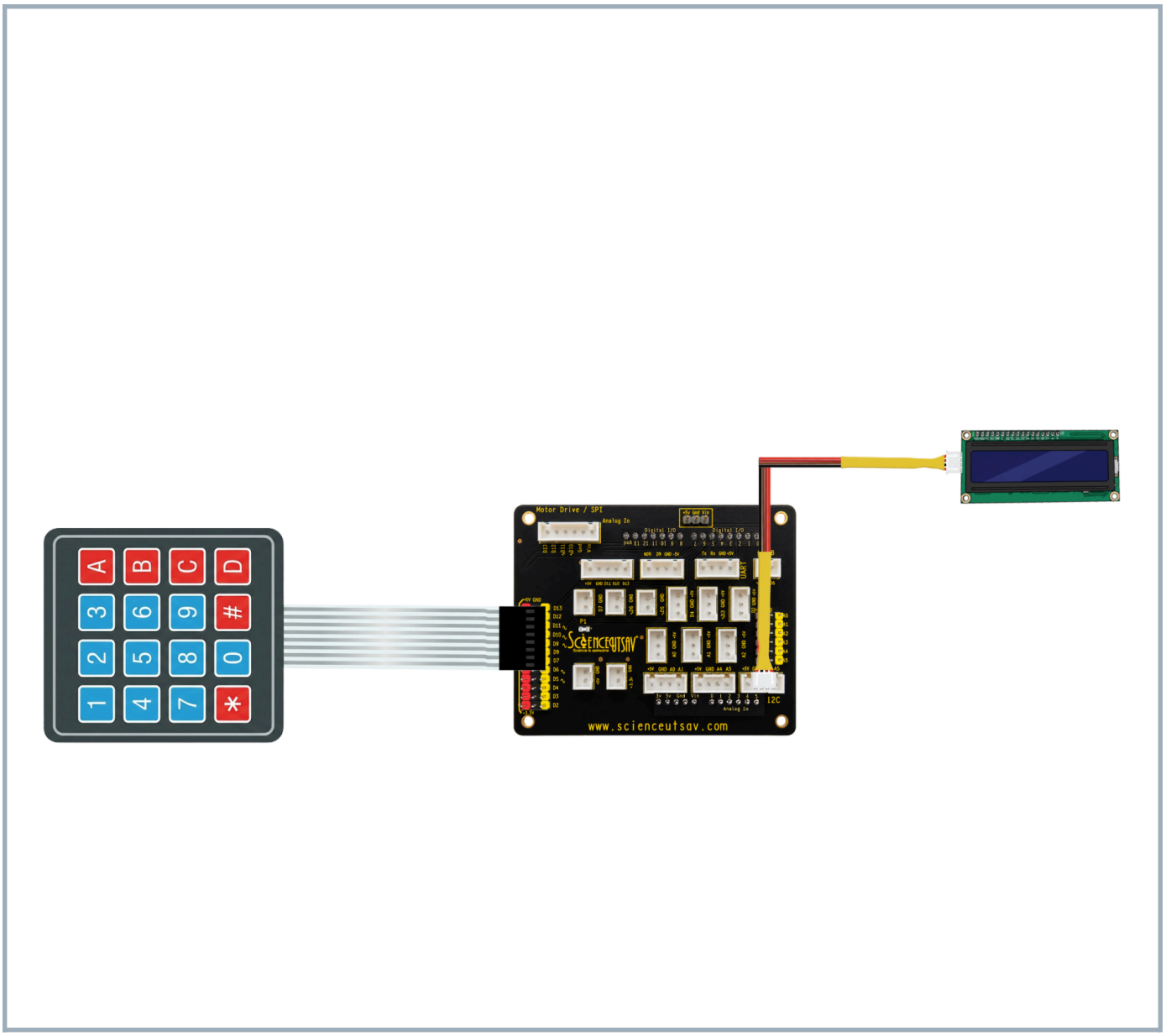
Show a random sequence briefly, then ask the player to re-enter it using the keypad.

Board	Arduino UNO
Components	LCD 16x2 I2C, 4x4 keypad, start button
Required libraries	Wire.h; LiquidCrystal_I2C.h; Keypad.h
Sketch file	07_Sequence_Memory_Game.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
4x4 keypad	R1	6	Keypad row wire.
4x4 keypad	R2	7	Keypad row wire.
4x4 keypad	R3	8	Keypad row wire.
4x4 keypad	R4	9	Keypad row wire.
4x4 keypad	C1	10	Keypad column wire.
4x4 keypad	C2	11	Keypad column wire.
4x4 keypad	C3	12	Keypad column wire.
4x4 keypad	C4	13	Keypad column wire.

Project 07: Sequence Memory Game — Circuit Diagram



Project 07: Sequence Memory Game — Arduino Code

Sketch file: 07_Sequence_Memory_Game.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);

String target = "", entered = "";
bool accepting = false;
const byte ROWS = 4, COLS = 4;
char keys[ROWS][COLS] = {
  {'1','2','3','A'}, {'4','5','6','B'},
  {'7','8','9','C'}, {'*','0','#','D'}
};
byte rowPins[ROWS] = {6,7,8,9};
byte colPins[COLS] = {10,11,12,13};
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

void startGame() {
  target = "";
  for (byte i = 0; i < 8; i++) target += char('0' + random(10));
  lcd.clear(); lcd.print("Remember:");
  lcd.setCursor(0,1); lcd.print(target);
  delay(6000);
  lcd.clear(); lcd.print("Re-enter 8 digits");
  lcd.setCursor(0,1);
  entered = "";
  accepting = true;
}

void showStartPrompt() {
  lcd.clear();
  lcd.print("Press any key");
  lcd.setCursor(0,1);
  lcd.print("to start");
}

void setup() {
  randomSeed(analogRead(A3));
  lcd.init(); lcd.backlight();
  showStartPrompt();
}
```

```
void loop() {
  char key = keypad.getKey();
  if (!key) return;

  if (!accepting) {
    startGame();
    return;
  }

  if (key >= '0' && key <= '9' && entered.length() < 8) {
    entered += key;
    lcd.print(key);
  }
  if (entered.length() == 8) {
    accepting = false;
    delay(300);
    lcd.clear();
    lcd.print(entered == target ? "Correct!" : "Try Again");
    delay(2000);
    showStartPrompt();
  }
}
```

PROJECT 08

Simple Calculator

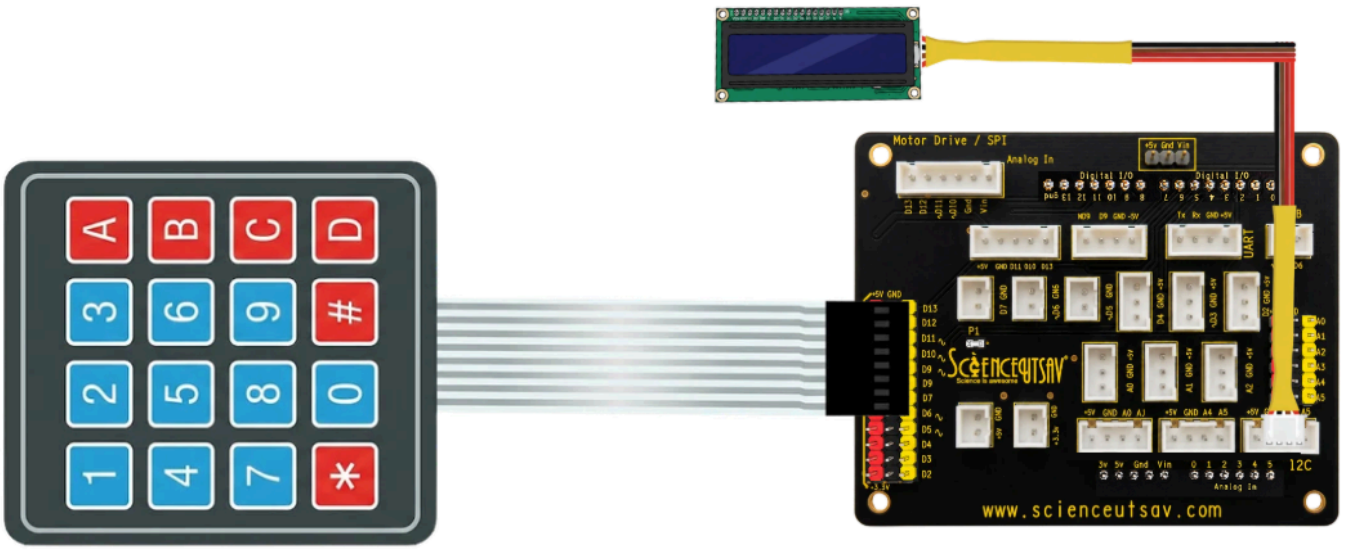
Enter numbers and operators on the keypad and display the result on the LCD.

Board	Arduino UNO
Components	4x4 keypad, LCD 16x2 I2C
Required libraries	Wire.h; LiquidCrystal_I2C.h; Keypad.h
Sketch file	08_Simple_Calculator.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
4x4 keypad	R1	6	Keypad row wire.
4x4 keypad	R2	7	Keypad row wire.
4x4 keypad	R3	8	Keypad row wire.
4x4 keypad	R4	9	Keypad row wire.
4x4 keypad	C1	10	Keypad column wire.
4x4 keypad	C2	11	Keypad column wire.
4x4 keypad	C3	12	Keypad column wire.
4x4 keypad	C4	13	Keypad column wire.

Project 08: Simple Calculator — Circuit Diagram



Project 08: Simple Calculator — Arduino Code

Sketch file: 08_Simple_Calculator.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
LiquidCrystal_I2C lcd(0x27,16,2);
const byte ROWS=4,COLS=4;
char keys[ROWS][COLS]={{'1','2','3','A'},{'4','5','6','B'},{'7','8','9','C'},{'*','0','#','D'}};
byte rowPins[ROWS]={6,7,8,9};
byte colPins[COLS]={10,11,12,13};
Keypad keypad=Keypad(makeKeymap(keys),rowPins,colPins,ROWS,COLS);
String current="";
long firstNumber=0;
char operation=0;

void resetCalculator(){
  current="";firstNumber=0;operation=0;
  lcd.clear();lcd.print("A+ B- Cx D/");
  lcd.setCursor(0,1);
}
void calculate(){
  if(!operation||current.length()==0)return;
  long second=current.toInt(),result=0;
  bool error=false;
  if(operation=='A')result=firstNumber+second;
  else if(operation=='B')result=firstNumber-second;
  else if(operation=='C')result=firstNumber*second;
  else if(operation=='D'){if(second==0)error=true;else result=firstNumber/second;}
  lcd.clear();lcd.print(error?"Error: divide 0":"Result:");
  if(!error){lcd.setCursor(0,1);lcd.print(result);}
  delay(1800);resetCalculator();
}
void setup(){lcd.init();lcd.backlight();resetCalculator();}
void loop(){
  char key=keypad.getKey();if(!key)return;
  if(isDigit(key)&&current.length()<9){current+=key;lcd.print(key);}
  else if(key=='*')resetCalculator();
  else if(key=='#')calculate();
  else if(key>='A'&&key<='D'&&current.length()){
    firstNumber=current.toInt();operation=key;current="";
    lcd.clear();lcd.print("Operator: ");lcd.print(key);
    lcd.setCursor(0,1);
  }
}
```

PROJECT 09

Password Check

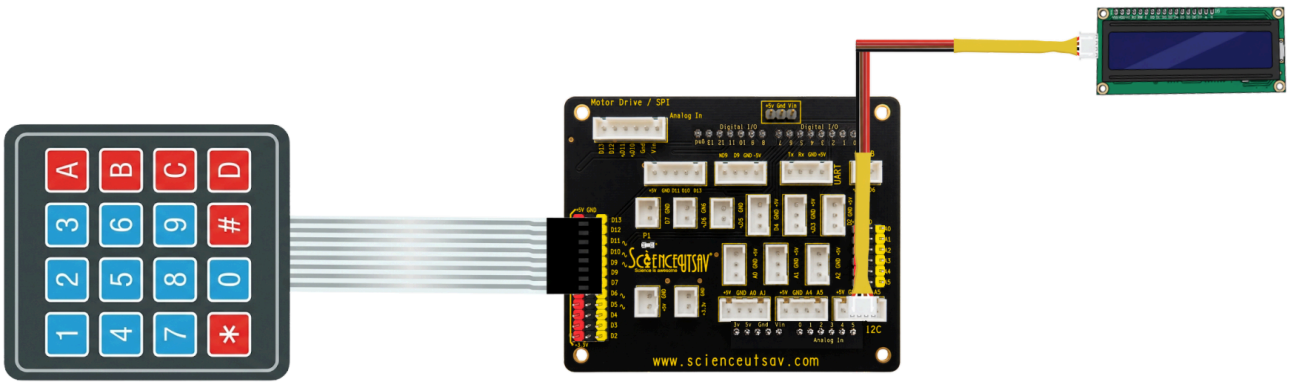
Enter and verify a conventional password using the keypad.

Board	Arduino UNO
Components	LCD 16x2 I2C, 4x4 keypad
Required libraries	Wire.h; LiquidCrystal_I2C.h; Keypad.h
Sketch file	09_Password_Check.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
4x4 keypad	R1	6	Keypad row wire.
4x4 keypad	R2	7	Keypad row wire.
4x4 keypad	R3	8	Keypad row wire.
4x4 keypad	R4	9	Keypad row wire.
4x4 keypad	C1	10	Keypad column wire.
4x4 keypad	C2	11	Keypad column wire.
4x4 keypad	C3	12	Keypad column wire.
4x4 keypad	C4	13	Keypad column wire.

Project 09: Password Check — Circuit Diagram



Project 09: Password Check — Arduino Code

Sketch file: 09_Password_Check.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);

const String PASSWORD = "2580";
String entered = "";
const byte ROWS = 4, COLS = 4;
char keys[ROWS][COLS] = {
  {'1','2','3','A'}, {'4','5','6','B'},
  {'7','8','9','C'}, {'*','0','#','D'}
};
byte rowPins[ROWS] = {6,7,8,9};
byte colPins[COLS] = {10,11,12,13};
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

void resetEntry() {
  entered = "";
  lcd.clear();
  lcd.print("Enter Password:");
  lcd.setCursor(0,1);
}
void setup() {
  lcd.init(); lcd.backlight();
  resetEntry();
}
void loop() {
  char key = keypad.getKey();
  if (!key) return;
  if (key == '*') resetEntry();
  else if (key == '#') {
    lcd.clear();
    lcd.print(entered == PASSWORD ? "Access Granted" : "Wrong Password");
    delay(1800);
    resetEntry();
  } else if (entered.length() < 12) {
    entered += key;
    lcd.print('*');
  }
}
```

PROJECT 10

Key Lock Door

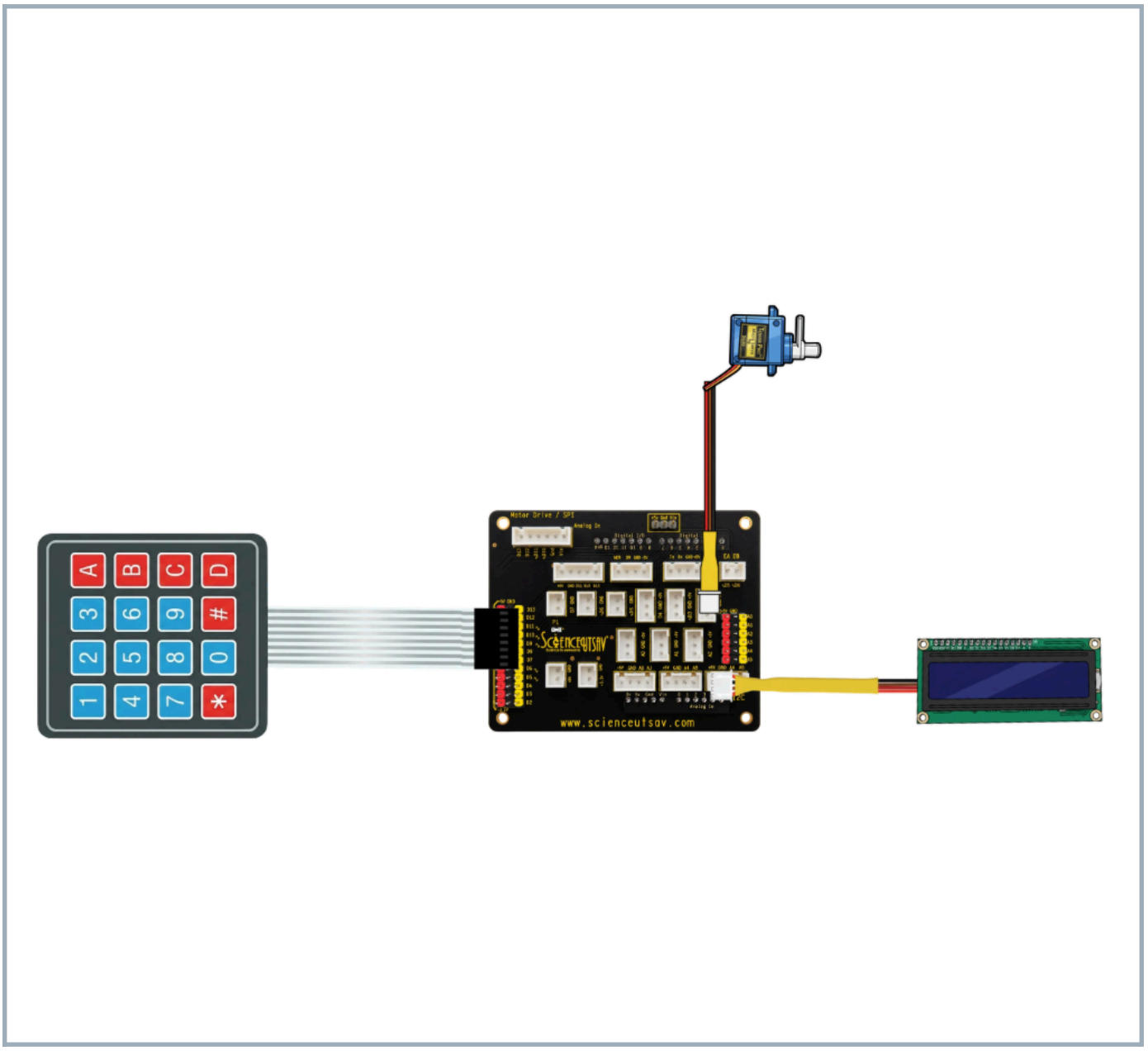
Lock or unlock a model door by pressing an assigned keypad key.

Board	Arduino UNO
Components	4x4 keypad, servo motor, LCD
Required libraries	Keypad.h; Servo.h
Sketch file	10_Key_Lock_Door.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
4x4 keypad	R1	6	Keypad row wire.
4x4 keypad	R2	7	Keypad row wire.
4x4 keypad	R3	8	Keypad row wire.
4x4 keypad	R4	9	Keypad row wire.
4x4 keypad	C1	10	Keypad column wire.
4x4 keypad	C2	11	Keypad column wire.
4x4 keypad	C3	12	Keypad column wire.
4x4 keypad	C4	13	Keypad column wire.
Servo motor	Signal	2	Control signal from Arduino.
Servo motor	VCC	5V / external 5V	External supply is preferred for reliable operation.
Servo motor	GND	GND	Common ground with Arduino.

Project 10: Key Lock Door — Circuit Diagram



Project 10: Key Lock Door — Arduino Code

Sketch file: 10_Key_Lock_Door.ino

```
#include <Keypad.h>
#include <Servo.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

Servo door;

// Servo
const byte SERVO_PIN = 2;

// Access key
const char ACCESS_KEY = 'A';

// LCD
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Keypad
const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};

byte rowPins[ROWS] = {6, 7, 8, 9};
byte colPins[COLS] = {10, 11, 12, 13};

Keypad keypad = Keypad(
  makeKeymap(keys),
  rowPins,
  colPins,
  ROWS,
  COLS
);

void setup() {

  // Servo setup
  door.attach(SERVO_PIN);
```

```
  door.write(0);

  // Serial monitor
  Serial.begin(9600);

  // LCD setup
  lcd.init();
  lcd.backlight();

  // Welcome message
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("SMART DOOR");
  lcd.setCursor(0, 1);
  lcd.print("LOCK SYSTEM");

  delay(2000);

  // Ready screen
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Enter Key:");
  lcd.setCursor(0, 1);
  lcd.print("Press A");
}

void loop() {

  char key = keypad.getKey();

  if (key) {

    Serial.print("Key pressed: ");
    Serial.println(key);

    // Correct key
    if (key == ACCESS_KEY) {

      Serial.println("Door unlocked");

      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("Access Granted");
      lcd.setCursor(0, 1);
      lcd.print("Door Unlocked");

      // Open door
```

```
      door.write(90);

      delay(3000);

      // Close door
      door.write(0);

      Serial.println("Door locked");

      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("Door Locked");
      lcd.setCursor(0, 1);
      lcd.print("Press A");

      delay(1500);

      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("Enter Key:");
      lcd.setCursor(0, 1);
      lcd.print("Press A");
    }

    // Wrong key
    else {

      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("Access Denied!");
      lcd.setCursor(0, 1);
      lcd.print("Wrong Key");

      Serial.println("Wrong key");

      delay(1500);

      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("Enter Key:");
      lcd.setCursor(0, 1);
      lcd.print("Press A");
    }
  }
}
```

PROJECT 11

3-Try Lock

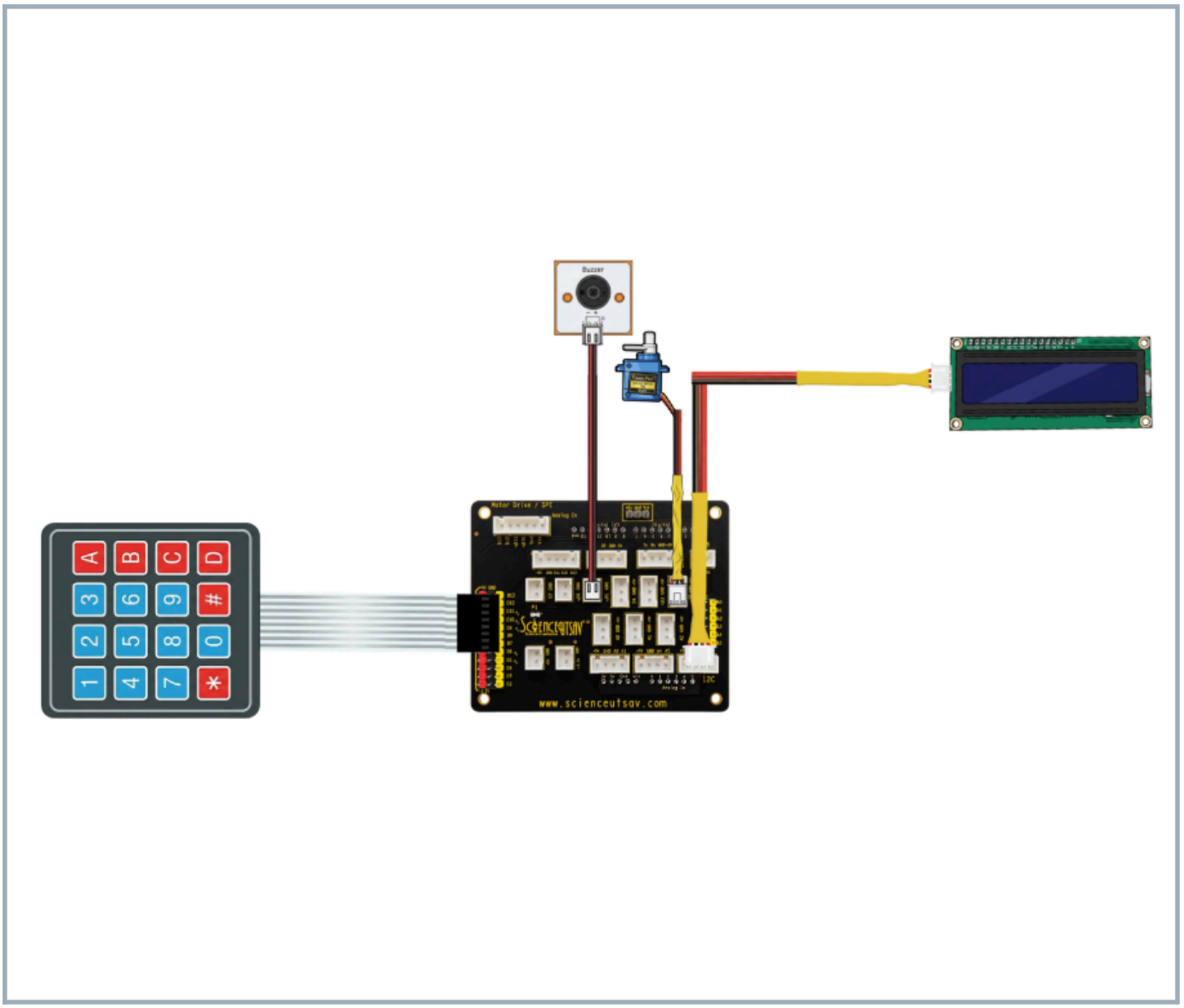
Trigger a permanent lock and alarm after three consecutive wrong password attempts.

Board	Arduino UNO
Components	4x4 keypad, servo motor, buzzer
Required libraries	Keypad.h; Servo.h
Sketch file	11_Three_Try_Lock.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
4x4 keypad	R1	6	Keypad row wire.
4x4 keypad	R2	7	Keypad row wire.
4x4 keypad	R3	8	Keypad row wire.
4x4 keypad	R4	9	Keypad row wire.
4x4 keypad	C1	10	Keypad column wire.
4x4 keypad	C2	11	Keypad column wire.
4x4 keypad	C3	12	Keypad column wire.
4x4 keypad	C4	13	Keypad column wire.
Servo motor	Signal	2	Control signal from Arduino.
Servo motor	VCC	5V / external 5V	External supply is preferred for reliable operation.
Servo motor	GND	GND	Common ground with Arduino.
Buzzer	Positive / signal	5	Negative terminal to GND.

Project 11: 3-Try Lock — Circuit Diagram



Project 11: 3-Try Lock — Arduino Code

Sketch file: 11_Three_Try_Lock.ino

```
/*
 ScienceUtsav RoboScientist Project
 Password Door Lock with LCD

 Board: Arduino UNO
 LCD I2C:
 SDA = A4
 SCL = A5
 Address = 0x27
*/

#include <Keypad.h>
#include <Servo.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ---------------- LCD ----------------
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ---------------- SERVO ----------------
Servo door;

const byte SERVO_PIN = 2;
const byte BUZZER = 5;

// ---------------- PASSWORD ----------------
const String PASSWORD = "2468";
String entered = "";

byte wrongAttempts = 0;
bool permanentlyLocked = false;

// ---------------- KEYPAD ----------------
const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
 {'1','2','3','A'},
 {'4','5','6','B'},
 {'7','8','9','C'},
 {'*','0','#','D'}
};

byte rowPins[ROWS] = {6, 7, 8, 9};
byte colPins[COLS] = {10, 11, 12, 13};
```

```
Keypad keypad = Keypad(
 makeKeymap(keys),
 rowPins,
 colPins,
 ROWS,
 COLS
);


// ================================================
// SETUP
// ================================================
void setup() {

 pinMode(BUZZER, OUTPUT);

 door.attach(SERVO_PIN);
 door.write(0);

 Serial.begin(9600);

 // LCD start
 lcd.init();
 lcd.backlight();

 lcd.setCursor(0, 0);
 lcd.print("PASSWORD LOCK");

 lcd.setCursor(0, 1);
 lcd.print("Enter Password");

 delay(2000);

 showPasswordScreen();
}


// ================================================
// LOOP
// ================================================
void loop() {

 // Permanent lock after 3 wrong attempts
 if (permanentlyLocked) {

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("SYSTEM LOCKED");

  lcd.setCursor(0, 1);
  lcd.print("RESET ARDUINO");
```

```
  tone(BUZZER, 1200, 200);

  delay(1000);

  return;
}


char key = keypad.getKey();

if (!key)
  return;


// ---------------------------------
// CLEAR PASSWORD
// ---------------------------------
if (key == '*') {

  entered = "";

  showPasswordScreen();
}


// ---------------------------------
// CHECK PASSWORD
// ---------------------------------
else if (key == '#') {

  if (entered == PASSWORD) {

    wrongAttempts = 0;

    Serial.println("Access granted");

    lcd.clear();

    lcd.setCursor(0, 0);
    lcd.print("ACCESS GRANTED");

    lcd.setCursor(0, 1);
    lcd.print("Door Opening");

    tone(BUZZER, 1500, 150);

    door.write(90);

    delay(2500);

    door.write(0);
```

```
   lcd.clear();

   lcd.setCursor(0, 0);
   lcd.print("DOOR CLOSED");

   delay(1000);

  }

  else {

   wrongAttempts++;

   Serial.print("Wrong attempt: ");
   Serial.println(wrongAttempts);

   lcd.clear();

   lcd.setCursor(0, 0);
   lcd.print("WRONG PASSWORD");

   lcd.setCursor(0, 1);
   lcd.print("Attempt: ");
   lcd.print(wrongAttempts);
   lcd.print("/3");

   tone(BUZZER, 500, 500);

   delay(1500);


   // Lock after 3 wrong attempts
   if (wrongAttempts >= 3) {

     permanentlyLocked = true;

     Serial.println("LOCKED until Arduino reset");

     return;
   }
  }

  entered = "";

  showPasswordScreen();
}


// --------------------------------
// PASSWORD INPUT
// --------------------------------
else {
```

```
  if (entered.length() < 12) {

    entered += key;

    lcd.setCursor(0, 1);

    // Show * instead of actual password
    for (byte i = 0; i < entered.length(); i++) {
      lcd.print("*");
    }
   }
  }
}


// ================================================
// PASSWORD SCREEN
// ================================================
void showPasswordScreen() {

 lcd.clear();

 lcd.setCursor(0, 0);
 lcd.print("Enter Password:");

 lcd.setCursor(0, 1);
}
v/*
 ScienceUtsav RoboScientist Project
 Password Door Lock with LCD

 Board: Arduino UNO
 LCD I2C:
 SDA = A4
 SCL = A5
 Address = 0x27
*/

#include <Keypad.h>
#include <Servo.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ---------------- LCD ----------------
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ---------------- SERVO ----------------
Servo door;

const byte SERVO_PIN = 2;
const byte BUZZER = 5;
```

```
// ---------------- PASSWORD ----------------
const String PASSWORD = "2468";
String entered = "";

byte wrongAttempts = 0;
bool permanentlyLocked = false;

// ---------------- KEYPAD ----------------
const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
 {'1','2','3','A'},
 {'4','5','6','B'},
 {'7','8','9','C'},
 {'*','0','#','D'}
};

byte rowPins[ROWS] = {6, 7, 8, 9};
byte colPins[COLS] = {10, 11, 12, 13};

Keypad keypad = Keypad(
 makeKeymap(keys),
 rowPins,
 colPins,
 ROWS,
 COLS
);


// ================================================
// SETUP
// ================================================
void setup() {

 pinMode(BUZZER, OUTPUT);

 door.attach(SERVO_PIN);
 door.write(0);

 Serial.begin(9600);

 // LCD start
 lcd.init();
 lcd.backlight();

 lcd.setCursor(0, 0);
 lcd.print("PASSWORD LOCK");

 lcd.setCursor(0, 1);
 lcd.print("Enter Password");
```

```
 delay(2000);

 showPasswordScreen();
}


// ================================================
// LOOP
// ================================================
void loop() {

 // Permanent lock after 3 wrong attempts
 if (permanentlyLocked) {

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("SYSTEM LOCKED");

  lcd.setCursor(0, 1);
  lcd.print("RESET ARDUINO");

  tone(BUZZER, 1200, 200);

  delay(1000);

  return;
 }


 char key = keypad.getKey();

 if (!key)
  return;


 // ---------------------------------
 // CLEAR PASSWORD
 // ---------------------------------
 if (key == '*') {

  entered = "";

  showPasswordScreen();
 }


 // ---------------------------------
 // CHECK PASSWORD
 // ---------------------------------
 else if (key == '#') {
```

```
if (entered == PASSWORD) {

  wrongAttempts = 0;

  Serial.println("Access granted");

  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("ACCESS GRANTED");

  lcd.setCursor(0, 1);
  lcd.print("Door Opening");

  tone(BUZZER, 1500, 150);

  door.write(90);

  delay(2500);

  door.write(0);

  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("DOOR CLOSED");

  delay(1000);

}

else {

  wrongAttempts++;

  Serial.print("Wrong attempt: ");
  Serial.println(wrongAttempts);

  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("WRONG PASSWORD");

  lcd.setCursor(0, 1);
  lcd.print("Attempt: ");
  lcd.print(wrongAttempts);
  lcd.print("/3");

  tone(BUZZER, 500, 500);

  delay(1500);
```

```
    // Lock after 3 wrong attempts
    if (wrongAttempts >= 3) {

      permanentlyLocked = true;

      Serial.println("LOCKED until Arduino reset");

      return;
    }
  }

  entered = "";

  showPasswordScreen();
 }


 // ---------------------------------
 // PASSWORD INPUT
 // ---------------------------------
 else {

  if (entered.length() < 12) {

    entered += key;

    lcd.setCursor(0, 1);

    // Show * instead of actual password
    for (byte i = 0; i < entered.length(); i++) {
      lcd.print("*");
    }
  }
 }
}


// ================================================
// PASSWORD SCREEN
// ================================================
void showPasswordScreen() {

 lcd.clear();

 lcd.setCursor(0, 0);
 lcd.print("Enter Password:");

 lcd.setCursor(0, 1);
}
```

PROJECT 12

Multi-User Access

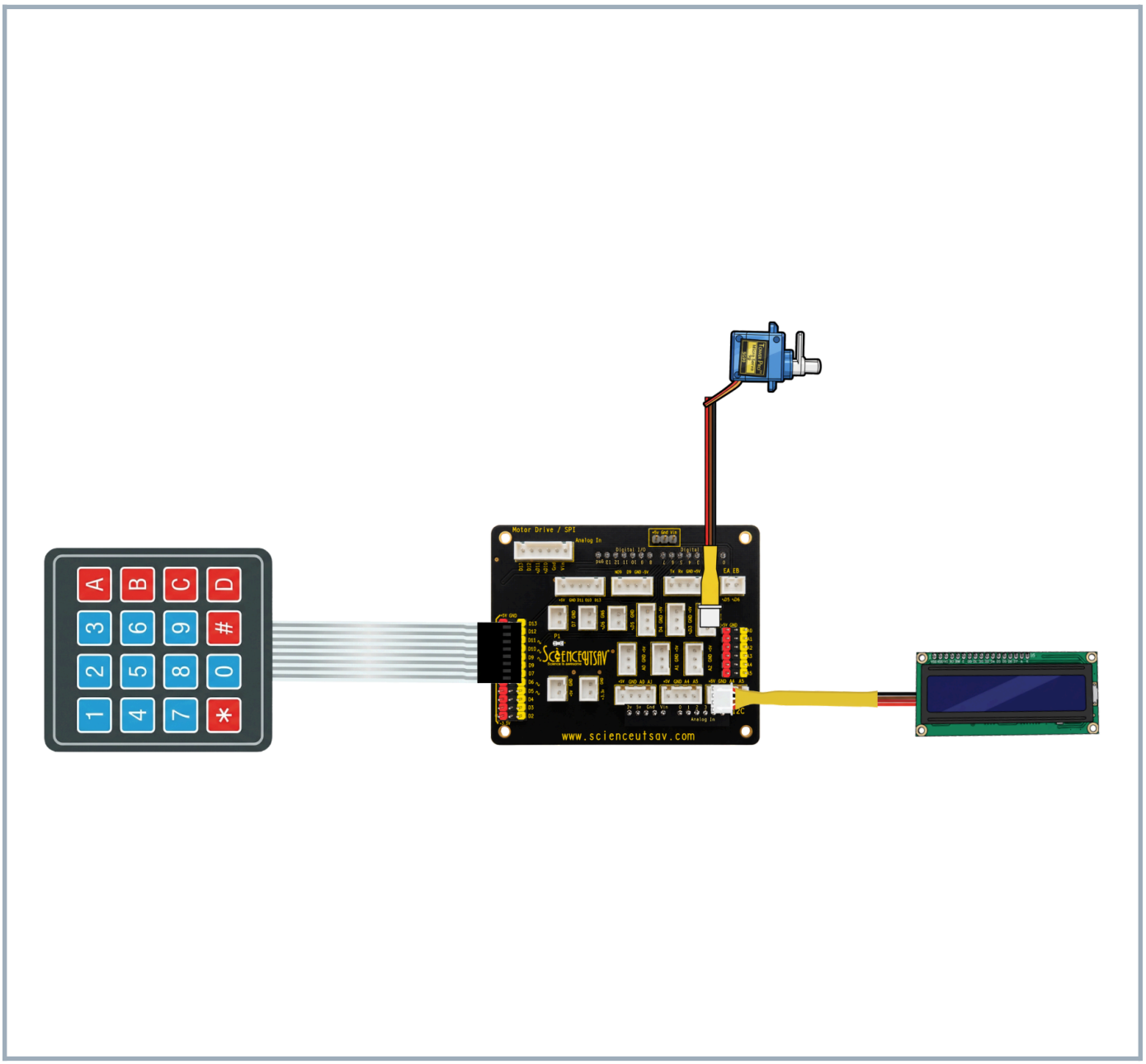
Allow several authorized passwords to open the same motorized gate and display the user name.

Board	Arduino UNO
Components	LCD 16x2 I2C, 4x4 keypad, Servo motor
Required libraries	Wire.h; LiquidCrystal_I2C.h; Keypad.h; Stepper.h
Sketch file	12_Multi_User_Access.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
4x4 keypad	R1	6	Keypad row wire.
4x4 keypad	R2	7	Keypad row wire.
4x4 keypad	R3	8	Keypad row wire.
4x4 keypad	R4	9	Keypad row wire.
4x4 keypad	C1	10	Keypad column wire.
4x4 keypad	C2	11	Keypad column wire.
4x4 keypad	C3	12	Keypad column wire.
4x4 keypad	C4	13	Keypad column wire.
Servo motor	Signal	2	Control signal from Arduino.
Servo motor	VCC	5V / external 5V	External supply is preferred for reliable operation.
Servo motor	GND	GND	Common ground with Arduino.

Project 12: Multi-User Access — Circuit Diagram



Project 12: Multi-User Access — Arduino Code

Sketch file: 12_Multi_User_Access.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
#include <Servo.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);

Servo gate;
const byte SERVO_PIN = 2;
const int GATE_CLOSED_ANGLE = 0;
const int GATE_OPEN_ANGLE = 90;

const String PASSWORDS[] = {"1111", "2222", "3580"};
const char* USERS[] = {"Admin", "Teacher", "Operator"};
const byte USER_COUNT = 3;
String entered = "";

const byte ROWS = 4, COLS = 4;
char keys[ROWS][COLS] = {
 {'1','2','3','A'}, {'4','5','6','B'},
 {'7','8','9','C'}, {'*','0','#','D'}
};
byte rowPins[ROWS] = {6,7,8,9};
byte colPins[COLS] = {10,11,12,13};
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

void prompt() {
 entered = "";
 lcd.clear(); lcd.print("Enter password");
 lcd.setCursor(0,1);
}
void setup() {
 gate.attach(SERVO_PIN);
 gate.write(GATE_CLOSED_ANGLE);
 lcd.init(); lcd.backlight();
 prompt();
}
```

```
void loop() {
  char key = keypad.getKey();
  if (!key) return;
  if (key == '*') prompt();
  else if (key == '#') {
    int user = -1;
    for (byte i = 0; i < USER_COUNT; i++) if (entered == PASSWORDS[i]) user = i;
    lcd.clear();
    if (user >= 0) {
      lcd.print("Welcome "); lcd.print(USERS[user]);
      gate.write(GATE_OPEN_ANGLE);
      delay(2500);
      gate.write(GATE_CLOSED_ANGLE);
    } else lcd.print("Access denied");
    delay(1400);
    prompt();
  } else if (entered.length() < 12) {
    entered += key;
    lcd.print('*');
  }
}
```

PROJECT 13

Tag Detector

Activate an LED and buzzer whenever an RFID tag is detected.

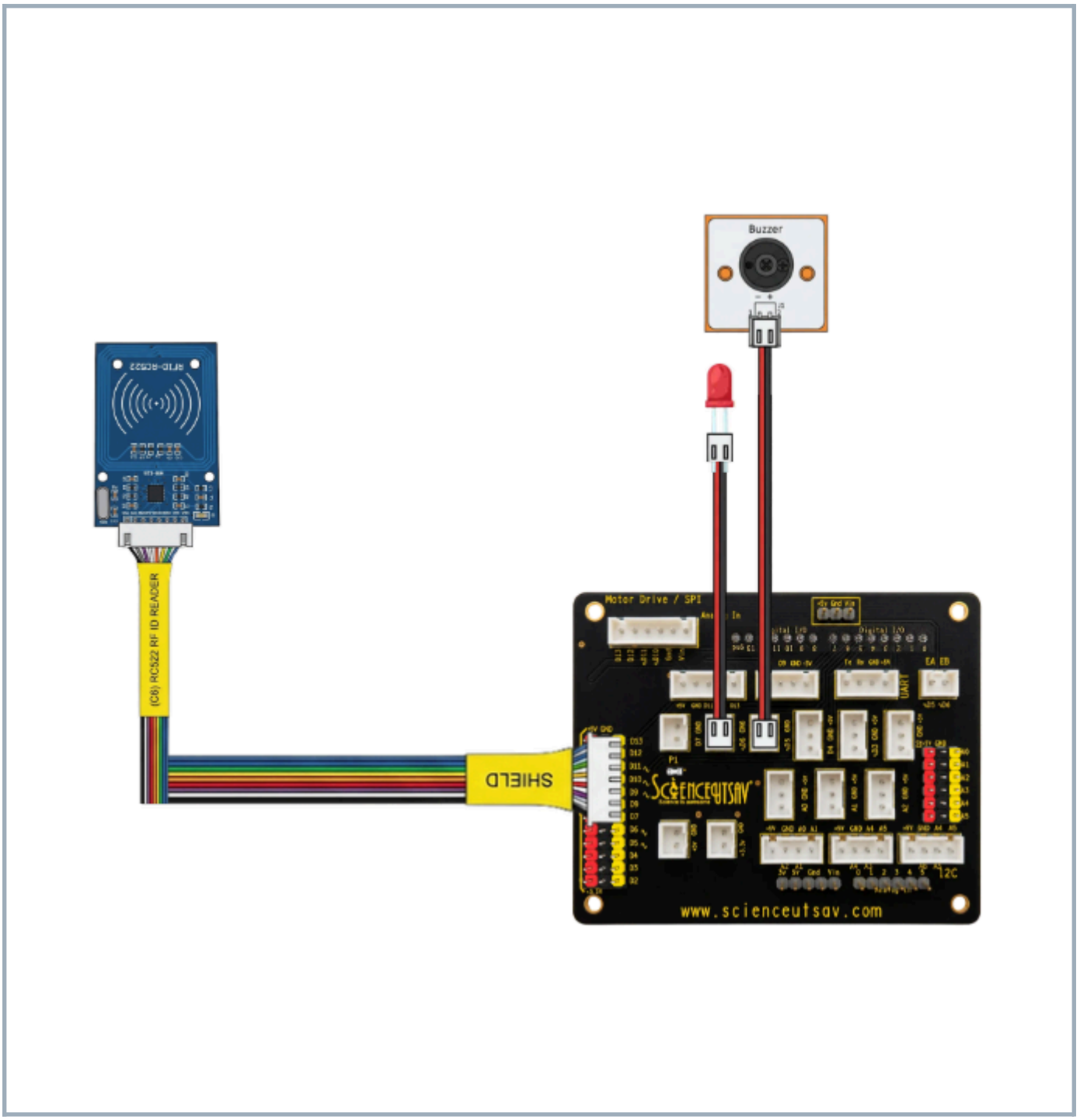
Board	Arduino UNO
Components	RC522 RFID reader, LED, buzzer
Required libraries	SPI.h; MFRC522.h
Sketch file	13_Tag_Detector.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
RC522 RFID reader	SDA / SS	D10	SPI chip-select line.
RC522 RFID reader	SCK	D13	SPI clock.
RC522 RFID reader	MOSI	D11	SPI data from UNO to RC522.
RC522 RFID reader	MISO	D12	SPI data from RC522 to UNO.
RC522 RFID reader	RST	D8	Reset pin.
RC522 RFID reader	3.3V	UNO 3.3V	Do not connect directly to a 5 V GPIO output.
RC522 RFID reader	GND	GND	Use the actual ground pin. D9 LOW is only an optional shield arrangement.
LED Pin	Anode	6	Use a 220–330 Ω resistor; cathode to GND.
Buzzer	Positive / signal	5	Negative terminal to GND.

Important: Use Project 18 to read each card UID, then replace the sample UIDs in this sketch.

Project 13: Tag Detector — Circuit Diagram



Project 13: Tag Detector — Arduino Code

Sketch file: 13_Tag_Detector.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <SPI.h>
#include <MFRC522.h>
MFRC522 rfid(10, 8);
const byte LED_PIN = 6;
const byte BUZZER = 5;

// IMPORTANT: Connect RC522 VCC to the UNO 3.3V supply.
// Do not connect RC522 VCC directly to a GPIO HIGH; UNO GPIO HIGH is about 5V.
// Set this true only if your shield has a real 3.3V regulator between D7 and RC522 VCC.
const bool SHIELD_HAS_3V3_REGULATOR = false;
const byte RFID_POWER_PIN = 7;
const byte RFID_GROUND_PIN = 9;
void setupRfidPower() {
  if (SHIELD_HAS_3V3_REGULATOR) {
    pinMode(RFID_GROUND_PIN, OUTPUT); digitalWrite(RFID_GROUND_PIN, LOW);
    pinMode(RFID_POWER_PIN, OUTPUT); digitalWrite(RFID_POWER_PIN, HIGH);
    delay(100);
  }
}

void setup() {
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER, OUTPUT);
  Serial.begin(9600);
  setupRfidPower();
  SPI.begin();
  rfid.PCD_Init();
  Serial.println("Tap any RFID tag");
}
void loop() {
  if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) return;
  Serial.println("Tag detected");
  digitalWrite(LED_PIN, HIGH);
  tone(BUZZER, 1800, 180);
  delay(500);
  digitalWrite(LED_PIN, LOW);
  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();
}
```

PROJECT 14

Person Identifier

Display the name assigned to the scanned RFID tag.

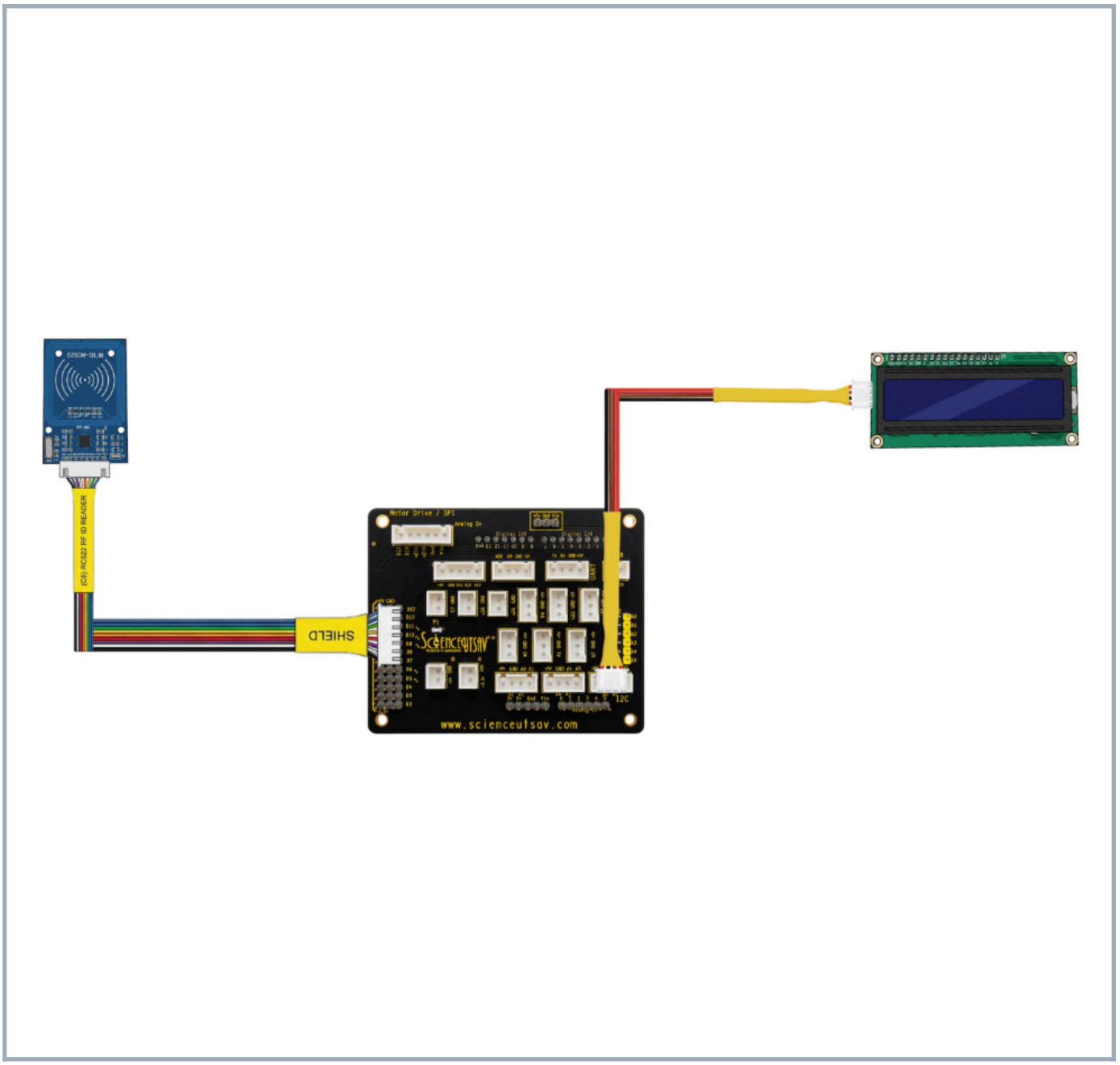
Board	Arduino UNO
Components	RC522 RFID reader, LCD 16x2 I2C
Required libraries	Wire.h; LiquidCrystal_I2C.h; SPI.h; MFRC522.h
Sketch file	14_Person_Identifier.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
RC522 RFID reader	SDA / SS	D10	SPI chip-select line.
RC522 RFID reader	SCK	D13	SPI clock.
RC522 RFID reader	MOSI	D11	SPI data from UNO to RC522.
RC522 RFID reader	MISO	D12	SPI data from RC522 to UNO.
RC522 RFID reader	RST	D8	Reset pin.
RC522 RFID reader	3.3V	UNO 3.3V	Do not connect directly to a 5 V GPIO output.
RC522 RFID reader	GND	GND	Use the actual ground pin. D9 LOW is only an optional shield arrangement.

Important: Use Project 18 to read each card UID, then replace the sample UIDs in this sketch.

Project 14: Person Identifier — Circuit Diagram



Project 14: Person Identifier — Arduino Code

Sketch file: 14_Person_Identifier.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SPI.h>
#include <MFRC522.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
MFRC522 rfid(10, 8);

// IMPORTANT: Connect RC522 VCC to the UNO 3.3V supply.
// Do not connect RC522 VCC directly to a GPIO HIGH; UNO GPIO HIGH is about 5V.
// Set this true only if your shield has a real 3.3V regulator between D7 and RC522 VCC.
const bool SHIELD_HAS_3V3_REGULATOR = false;
const byte RFID_POWER_PIN = 7;
const byte RFID_GROUND_PIN = 9;
void setupRfidPower() {
 if (SHIELD_HAS_3V3_REGULATOR) {
  pinMode(RFID_GROUND_PIN, OUTPUT); digitalWrite(RFID_GROUND_PIN, LOW);
  pinMode(RFID_POWER_PIN, OUTPUT); digitalWrite(RFID_POWER_PIN, HIGH);
  delay(100);
 }
}

String uidText() {
 String s = "";
 for (byte i = 0; i < rfid.uid.size; i++) {
  if (rfid.uid.uidByte[i] < 0x10) s += "0";
  s += String(rfid.uid.uidByte[i], HEX);
 }
 s.toUpperCase();
 return s;
}
const char* identify(const String &uid) {
 // Replace these sample UIDs with values printed by Project 18.
 if (uid == "A1B2C3D4") return "Aarav";
 if (uid == "11223344") return "Diya";
 if (uid == "55667788") return "Teacher";
 return "Unknown Person";
}
```

```
void setup() {
  Serial.begin(9600);
  lcd.init(); lcd.backlight();
  lcd.print("Tap your card");
  setupRfidPower(); SPI.begin(); rfid.PCD_Init();
}
void loop() {
  if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) return;
  String uid = uidText();
  const char* name = identify(uid);
  Serial.print(uid); Serial.print(","); Serial.println(name);
  lcd.clear(); lcd.print("Person:");
  lcd.setCursor(0,1); lcd.print(name);
  rfid.PICC_HaltA(); rfid.PCD_StopCrypto1();
  delay(1500);
  lcd.clear(); lcd.print("Tap your card");
}
```

PROJECT 15

RFID Door Unlock

Unlock a model door only for permitted RFID tags.

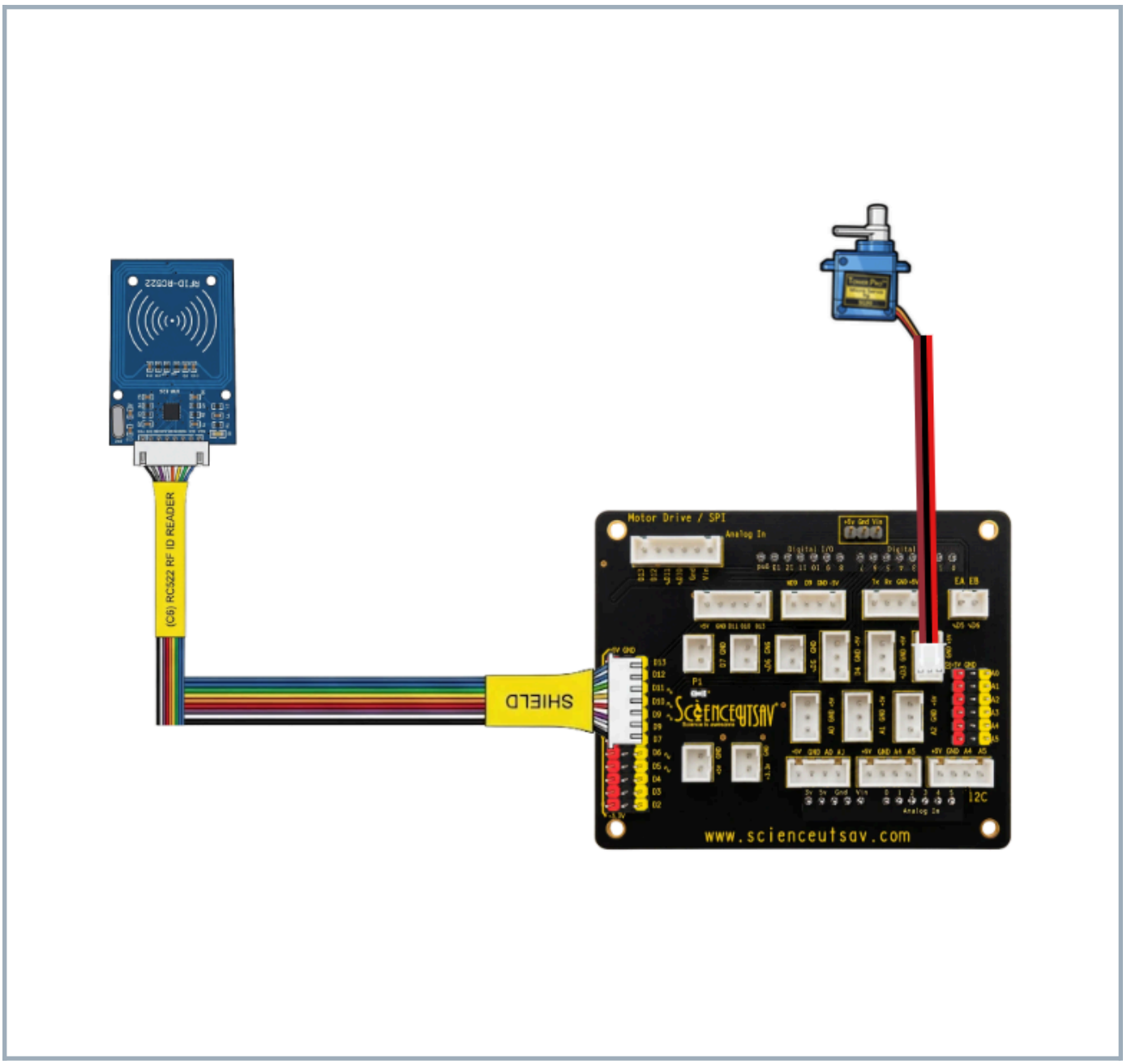
Board	Arduino UNO
Components	RC522 RFID reader, servo motor, buzzer
Required libraries	SPI.h; MFRC522.h; Servo.h
Sketch file	15_RFID_Door_Unlock.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
RC522 RFID reader	SDA / SS	D10	SPI chip-select line.
RC522 RFID reader	SCK	D13	SPI clock.
RC522 RFID reader	MOSI	D11	SPI data from UNO to RC522.
RC522 RFID reader	MISO	D12	SPI data from RC522 to UNO.
RC522 RFID reader	RST	D8	Reset pin.
RC522 RFID reader	3.3V	UNO 3.3V	Do not connect directly to a 5 V GPIO output.
RC522 RFID reader	GND	GND	Use the actual ground pin. D9 LOW is only an optional shield arrangement.
Servo motor	Signal	2	Control signal from Arduino.
Servo motor	VCC	5V / external 5V	External supply is preferred for reliable operation.
Servo motor	GND	GND	Common ground with Arduino.

Important: Use Project 18 to read each card UID, then replace the sample UIDs in this sketch.

Project 15: RFID Door Unlock — Circuit Diagram



Project 15: RFID Door Unlock — Arduino Code

Sketch file: 15_RFID_Door_Unlock.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <SPI.h>
#include <MFRC522.h>
#include <Servo.h>
MFRC522 rfid(10, 8);
Servo door;
const byte SERVO_PIN = 2;

// IMPORTANT: Connect RC522 VCC to the UNO 3.3V supply.
// Do not connect RC522 VCC directly to a GPIO HIGH; UNO GPIO HIGH is about 5V.
// Set this true only if your shield has a real 3.3V regulator between D7 and RC522 VCC.
const bool SHIELD_HAS_3V3_REGULATOR = false;
const byte RFID_POWER_PIN = 7;
const byte RFID_GROUND_PIN = 9;
void setupRfidPower() {
 if (SHIELD_HAS_3V3_REGULATOR) {
   pinMode(RFID_GROUND_PIN, OUTPUT); digitalWrite(RFID_GROUND_PIN, LOW);
   pinMode(RFID_POWER_PIN, OUTPUT); digitalWrite(RFID_POWER_PIN, HIGH);
   delay(100);
 }
}

String uidText() {
 String s="";
 for(byte i=0;i<rfid.uid.size;i++){ if(rfid.uid.uidByte[i]<16)s+="0"; s+=String(rfid.uid.uidByte[i],HEX); }
 s.toUpperCase(); return s;
}
bool authorized(const String &uid) {
 return uid=="A1B2C3D4" || uid=="11223344"; // Replace with your card UIDs.
}
void setup() {
 door.attach(SERVO_PIN); door.write(0);
 Serial.begin(9600);
 setupRfidPower(); SPI.begin(); rfid.PCD_Init();
}
```

```
void loop() {
  if(!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) return;
  String uid=uidText();
  if(authorized(uid)){
    Serial.println("Access granted");
    door.write(90); delay(3000); door.write(0);
  } else {
    Serial.println("Access denied");
  }
  rfid.PICC_HaltA(); rfid.PCD_StopCrypto1();
  delay(500);
}
```

PROJECT 16

Smart Equipment Log

Grant access only when equipment is free; tap in and tap out to track usage.

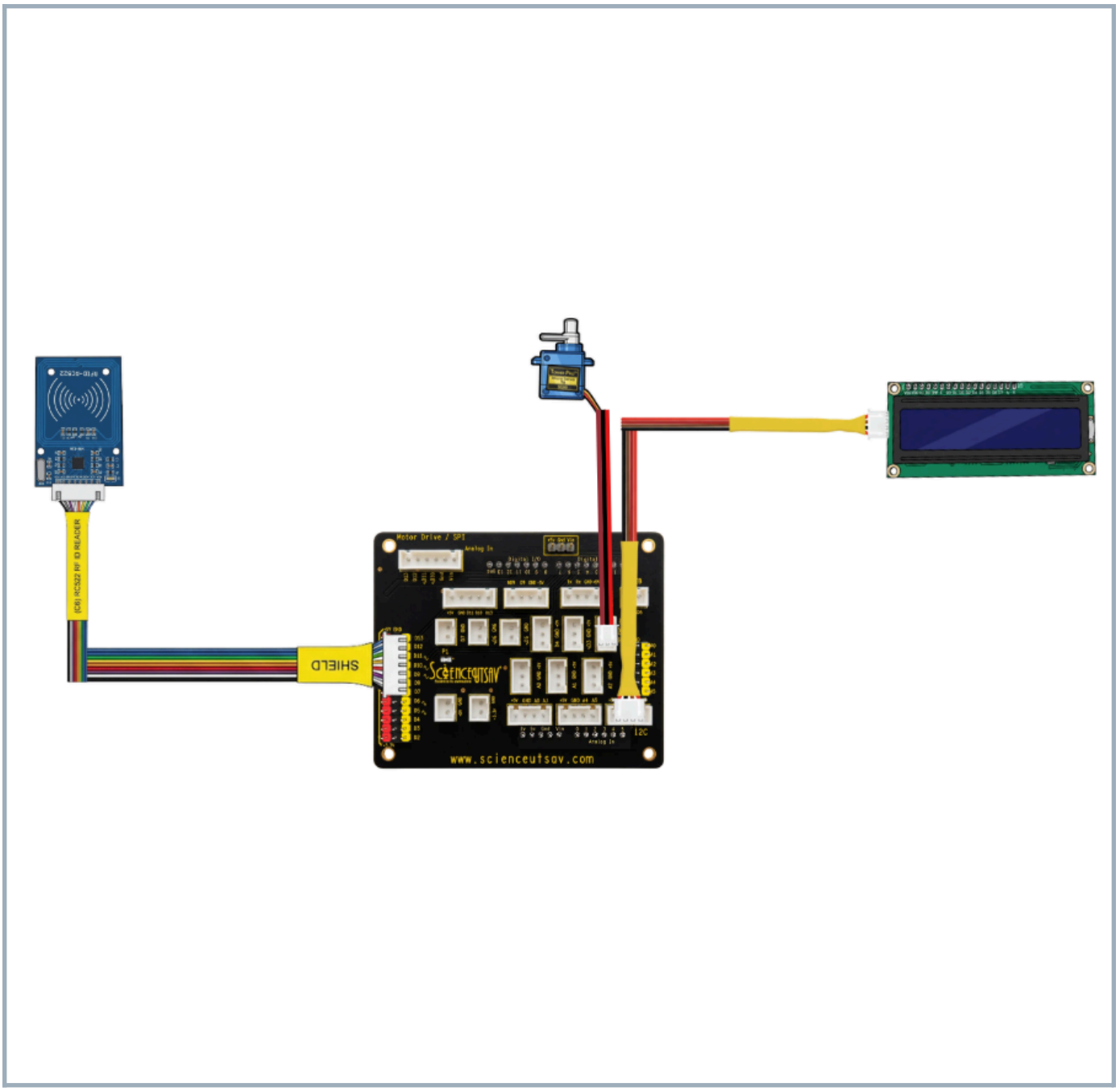
Board	Arduino UNO
Components	RC522 RFID reader, LCD 16x2 I2C, servo motor
Required libraries	Wire.h; LiquidCrystal_I2C.h; SPI.h; MFRC522.h; Servo.h
Sketch file	16_Smart_Equipment_Log.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
RC522 RFID reader	SDA / SS	D10	SPI chip-select line.
RC522 RFID reader	SCK	D13	SPI clock.
RC522 RFID reader	MOSI	D11	SPI data from UNO to RC522.
RC522 RFID reader	MISO	D12	SPI data from RC522 to UNO.
RC522 RFID reader	RST	D8	Reset pin.
RC522 RFID reader	3.3V	UNO 3.3V	Do not connect directly to a 5 V GPIO output.
RC522 RFID reader	GND	GND	Use the actual ground pin. D9 LOW is only an optional shield arrangement.
Servo motor	Signal	2	Control signal from Arduino.
Servo motor	VCC	5V / external 5V	External supply is preferred for reliable operation.
Servo motor	GND	GND	Common ground with Arduino.

Important: Use Project 18 to read each card UID, then replace the sample UIDs in this sketch.

Project 16: Smart Equipment Log — Circuit Diagram



Project 16: Smart Equipment Log — Arduino Code

Sketch file: 16_Smart_Equipment_Log.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SPI.h>
#include <MFRC522.h>
#include <Servo.h>
LiquidCrystal_I2C lcd(0x27,16,2);
MFRC522 rfid(10,8);
Servo equipmentLock;
const byte SERVO_PIN=2;
bool equipmentInUse=false;
String activeUid="";

// IMPORTANT: Connect RC522 VCC to the UNO 3.3V supply.
// Do not connect RC522 VCC directly to a GPIO HIGH; UNO GPIO HIGH is about 5V.
// Set this true only if your shield has a real 3.3V regulator between D7 and RC522 VCC.
const bool SHIELD_HAS_3V3_REGULATOR = false;
const byte RFID_POWER_PIN = 7;
const byte RFID_GROUND_PIN = 9;
void setupRfidPower() {
 if (SHIELD_HAS_3V3_REGULATOR) {
   pinMode(RFID_GROUND_PIN, OUTPUT); digitalWrite(RFID_GROUND_PIN, LOW);
   pinMode(RFID_POWER_PIN, OUTPUT); digitalWrite(RFID_POWER_PIN, HIGH);
   delay(100);
 }
}

String uidText(){
 String s="";
 for(byte i=0;i<rfid.uid.size;i++){if(rfid.uid.uidByte[i]<16)s+="0";s+=String(rfid.uid.uidByte[i],HEX);}
 s.toUpperCase(); return s;
}
void showReady(){
 lcd.clear();
 lcd.print(equipmentInUse ? "IN USE - tap out" : "FREE - tap in");
 if(equipmentInUse){lcd.setCursor(0,1);lcd.print(activeUid.substring(0,8));}
}
void setup(){
 Serial.begin(9600);
 lcd.init();lcd.backlight();
 equipmentLock.attach(SERVO_PIN);equipmentLock.write(0);
 setupRfidPower();SPI.begin();rfid.PCD_Init();
```

```
 showReady();
}
void loop(){
 if(!rfid.PICC_IsNewCardPresent()||!rfid.PICC_ReadCardSerial())return;
 String uid=uidText();
 if(!equipmentInUse){
  equipmentInUse=true;activeUid=uid;
  equipmentLock.write(90);
  Serial.print("TAP_IN,");Serial.print(uid);Serial.print(",");Serial.println(millis());
 }else if(uid==activeUid){
  equipmentInUse=false;
  equipmentLock.write(0);
  Serial.print("TAP_OUT,");Serial.print(uid);Serial.print(",");Serial.println(millis());
  activeUid="";
 }else{
  lcd.clear();lcd.print("Equipment Busy");
  lcd.setCursor(0,1);lcd.print("Access denied");
  delay(1500);
 }
 rfid.PICC_HaltA();rfid.PCD_StopCrypto1();
 showReady();delay(500);
}
```

PROJECT 17

Attendance Gate

Record an attendance entry whenever an employee taps an RFID card.

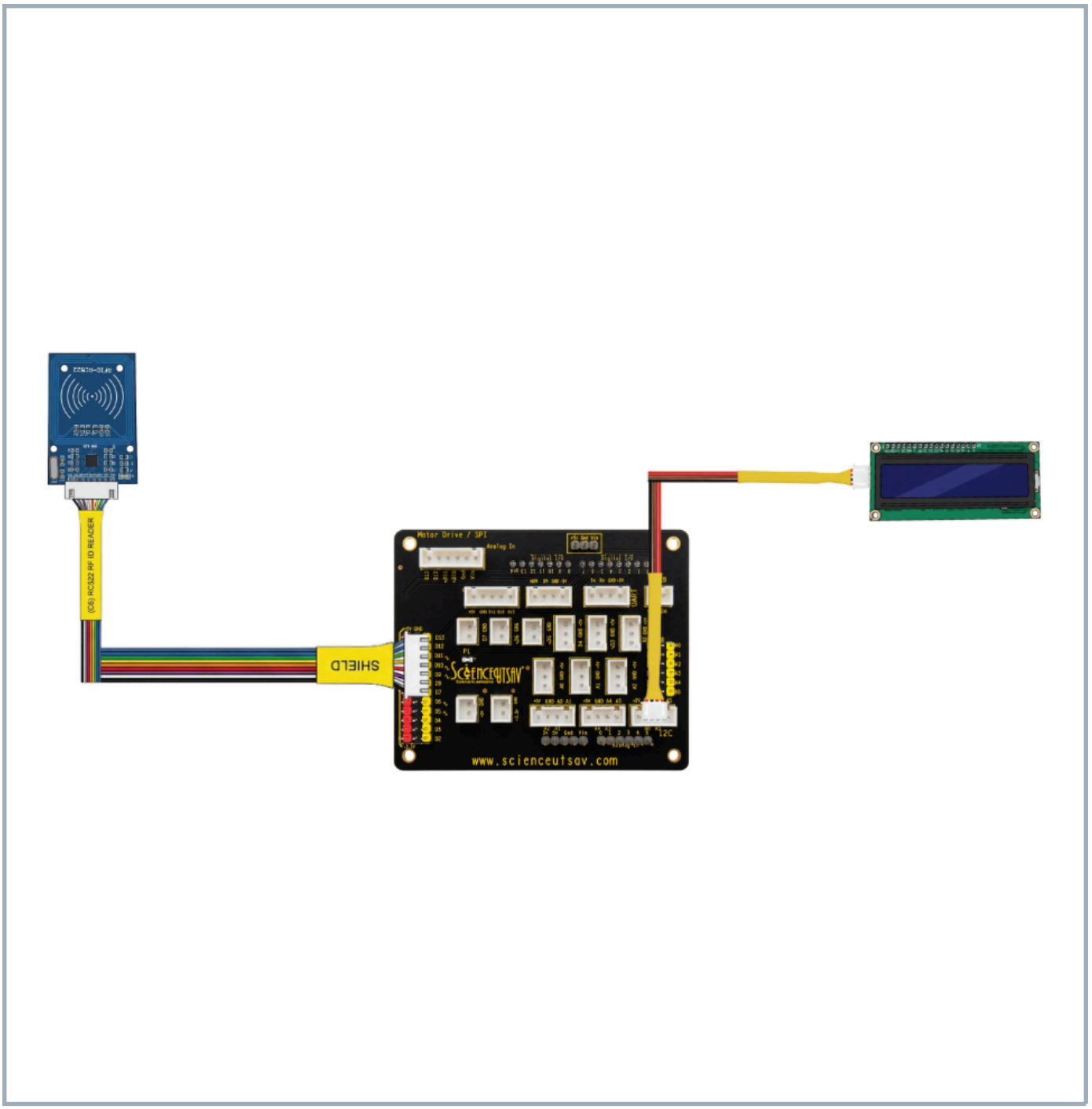
Board	Arduino UNO
Components	RC522 RFID reader, LCD 16x2 I2C
Required libraries	Wire.h; LiquidCrystal_I2C.h; SPI.h; MFRC522.h
Sketch file	17_Attendance_Gate.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
RC522 RFID reader	SDA / SS	D10	SPI chip-select line.
RC522 RFID reader	SCK	D13	SPI clock.
RC522 RFID reader	MOSI	D11	SPI data from UNO to RC522.
RC522 RFID reader	MISO	D12	SPI data from RC522 to UNO.
RC522 RFID reader	RST	D8	Reset pin.
RC522 RFID reader	3.3V	UNO 3.3V	Do not connect directly to a 5 V GPIO output.
RC522 RFID reader	GND	GND	Use the actual ground pin. D9 LOW is only an optional shield arrangement.

Important: Attendance is printed as CSV-style text to the Serial Monitor and the count is held in RAM. Add RTC/SD/cloud storage for permanent dated records.

Project 17: Attendance Gate — Circuit Diagram



Project 17: Attendance Gate — Arduino Code

Sketch file: 17_Attendance_Gate.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SPI.h>
#include <MFRC522.h>
LiquidCrystal_I2C lcd(0x27,16,2);
MFRC522 rfid(10,8);
unsigned int attendanceCount=0;
String lastUid="";
unsigned long lastScan=0;

// IMPORTANT: Connect RC522 VCC to the UNO 3.3V supply.
// Do not connect RC522 VCC directly to a GPIO HIGH; UNO GPIO HIGH is about 5V.
// Set this true only if your shield has a real 3.3V regulator between D7 and RC522 VCC.
const bool SHIELD_HAS_3V3_REGULATOR = false;
const byte RFID_POWER_PIN = 7;
const byte RFID_GROUND_PIN = 9;
void setupRfidPower() {
 if (SHIELD_HAS_3V3_REGULATOR) {
   pinMode(RFID_GROUND_PIN, OUTPUT); digitalWrite(RFID_GROUND_PIN, LOW);
   pinMode(RFID_POWER_PIN, OUTPUT); digitalWrite(RFID_POWER_PIN, HIGH);
   delay(100);
 }
}

String uidText(){
 String s="";
 for(byte i=0;i<rfid.uid.size;i++){if(rfid.uid.uidByte[i]<16)s+="0";s+=String(rfid.uid.uidByte[i],HEX);}
 s.toUpperCase();return s;
}
const char* nameFor(const String& uid){
 if(uid=="A1B2C3D4")return "Aarav";
 if(uid=="11223344")return "Diya";
 return "Unknown";
}
void setup(){
 Serial.begin(9600);
 Serial.println("count,uid,name,milliseconds");
 lcd.init();lcd.backlight();lcd.print("Tap attendance");
 setupRfidPower();SPI.begin();rfid.PCD_Init();
}
```

```
void loop(){
 if(!rfid.PICC_IsNewCardPresent()||!rfid.PICC_ReadCardSerial())return;
 String uid=uidText();
 if(uid==lastUid && millis()-lastScan<10000){
  lcd.clear();lcd.print("Already marked");
 }else{
  attendanceCount++;lastUid=uid;lastScan=millis();
  const char* name=nameFor(uid);
  Serial.print(attendanceCount);Serial.print(",");
  Serial.print(uid);Serial.print(",");Serial.print(name);Serial.print(",");
  Serial.println(millis());
  lcd.clear();lcd.print("Welcome ");lcd.print(name);
  lcd.setCursor(0,1);lcd.print("Count: ");lcd.print(attendanceCount);
 }
 rfid.PICC_HaltA();rfid.PCD_StopCrypto1();
 delay(1500);lcd.clear();lcd.print("Tap attendance");
}
```

PROJECT 18

Multi-Safety Locker

Require both an authorized RFID card and the correct keypad password to unlock the locker.

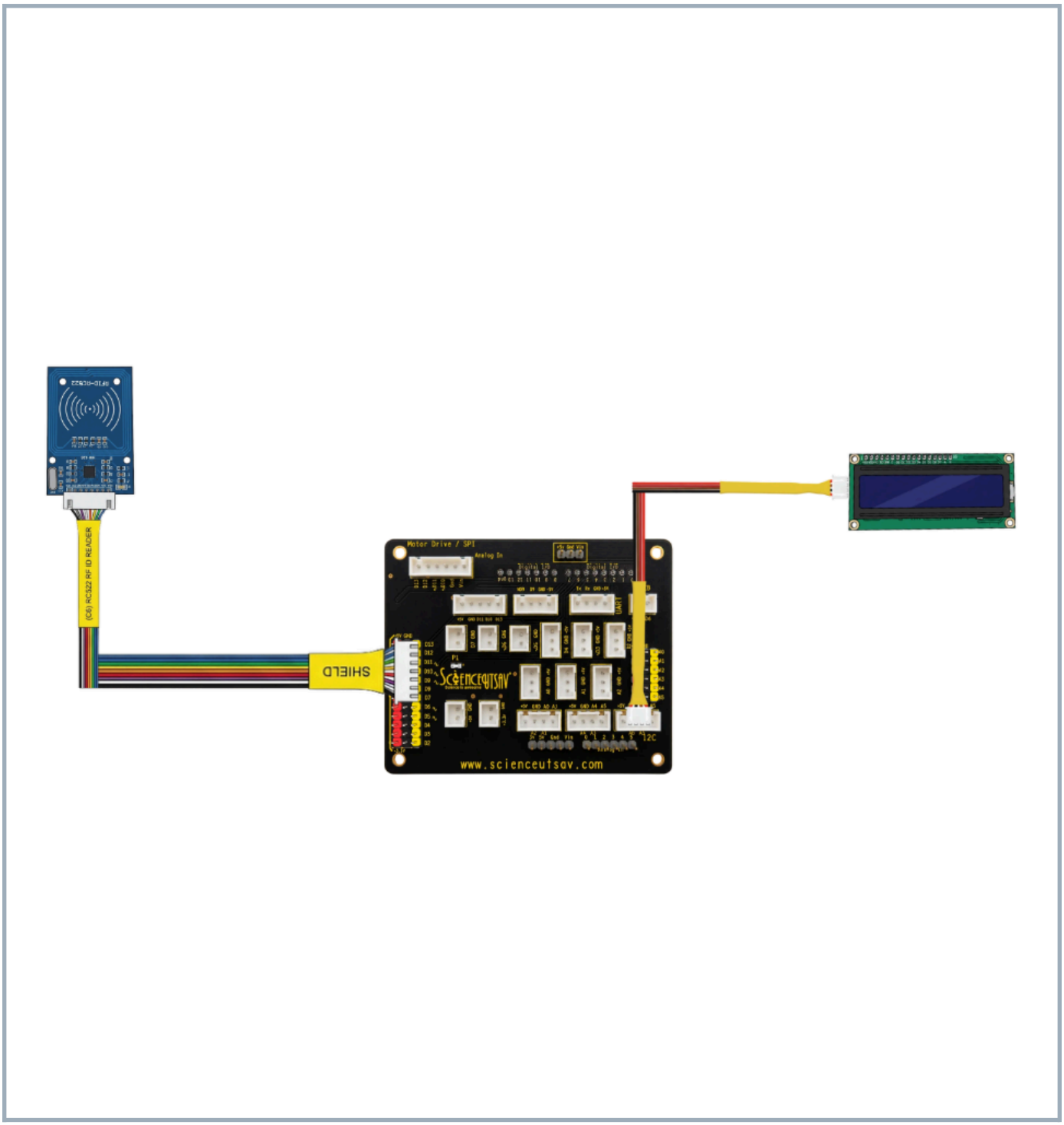
Board	Arduino UNO
Components	RC522 RFID reader, LCD 16x2 I2C, 4x4 keypad, servo motor
Required libraries	Wire.h; LiquidCrystal_I2C.h; SPI.h; MFRC522.h; Keypad.h; Servo.h
Sketch file	18_Multi_Safety_Locker.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
RC522 RFID reader	SDA / SS	D10	SPI chip-select line.
RC522 RFID reader	SCK	D13	SPI clock.
RC522 RFID reader	MOSI	D11	SPI data from UNO to RC522.
RC522 RFID reader	MISO	D12	SPI data from RC522 to UNO.
RC522 RFID reader	RST	D8	Reset pin.
RC522 RFID reader	3.3V	UNO 3.3V	Do not connect directly to a 5 V GPIO output.
RC522 RFID reader	GND	GND	Use the actual ground pin. D9 LOW is only an optional shield arrangement.

Important: Replace the sample RFID UIDs with your own card UIDs from Project 18. Add further UIDs to the authorized-card list as required.

Project 18: Multi-Safety Locker — Circuit Diagram



Project 18: Multi-Safety Locker — Arduino Code

Sketch file: 18_Multi_Safety_Locker.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SPI.h>
#include <MFRC522.h>
#include <Servo.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
MFRC522 rfid(10, 8);
Servo locker;

const byte SERVO_PIN = 6;

// ONLY 3 RFID TAGS
const byte TAG_COUNT = 3;

// Replace these with your real RFID UIDs
String tagUIDs[TAG_COUNT] = {
 "0C24A605",   // Tag 0
 "71753918",   // Tag 1
 "FA9AA505"    // Tag 2
};

// Default 4-tag password
// Scan Tag 0 -> Tag 1 -> Tag 2 -> Tag 0
byte code[4] = {0, 1, 2, 0};

byte attempt[4];
byte attemptLen = 0;

String uidText() {
 String s = "";

 for (byte i = 0; i < rfid.uid.size; i++) {
  if (rfid.uid.uidByte[i] < 0x10) {
   s += "0";
  }

  s += String(rfid.uid.uidByte[i], HEX);
 }

 s.toUpperCase();
```

```
  return s;
}

// Returns 0, 1 or 2 depending on RFID tag
int tagDigit(const String &uid) {

  for (byte i = 0; i < TAG_COUNT; i++) {
    if (uid == tagUIDs[i]) {
      return i;
    }
  }

  return -1;
}

void showGuessPrompt() {
  attemptLen = 0;

  lcd.clear();
  lcd.print("Scan 4 tags");

  lcd.setCursor(0, 1);
  lcd.print("Code: ");
}

void unlockDoor() {

  lcd.clear();
  lcd.print("Unlocked!");

  locker.write(90);

  delay(4000);

  locker.write(0);

  showGuessPrompt();
}

void retryPrompt() {

  lcd.clear();
  lcd.print("Wrong Code!");

  delay(2000);

  showGuessPrompt();
}

void setup() {

  Serial.begin(9600);

  locker.attach(SERVO_PIN);
  locker.write(0);
```

```
 lcd.init();
 lcd.backlight();

 SPI.begin();
 rfid.PCD_Init();

 Serial.println("RFID ready");

 showGuessPrompt();
}

void loop() {

 if (!rfid.PICC_IsNewCardPresent()) {
  return;
 }

 if (!rfid.PICC_ReadCardSerial()) {
  return;
 }

 String uid = uidText();

 Serial.print("UID: ");
 Serial.println(uid);

 int d = tagDigit(uid);

 rfid.PICC_HaltA();
 rfid.PCD_StopCrypto1();

 if (d < 0) {

  lcd.clear();
  lcd.print("Unknown Tag");

  delay(1000);

  showGuessPrompt();
  return;
 }

 attempt[attemptLen] = d;
 attemptLen++;

 lcd.setCursor(6 + attemptLen - 1, 1);
 lcd.print(d);

 if (attemptLen == 4) {

  bool correct = true;

  for (byte i = 0; i < 4; i++) {
   if (attempt[i] != code[i]) {
    correct = false;
   }
```

```
  }

  if (correct) {
    unlockDoor();
  }
  else {
    retryPrompt();
  }
 }

 delay(500);
}
```

PROJECT 19

Motion LED

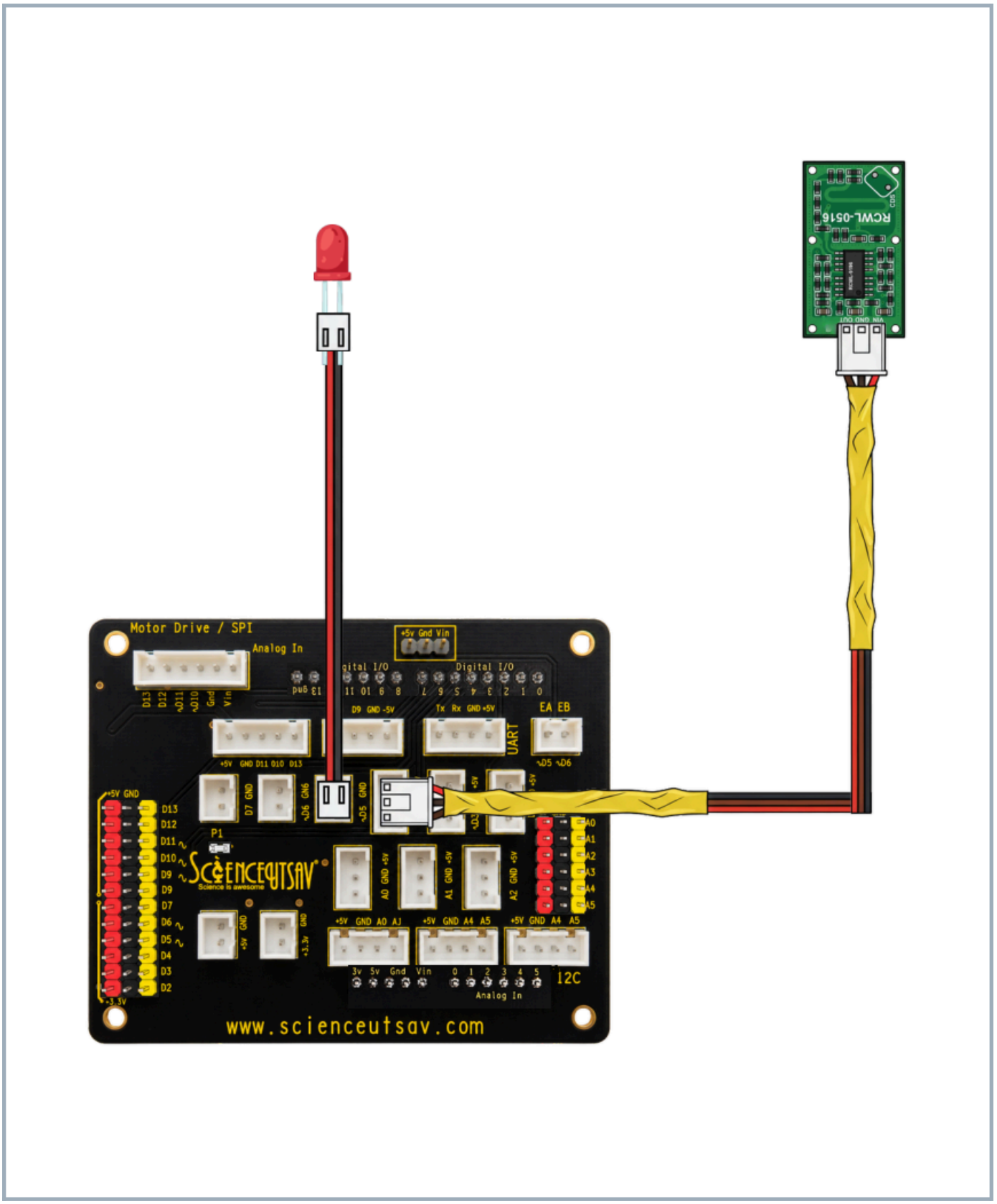
Turn on an LED when motion is detected by the microwave radar sensor.

Board	Arduino UNO
Components	Microwave radar sensor, LED
Required libraries	Built-in Arduino functions only
Sketch file	19_Motion_LED.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
Microwave radar sensor	OUT	4	Digital motion output.
Microwave radar sensor	VCC	5V	Check the exact radar module specification.
Microwave radar sensor	GND	GND	Common ground.
LED Pin	Anode	5	Use a 220–330 Ω resistor; cathode to GND.

Project 19: Motion LED — Circuit Diagram



Project 19: Motion LED — Arduino Code

Sketch file: 19_Motion_LED.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

const byte RADAR_PIN=4;
const byte LED_PIN=5;
void setup(){pinMode(RADAR_PIN,INPUT);pinMode(LED_PIN,OUTPUT);}
void loop(){digitalWrite(LED_PIN,digitalRead(RADAR_PIN));}
```

PROJECT 20

Intrusion Lock

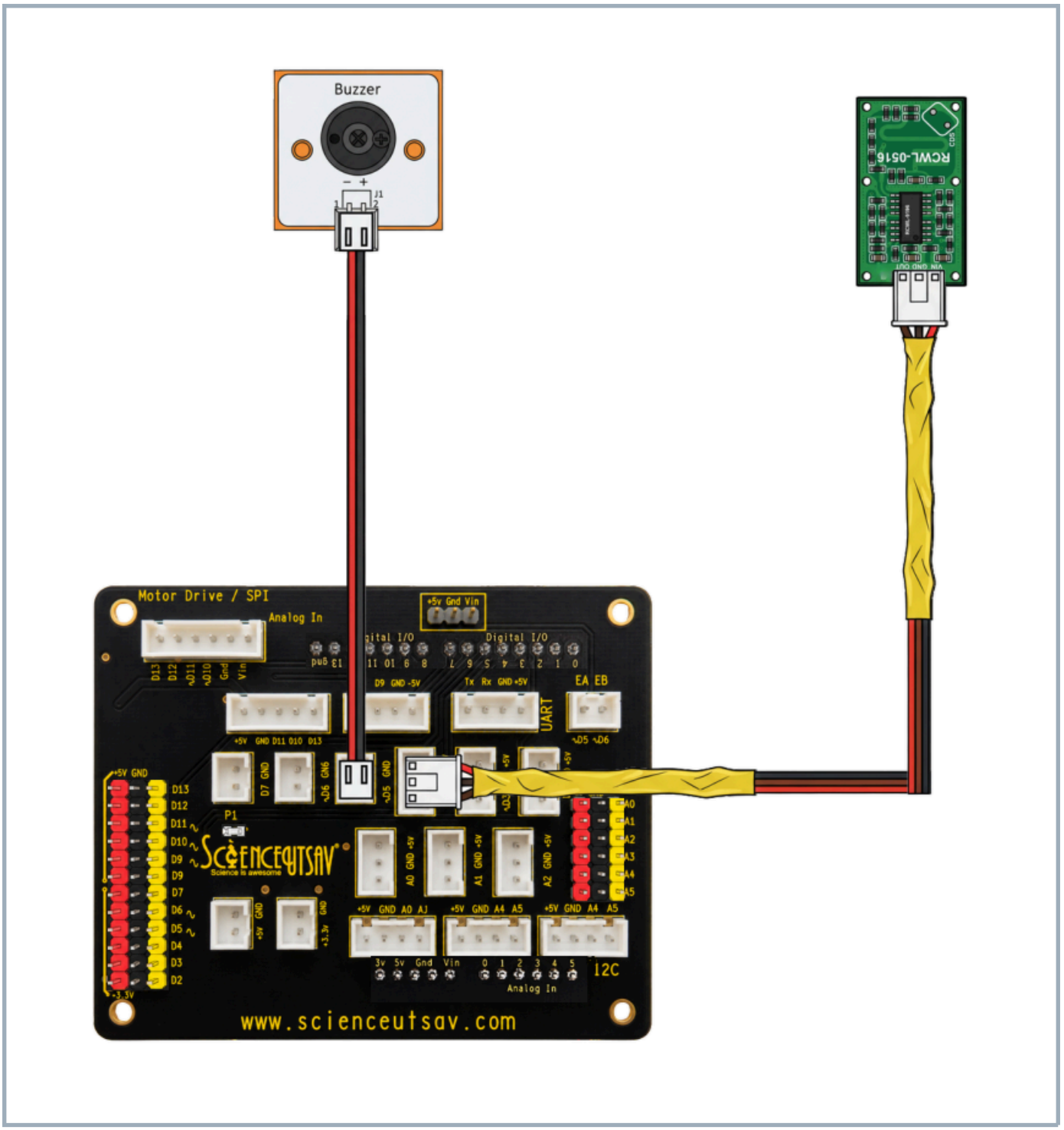
Lock the door and trigger an alarm when motion is detected.

Board	Arduino UNO
Components	Microwave radar sensor, servo motor, buzzer
Required libraries	Servo.h
Sketch file	20_Intrusion_Lock.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
Servo motor	Signal	3	Control signal from Arduino.
Servo motor	VCC	5V / external 5V	External supply is preferred for reliable operation.
Servo motor	GND	GND	Common ground with Arduino.
Microwave radar sensor	OUT	2	Digital motion output.
Microwave radar sensor	VCC	5V	Check the exact radar module specification.
Microwave radar sensor	GND	GND	Common ground.
Buzzer	Positive / signal	5	Negative terminal to GND.

Project 20: Intrusion Lock — Circuit Diagram



Project 20: Intrusion Lock — Arduino Code

Sketch file: 20_Intrusion_Lock.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

const byte RADAR_PIN=4;
const byte BUZZER=5;

enum SirenState { IDLE, SIREN_ON, SIREN_BREAK };
SirenState state = IDLE;
unsigned long stateStart = 0;

const unsigned long ON_DURATION = 3000;    // buzzer on phase
const unsigned long BREAK_DURATION = 2000; // forced silent phase
// ON_DURATION + BREAK_DURATION = 5000ms max cycle

const unsigned int SIREN_MIN_FREQ = 500;
const unsigned int SIREN_MAX_FREQ = 1200;
const unsigned long SIREN_SWEEP_MS = 800; // time for one up-or-down sweep

void updateSiren(unsigned long elapsed){
 unsigned long phase = elapsed % (SIREN_SWEEP_MS * 2);
 unsigned int freq;
 if(phase < SIREN_SWEEP_MS){
  // rising sweep
  freq = SIREN_MIN_FREQ + (long)(SIREN_MAX_FREQ - SIREN_MIN_FREQ) * phase / SIREN_SWEEP_MS;
 } else {
  // falling sweep
  unsigned long down = phase - SIREN_SWEEP_MS;
  freq = SIREN_MAX_FREQ - (long)(SIREN_MAX_FREQ - SIREN_MIN_FREQ) * down / SIREN_SWEEP_MS;
 }
 tone(BUZZER, freq);
}

void setup(){
 pinMode(RADAR_PIN,INPUT);
 pinMode(BUZZER,OUTPUT);
}
```

```
void loop(){
 bool motion = digitalRead(RADAR_PIN);
 unsigned long now = millis();

 switch(state){
  case IDLE:
   if(motion){
    state = SIREN_ON;
    stateStart = now;
   }
   break;

  case SIREN_ON:
   if(!motion){
    // motion stopped early, turn off immediately
    noTone(BUZZER);
    state = IDLE;
   } else if(now - stateStart >= ON_DURATION){
    // 3s elapsed, forced break regardless of continued motion
    noTone(BUZZER);
    state = SIREN_BREAK;
    stateStart = now;
   } else {
    updateSiren(now - stateStart);
   }
   break;

  case SIREN_BREAK:
   if(now - stateStart >= BREAK_DURATION){
    // 5s cycle complete, re-check motion
    if(motion){
     state = SIREN_ON;
     stateStart = now;
    } else {
     state = IDLE;
    }
   }
   break;
 }
}
```

PROJECT 21

Motion Lamp

Switch a lamp on automatically when motion is detected.

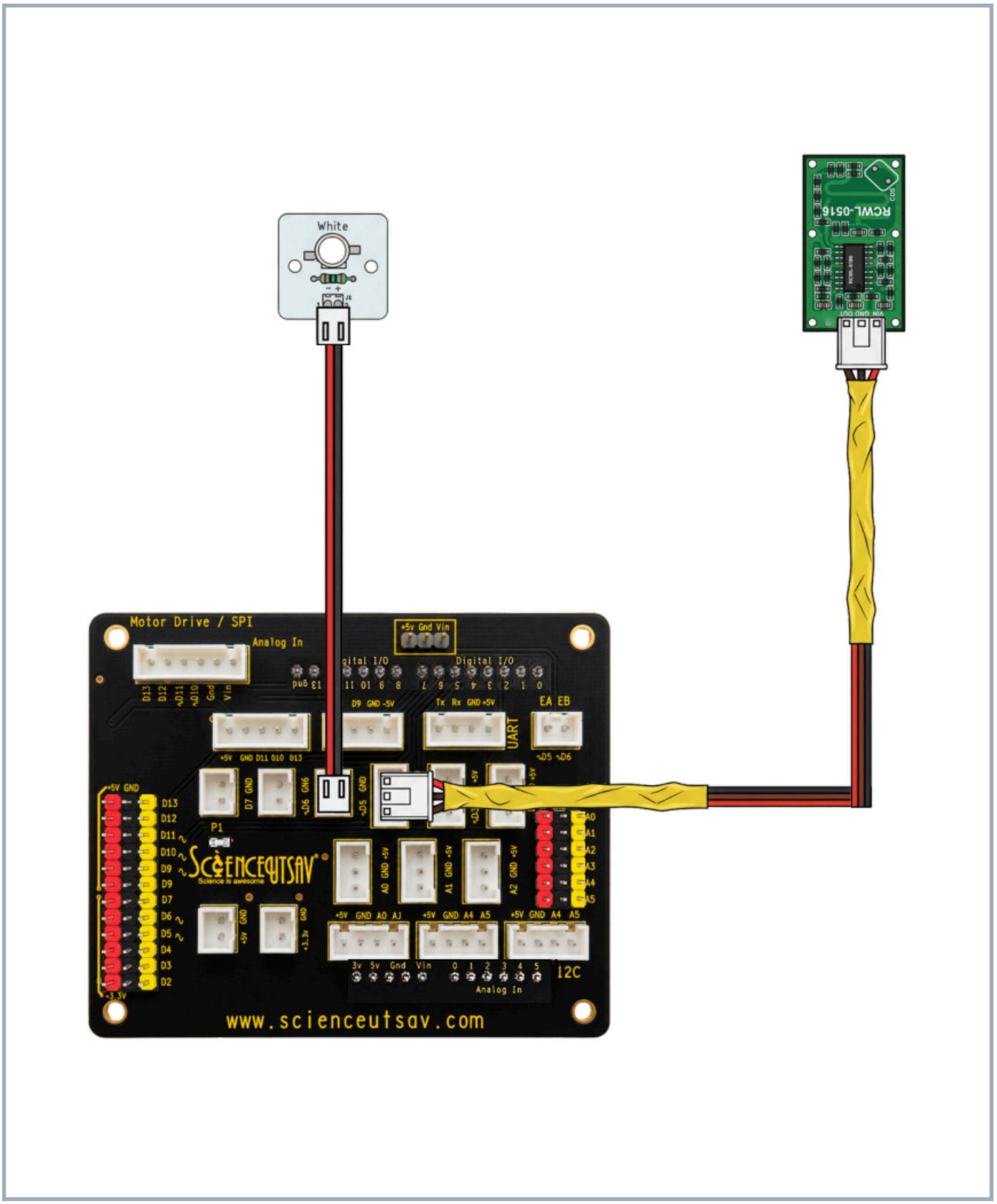
Board	Arduino UNO
Components	Microwave radar sensor, 1 W LED/lamp with driver
Required libraries	Built-in Arduino functions only
Sketch file	21_Motion_Lamp.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
Microwave radar sensor	OUT	4	Digital motion output.
Microwave radar sensor	VCC	5V	Check the exact radar module specification.
Microwave radar sensor	GND	GND	Common ground.
1 W lamp driver	Control input	5	Arduino control signal only. Use a transistor/MOSFET and external supply; never drive a 1 W LED directly.

Important: Do not connect a 1 W LED directly to D5. D5 must control a transistor/MOSFET driver circuit.

Project 21: Motion Lamp — Circuit Diagram



Project 21: Motion Lamp — Arduino Code

Sketch file: 21_Motion_Lamp.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

/*
  SAFETY: D5 is only a control signal. Never connect a 1W LED directly to an
  Arduino pin. Use a transistor/MOSFET driver, current-limiting resistor and
  suitable external power supply.
*/
const byte RADAR_PIN=4;
const byte LAMP_CONTROL=5;
void setup(){pinMode(RADAR_PIN,INPUT);pinMode(LAMP_CONTROL,OUTPUT);}
void loop(){digitalWrite(LAMP_CONTROL,digitalRead(RADAR_PIN));}
```

PROJECT 22

Motion Alarm

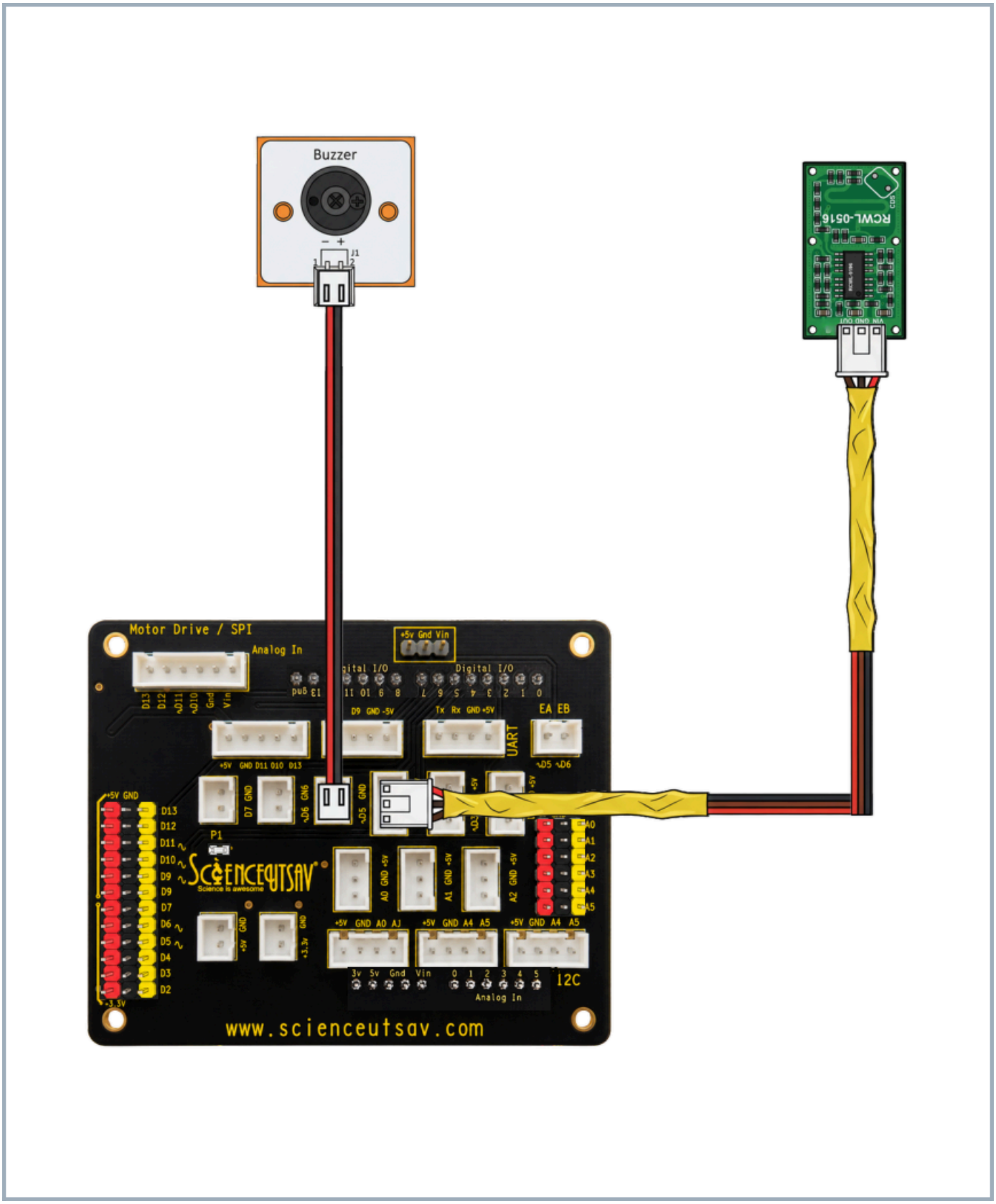
Trigger a buzzer alarm when motion is detected.

Board	Arduino UNO
Components	Microwave radar sensor, buzzer
Required libraries	Built-in Arduino functions only
Sketch file	22_Motion_Alarm.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
Microwave radar sensor	OUT	4	Digital motion output.
Microwave radar sensor	VCC	5V	Check the exact radar module specification.
Microwave radar sensor	GND	GND	Common ground.
Buzzer	Positive / signal	5	Negative terminal to GND.

Project 22: Motion Alarm — Circuit Diagram



Project 22: Motion Alarm — Arduino Code

Sketch file: 22_Motion_Alarm.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

const byte RADAR_PIN=4;
const byte BUZZER=5;
void setup(){pinMode(RADAR_PIN,INPUT);pinMode(BUZZER,OUTPUT);}
void loop(){
  if(digitalRead(RADAR_PIN))tone(BUZZER,1200);
  else noTone(BUZZER);
}
```

PROJECT 23

Auto Light + Alert

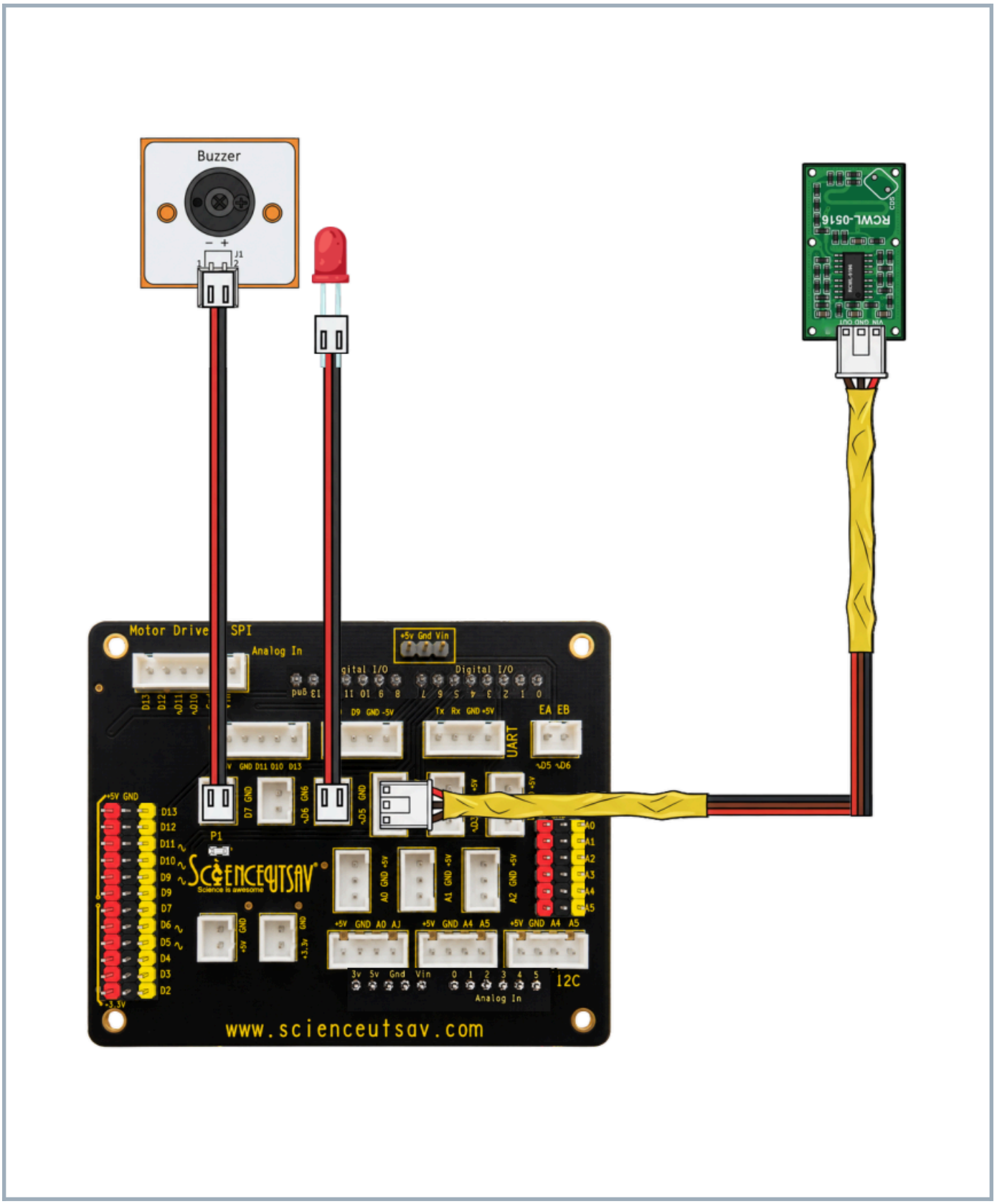
Switch on a light and buzzer alert simultaneously when motion is detected.

Board	Arduino UNO
Components	Microwave radar sensor, LED, buzzer
Required libraries	Built-in Arduino functions only
Sketch file	23_Auto_Light_Alert.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
Microwave radar sensor	OUT	4	Digital motion output.
Microwave radar sensor	VCC	5V	Check the exact radar module specification.
Microwave radar sensor	GND	GND	Common ground.
LED Pin	Anode	5	Use a 220–330 Ω resistor; cathode to GND.
Buzzer	Positive / signal	7	Negative terminal to GND.

Project 23: Auto Light + Alert — Circuit Diagram



Project 23: Auto Light + Alert — Arduino Code

Sketch file: 23_Auto_Light_Alert.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

const byte RADAR_PIN=4;
const byte BUZZER=7;
const byte LED_PIN=5;
void setup(){
  pinMode(RADAR_PIN,INPUT);pinMode(BUZZER,OUTPUT);pinMode(LED_PIN,OUTPUT);
}
void loop(){
  bool motion=digitalRead(RADAR_PIN);
  digitalWrite(LED_PIN,motion);
  if(motion)tone(BUZZER,1400,180);else noTone(BUZZER);
  delay(250);
}
```

PROJECT 24

Automatic Gate System

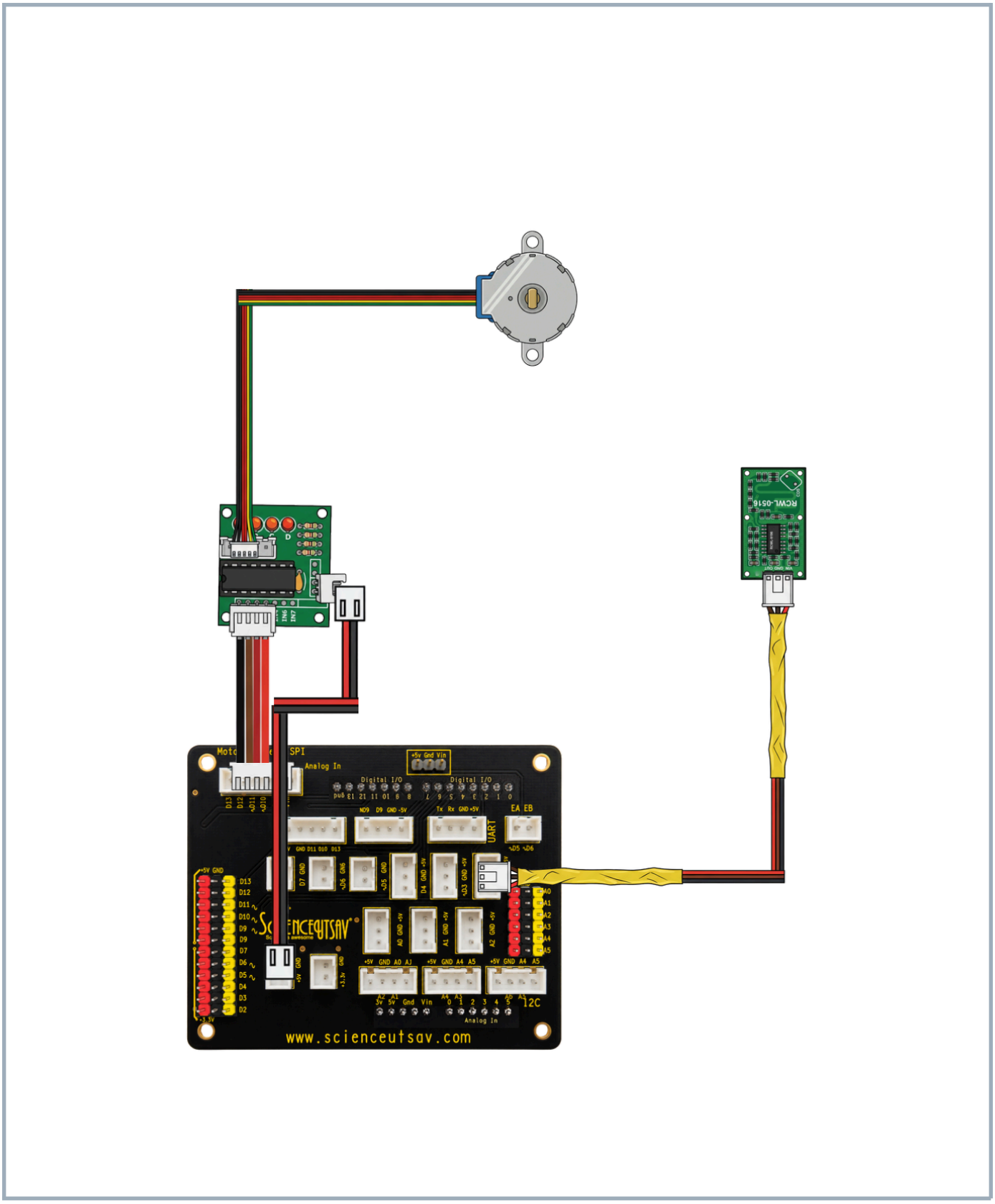
Open a motorized gate when the radar sensor detects a person approaching.

Board	Arduino UNO
Components	Microwave radar sensor, stepper motor with driver
Required libraries	Stepper.h
Sketch file	24_Automatic_Gate_System.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
ULN2003 stepper driver	IN1	10	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN2	11	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN3	12	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN4	13	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	GND	GND	Join motor-supply ground and Arduino ground.
Microwave radar sensor	OUT	2	Digital motion output.
Microwave radar sensor	VCC	5V	Check the exact radar module specification.
Microwave radar sensor	GND	GND	Common ground.

Project 24: Automatic Gate System — Circuit Diagram



Project 24: Automatic Gate System — Arduino Code

Sketch file: 24_Automatic_Gate_System.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Stepper.h>
const int STEPS_PER_REV=2048; // 28BYJ-48 with ULN2003; calibrate for your motor.
// Your shield pins are IN1=D10, IN2=D11, IN3=D12, IN4=D13.
// The Stepper library sequence for 28BYJ-48 is commonly IN1,IN3,IN2,IN4:
Stepper motor(STEPS_PER_REV,10,12,11,13);

const byte RADAR_PIN=2;
bool gateOpen=false;
unsigned long lastMotion=0;
void setup(){pinMode(RADAR_PIN,INPUT);motor.setSpeed(12);}
void loop(){
 if(digitalRead(RADAR_PIN)){
  lastMotion=millis();
  if(!gateOpen){motor.step(STEPS_PER_REV/4);gateOpen=true;}
 }
 if(gateOpen && millis()-lastMotion>5000){
  motor.step(-STEPS_PER_REV/4);gateOpen=false;
 }
}
```

PROJECT 25

Guest Welcome Box

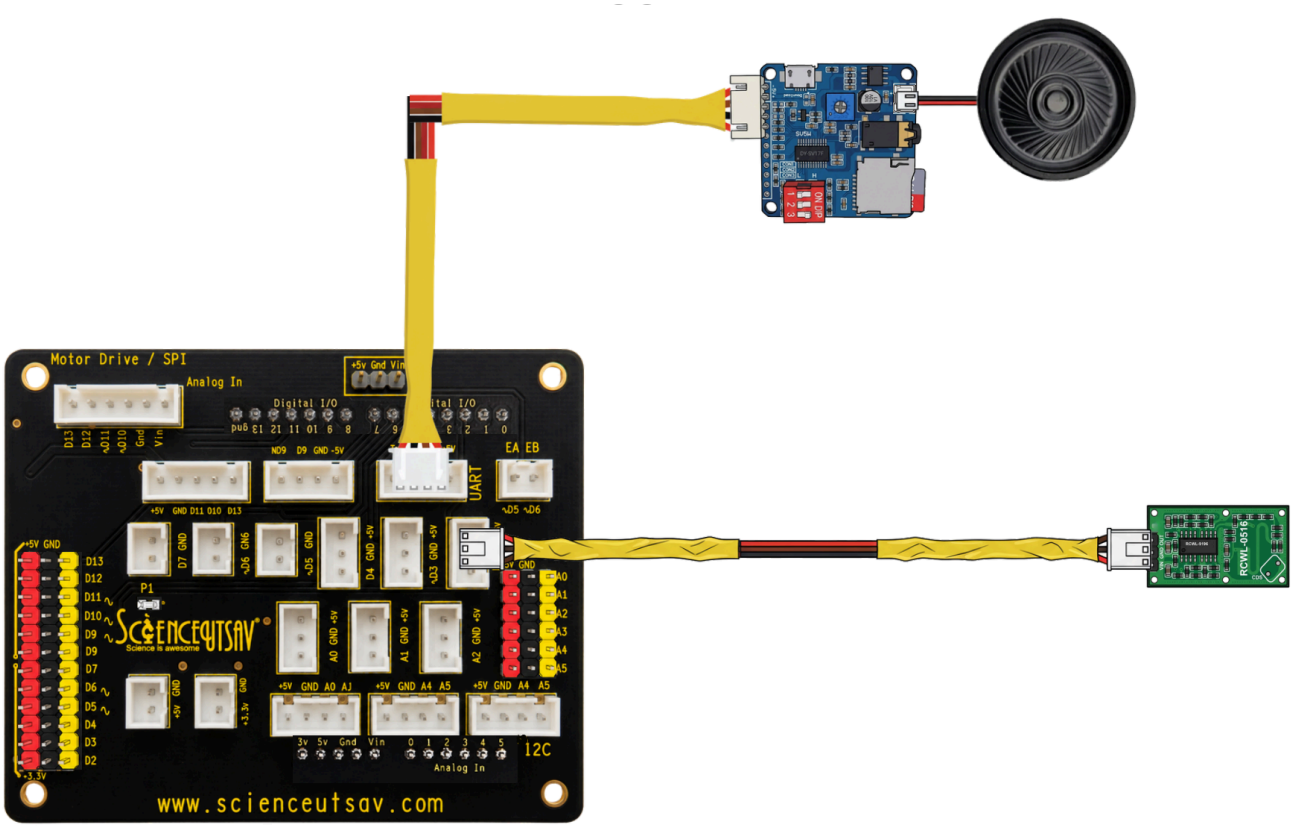
Play a welcome audio message when motion is detected.

Board	Arduino UNO
Components	Microwave radar sensor, DFPlayer Mini, speaker, microSD card
Required libraries	SoftwareSerial.h; DFRobotDFPlayerMini.h
Sketch file	25_Guest_Welcome_Box.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
DY-SW05 MP3	TX	0	Connect DFPlayer TX to Arduino SoftwareSerial RX.
DY-SW05 MP3	RX	1	Connect through about a 1 kΩ series resistor.
DY-SW05 MP3	VCC	5V	Use a stable supply.
DY-SW05 MP3	GND	GND	Common ground with Arduino.
Speaker	SPK1 / SPK2	DFPlayer SPK1 / SPK2	No Arduino I/O pin is required.
Microwave radar sensor	OUT	2	Digital motion output.
Microwave radar sensor	VCC	5V	Check the exact radar module specification.
Microwave radar sensor	GND	GND	Common ground.

Project 25: Guest Welcome Box — Circuit Diagram



Project 25: Guest Welcome Box — Arduino Code

Sketch file: 25_Guest_Welcome_Box.ino

```
// DY-SW05 MP3 Player + Radar Motion Sensor
const byte RADAR_PIN = 2;
bool previousMotion = false;
unsigned long lastWelcome = 0;
const unsigned long WELCOME_DELAY = 10000;  // 10 seconds
// ---------------- MP3 COMMAND FUNCTION ----------------
void sendMP3Command(uint8_t cmd, const uint8_t *data, uint8_t len) {
  uint8_t pkt[5 + len];
  pkt[0] = 0xAA;
  pkt[1] = cmd;
  pkt[2] = len;
  for (uint8_t i = 0; i < len; i++) {
    pkt[3 + i] = data[i];
  }
  uint16_t sum = 0;
  for (uint8_t i = 0; i < 3 + len; i++) {
    sum += pkt[i];
  }
  pkt[3 + len] = sum & 0xFF;
  Serial.write(pkt, 4 + len);
}
// ---------------- PLAY TRACK ----------------
void playWelcome() {
  // Select Track 01.mp3
  uint8_t track[] = {0x00, 0x01};
  sendMP3Command(0x07, track, 2);
  delay(100);
  // Play
  sendMP3Command(0x02, nullptr, 0);
}
// ---------------- SETUP ----------------
void setup() {
  pinMode(RADAR_PIN, INPUT);
  // Arduino hardware serial
  // RX = pin 0
  // TX = pin 1
  Serial.begin(9600);
  delay(1000);
  // Select SD Card
  uint8_t drive[] = {0x01};
  sendMP3Command(0x0B, drive, 1);
  delay(500);
```

```
  // Volume 0x00 - 0x1E
  // 0x1E = maximum
  uint8_t volume[] = {0x1E};
  sendMP3Command(0x13, volume, 1);
  delay(500);
}
// ---------------- LOOP ----------------
void loop() {
  bool motion = digitalRead(RADAR_PIN);
  // Trigger only when motion changes LOW → HIGH
  if (motion && !previousMotion) {
    if (millis() - lastWelcome >= WELCOME_DELAY) {
      lastWelcome = millis();
      playWelcome();
    }
  }
  previousMotion = motion;
}
```

PROJECT 26

Air Quality / Gas Reader

Read the raw MQ sensor value and show it on the LCD and Serial Monitor.

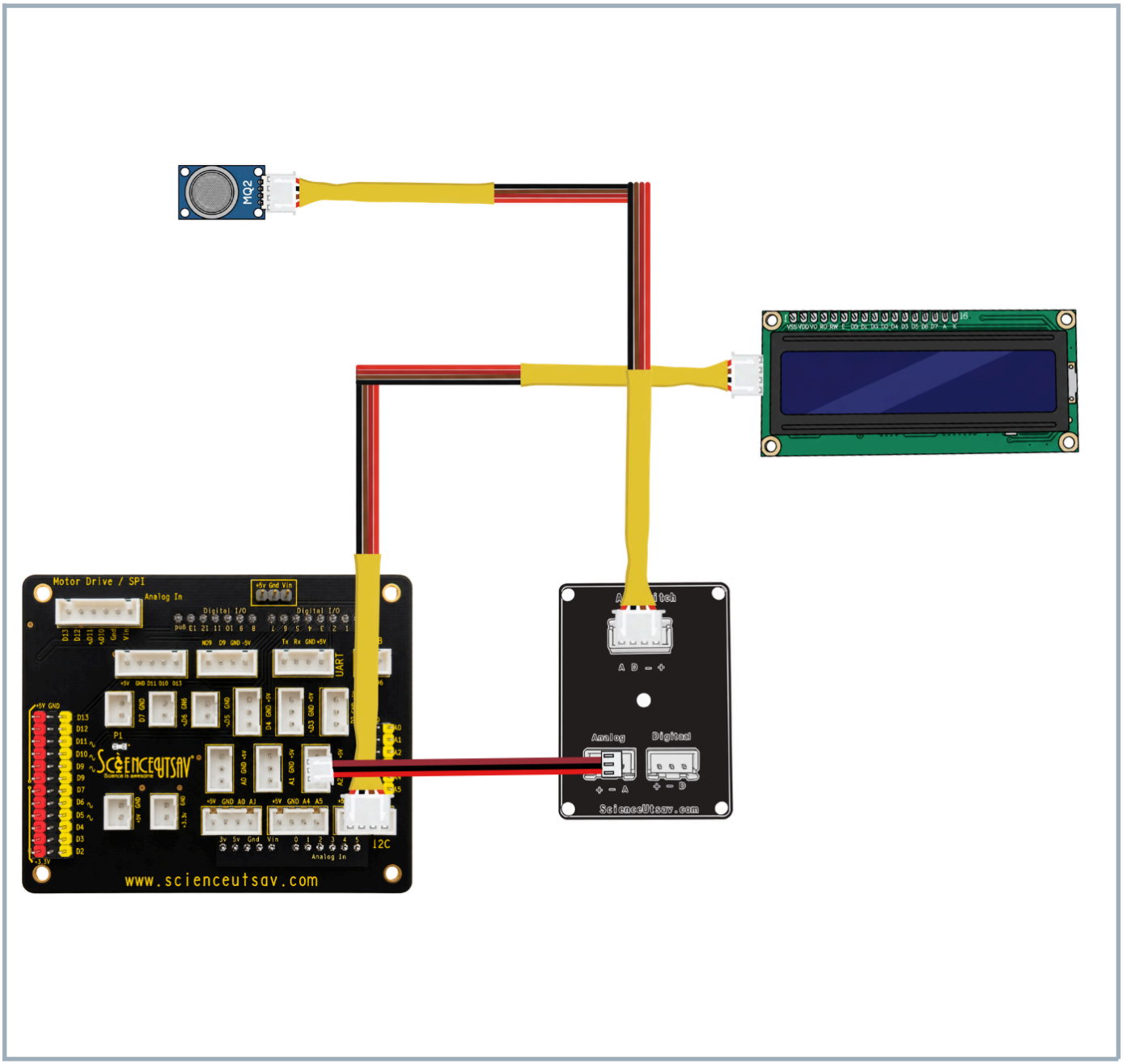
Board	Arduino UNO
Components	MQ135/MQ6/MQ2 sensor, LCD 16x2 I2C
Required libraries	Wire.h; LiquidCrystal_I2C.h
Sketch file	33_Air_Quality_Gas_Reader.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
MQ gas/air-quality sensor	AO	A2	Analog output; warm up and calibrate before setting thresholds.
MQ gas/air-quality sensor	VCC		MQ heater modules draw significant current.
MQ gas/air-quality sensor	GND		Common ground.

Important: Calibrate the MQ sensor in the actual environment and adjust the threshold constants before demonstration.

Project 26: Air Quality / Gas Reader — Circuit Diagram



Project 26: Air Quality / Gas Reader — Arduino Code

Sketch file: 26_Air_Quality_Gas_Reader.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

const byte MQ_AO_PIN = A2;

// Change this value after checking your normal sensor reading
const int GAS_THRESHOLD = 400;

unsigned long lastRead = 0;

void setup() {
 Serial.begin(9600);

 lcd.init();
 lcd.backlight();

 lcd.setCursor(0, 0);
 lcd.print("MQ Warming Up");
 lcd.setCursor(0, 1);
 lcd.print("Please Wait...");

 delay(5000);

 lcd.clear();
}

void loop() {

 if (millis() - lastRead < 500)
  return;

 lastRead = millis();

 // Read analog value from MQ sensor
 int gasValue = analogRead(MQ_AO_PIN);

 // Decide gas detected/not detected based on analog value
 bool gasDetected = gasValue >= GAS_THRESHOLD;
```

```
 // Serial Monitor
 Serial.print("Gas Level: ");
 Serial.print(gasValue);

 Serial.print("   Status: ");

 if (gasDetected) {
  Serial.println("DETECTED");
 }
 else {
  Serial.println("NORMAL");
 }

 // LCD
 lcd.clear();

 lcd.setCursor(0, 0);
 lcd.print("Gas Level:");
 lcd.print(gasValue);

 lcd.setCursor(0, 1);

 if (gasDetected) {
  lcd.print("Detected: YES");
 }
 else {
  lcd.print("Detected: NO ");
 }
}
```

PROJECT 27

Safe / Unsafe Level Display

Display the measured air-quality value and a Safe or Unsafe status.

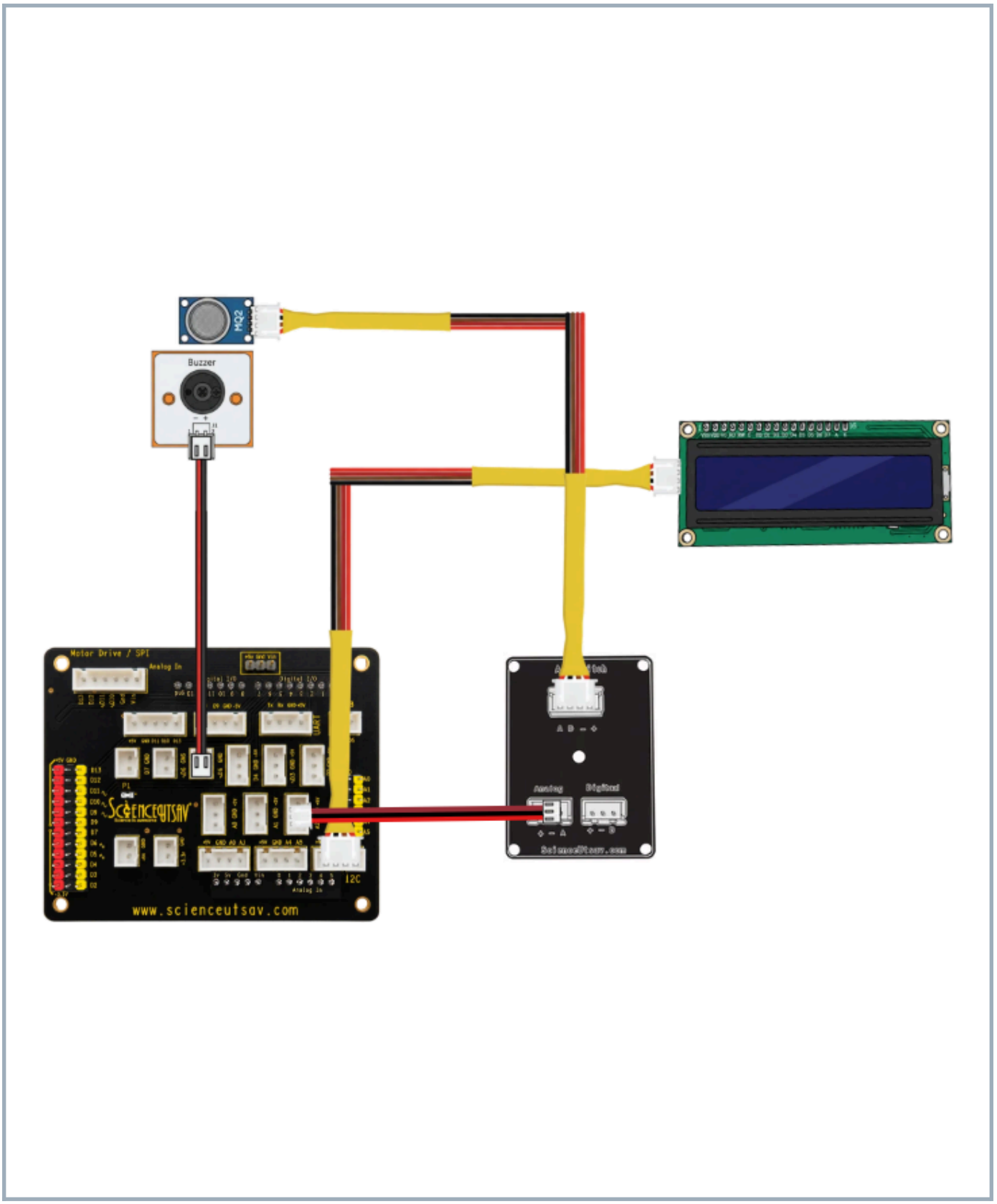
Board	Arduino UNO
Components	MQ135 sensor, LCD 16x2 I2C
Required libraries	Wire.h; LiquidCrystal_I2C.h
Sketch file	27_Safe_Unsafe_Level_Display.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
MQ gas/air-quality sensor	AO	A2	Analog output; warm up and calibrate before setting thresholds.
MQ gas/air-quality sensor	VCC	5V	MQ heater modules draw significant current.
MQ gas/air-quality sensor	GND	GND	Common ground.

Important: Calibrate the MQ sensor in the actual environment and adjust the threshold constants before demonstration.

Project 27: Safe / Unsafe Level Display — Circuit Diagram



Project 27: Safe / Unsafe Level Display — Arduino Code

Sketch file: 27_Safe_Unsafe_Level_Display.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

const byte MQ_PIN = A2;
const byte BUZZER_PIN = 5;

const int THRESHOLD = 450;

void setup() {
 lcd.init();
 lcd.backlight();

 pinMode(BUZZER_PIN, OUTPUT);
}

void loop() {

 int value = analogRead(MQ_PIN);

 lcd.setCursor(0, 0);
 lcd.print("Air Value:      ");

 lcd.setCursor(10, 0);
 lcd.print(value);

 lcd.setCursor(0, 1);

 if (value < THRESHOLD) {

  lcd.print("Status: SAFE    ");

  noTone(BUZZER_PIN);
 }
 else {

  lcd.print("Status: UNSAFE  ");

  tone(BUZZER_PIN, 1800);
 }

 delay(300);
```

```
}
```

PROJECT 28

PROJECT 28

RFID Player

Link RFID cards to different tracks and play the matching track when tapped.

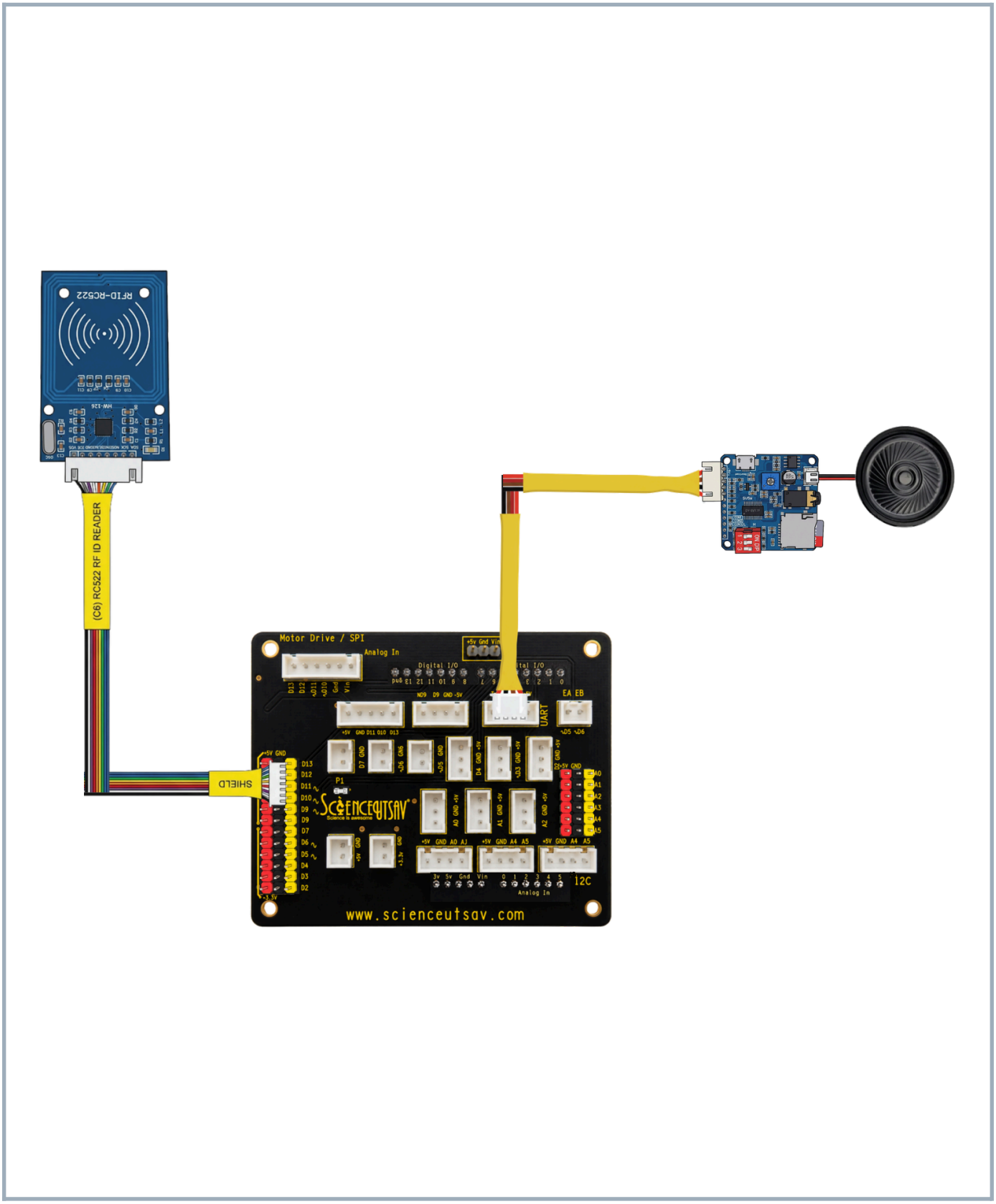
Board	Arduino UNO
Components	RC522 RFID reader, DFPlayer Mini, speaker, microSD card
Required libraries	SPI.h; MFRC522.h; SoftwareSerial.h; DFRobotDFPlayerMini.h
Sketch file	72_RFID_Player.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
RC522 RFID reader	SDA / SS	D10	SPI chip-select line.
RC522 RFID reader	SCK	D13	SPI clock.
RC522 RFID reader	MOSI	D11	SPI data from UNO to RC522.
RC522 RFID reader	MISO	D12	SPI data from RC522 to UNO.
RC522 RFID reader	RST	D8	Reset pin.
RC522 RFID reader	3.3V	D9	Do not connect directly to a 5 V GPIO output.
RC522 RFID reader	GND	D7	Use the actual ground pin. D9 LOW is only an optional shield arrangement.
DY-SW05 MP3	TX	0	Connect DFPlayer TX to Arduino SoftwareSerial RX.
DY-SW05 MP3	RX	1	Connect through about a 1 kΩ series resistor.
DY-SW05 MP3	VCC	5V	Use a stable supply.
DY-SW05 MP3	GND	GND	Common ground with Arduino.
Speaker	SPK1 / SPK2	DFPlayer SPK1 / SPK2	No Arduino I/O pin is required.

Important: Use Project 18 to read each card UID, then replace the sample UIDs in this sketch.

Project 28: RFID Player — Circuit Diagram



Project 28: RFID Player — Arduino Code

Sketch file: 72_RFID_Player.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

/*
  RFID occupies D7-D13, so this combined project moves DY-SW05 SoftwareSerial:
  DY-SW05 TX  -> Arduino D5
  DY-SW05 RX  -> Arduino D6
  DY-SW05 GND -> Arduino GND
  DY-SW05 5V  -> stable 5V supply
*/
#include <SPI.h>
#include <MFRC522.h>
#include <SoftwareSerial.h>
MFRC522 rfid(10,8);
SoftwareSerial mp3Serial(5,6);

// IMPORTANT: Connect RC522 VCC to the UNO 3.3V supply.
// Do not connect RC522 VCC directly to a GPIO HIGH; UNO GPIO HIGH is about 5V.
// Set this true only if your shield has a real 3.3V regulator between D7 and RC522 VCC.
const bool SHIELD_HAS_3V3_REGULATOR = false;
const byte RFID_POWER_PIN = 7;
const byte RFID_GROUND_PIN = 9;
void setupRfidPower() {
  if (SHIELD_HAS_3V3_REGULATOR) {
    pinMode(RFID_GROUND_PIN, OUTPUT); digitalWrite(RFID_GROUND_PIN, LOW);
    pinMode(RFID_POWER_PIN, OUTPUT); digitalWrite(RFID_POWER_PIN, HIGH);
    delay(100);
  }
}

void sendCommand(byte cmd, byte* data, byte dataLen){
  byte sum=0xAA+cmd+dataLen;
  mp3Serial.write(0xAA);mp3Serial.write(cmd);mp3Serial.write(dataLen);
  for(byte i=0;i<dataLen;i++){mp3Serial.write(data[i]);sum+=data[i];}
  mp3Serial.write(sum);
}
void setVolume(byte level){byte d[1]={level};sendCommand(0x13,d,1);}
void playTrack(byte trackNum){byte d[2]={0,trackNum};sendCommand(0x0F,d,2);}

String uidText(){
  String s="";
  for(byte i=0;i<rfid.uid.size;i++){if(rfid.uid.uidByte[i]<16)s+="0";s+=String(rfid.uid.uidByte[i],HEX);}
  s.toUpperCase();return s;
}
```

```
byte trackFor(const String& uid){
 if(uid=="A1B2C3D4")return 1;
 if(uid=="11223344")return 2;
 if(uid=="55667788")return 3;
 return 0;
}
void setup(){
 Serial.begin(9600);
 mp3Serial.begin(9600);
 delay(500);
 setVolume(23);
 setupRfidPower();SPI.begin();rfid.PCD_Init();
}
void loop(){
 if(!rfid.PICC_IsNewCardPresent()||!rfid.PICC_ReadCardSerial())return;
 String uid=uidText();byte track=trackFor(uid);
 if(track){playTrack(track);Serial.print("Track ");Serial.println(track);}
 else Serial.println("Unassigned RFID tag");
 rfid.PICC_HaltA();rfid.PCD_StopCrypto1();delay(700);
}
```

PROJECT 29

Air Quality Indicator

Use green, yellow and red LEDs plus a buzzer to indicate air-quality ranges.

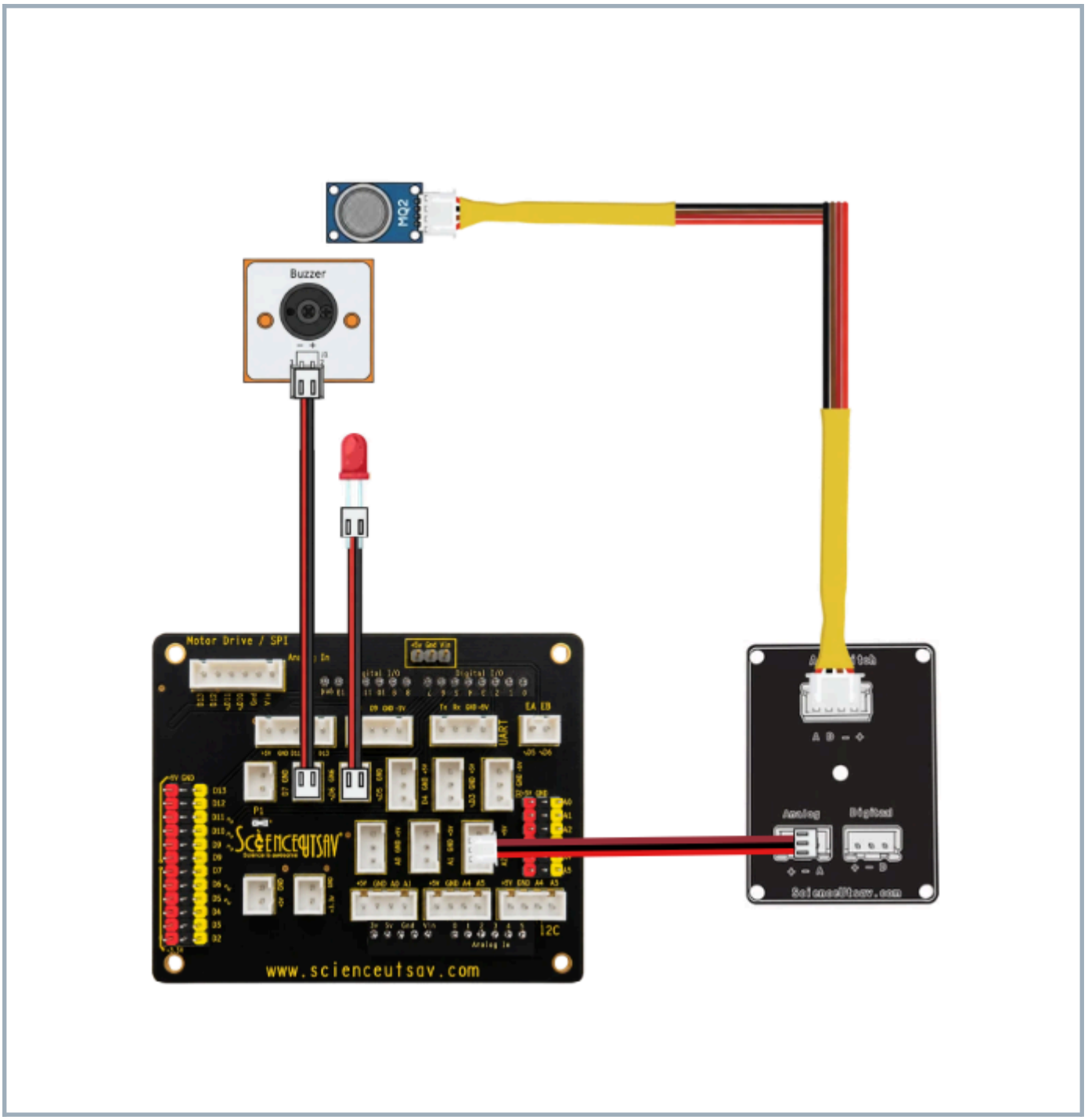
Board	Arduino UNO
Components	MQ135 sensor, 3 LEDs, buzzer
Required libraries	Built-in Arduino functions only
Sketch file	29_Air_Quality_Indicator.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
MQ gas/air-quality sensor	AO	A0	Analog output; warm up and calibrate before setting thresholds.
MQ gas/air-quality sensor	VCC	5V	MQ heater modules draw significant current.
MQ gas/air-quality sensor	GND	GND	Common ground.
Green LED	Anode	2	Use a 220–330 Ω resistor; cathode to GND.
Yellow LED	Anode	3	Use a 220–330 Ω resistor; cathode to GND.
Red LED	Anode	4	Use a 220–330 Ω resistor; cathode to GND.
Buzzer	Positive / signal	5	Negative terminal to GND.

Important: Calibrate the MQ sensor in the actual environment and adjust the threshold constants before demonstration.

Project 29: Air Quality Indicator — Circuit Diagram



Project 29: Air Quality Indicator — Arduino Code

Sketch file: 29_Air_Quality_Indicator.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

const byte MQ_PIN=A2;
const byte RED_LED=5;
const byte BUZZER=6;
const int DANGER_LEVEL=500;

const unsigned int SIREN_MIN_FREQ=500;
const unsigned int SIREN_MAX_FREQ=1200;
const unsigned long SIREN_SWEEP_MS=400; // time for one up-or-down sweep; also LED blink half-period

void setup(){
 pinMode(RED_LED,OUTPUT);
 pinMode(BUZZER,OUTPUT);
}
void loop(){
 int value=analogRead(MQ_PIN);
 bool danger=value>=DANGER_LEVEL;

 if(danger){
  unsigned long phase=millis()%(SIREN_SWEEP_MS*2);
  unsigned int freq;
  bool ledOn;
  if(phase<SIREN_SWEEP_MS){
   // rising sweep
   freq=SIREN_MIN_FREQ+(long)(SIREN_MAX_FREQ-SIREN_MIN_FREQ)*phase/SIREN_SWEEP_MS;
   ledOn=true;
  } else {
   // falling sweep
   unsigned long down=phase-SIREN_SWEEP_MS;
   freq=SIREN_MAX_FREQ-(long)(SIREN_MAX_FREQ-SIREN_MIN_FREQ)*down/SIREN_SWEEP_MS;
   ledOn=false;
  }
  tone(BUZZER,freq);
  digitalWrite(RED_LED,ledOn);
 } else {
  noTone(BUZZER);
  digitalWrite(RED_LED,LOW);
 }
}
```

PROJECT 30

Clean-Air Safety Door

Unlock the door only when the measured air quality is within the safe limit.

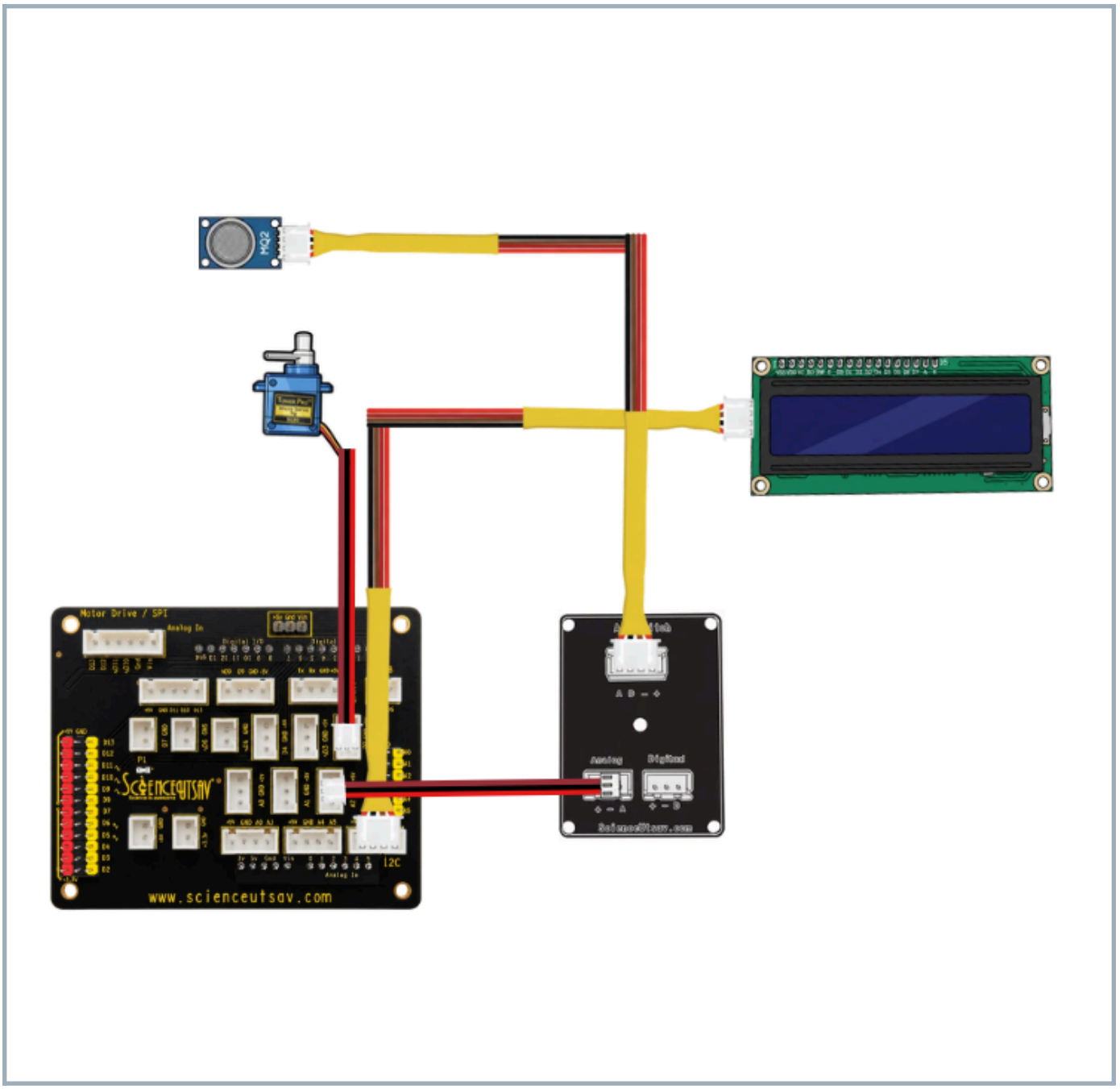
Board	Arduino UNO
Components	MQ135 sensor, LCD 16x2 I2C, servo motor
Required libraries	Wire.h; LiquidCrystal_I2C.h; Servo.h
Sketch file	30_Clean_Air_Safety_Door.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
Servo motor	Signal	2	Control signal from Arduino.
Servo motor	VCC	5V / external 5V	External supply is preferred for reliable operation.
Servo motor	GND	GND	Common ground with Arduino.
MQ gas/air-quality sensor	AD switch	A2	Analog output; warm up and calibrate before setting thresholds.
MQ gas/air-quality sensor	VCC	5V	MQ heater modules draw significant current.
MQ gas/air-quality sensor	GND	GND	Common ground.

Important: Calibrate the MQ sensor in the actual environment and adjust the threshold constants before demonstration.

Project 30: Clean-Air Safety Door — Circuit Diagram



Project 30: Clean-Air Safety Door — Arduino Code

Sketch file: 30_Clean_Air_Safety_Door.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>
LiquidCrystal_I2C lcd(0x27,16,2);
Servo door;
const byte MQ_PIN=A2;
const byte SERVO_PIN=2;
const int SAFE_LIMIT=450;
const int UNSAFE_LIMIT=480; // must clear this margin above SAFE_LIMIT before re-locking
bool safe=true; // assume safe on startup; door opens once loop runs

void setup(){
 door.attach(SERVO_PIN);door.write(0); // starts locked until first safe reading
 lcd.init();lcd.backlight();
}
void loop(){
 int value=analogRead(MQ_PIN);

 if(safe && value>=UNSAFE_LIMIT) safe=false;
 else if(!safe && value<SAFE_LIMIT) safe=true;

 door.write(safe?90:0);
 lcd.setCursor(0,0);lcd.print("Air:");lcd.print(value);lcd.print("       ");
 lcd.setCursor(0,1);lcd.print(safe?"Door: UNLOCKED ":"Door: LOCKED   ");
 delay(300);
}
```

PROJECT 31

Multi-Safety Door

Unlock the door only when both gas level and air-quality level are safe.

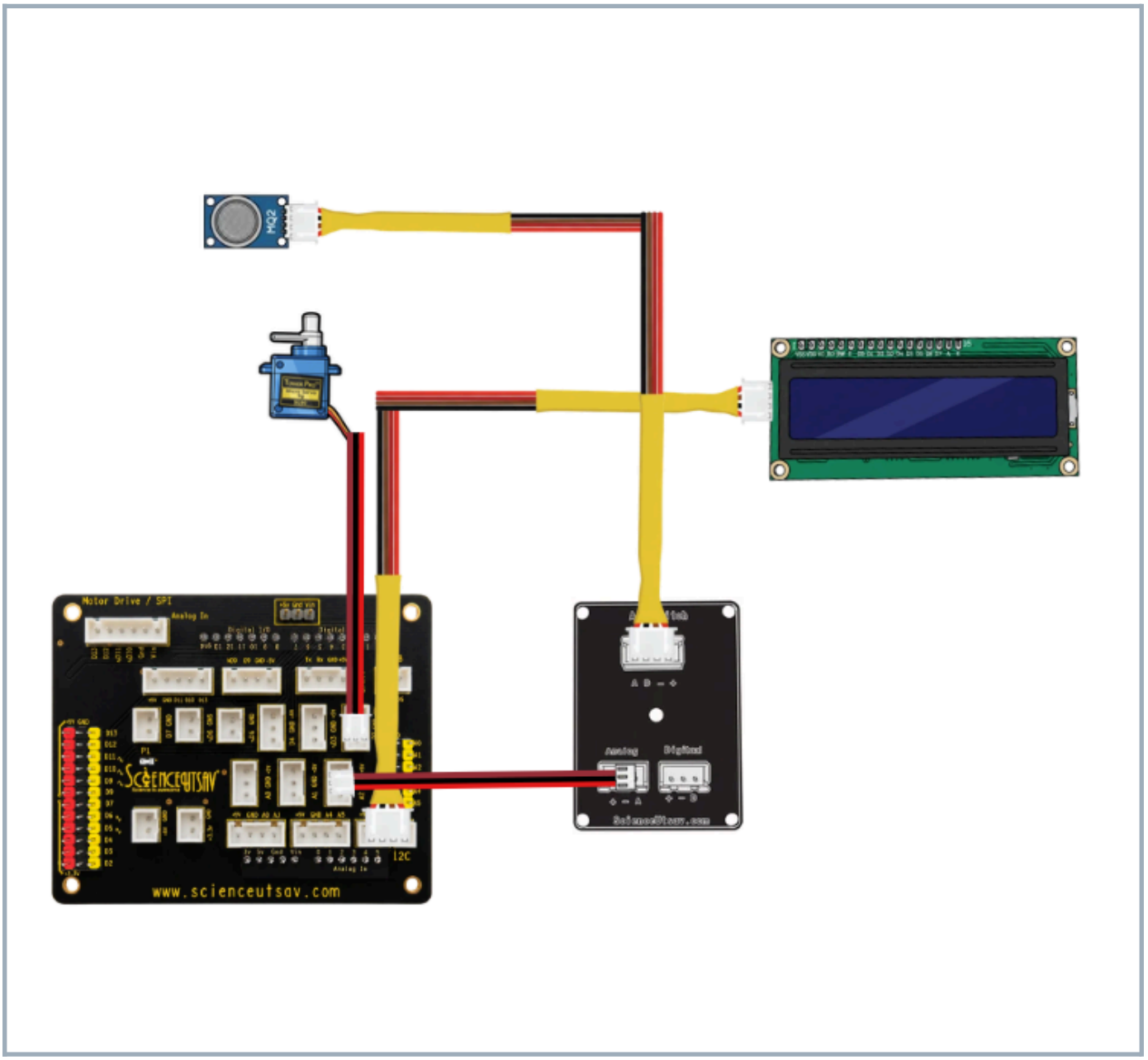
Board	Arduino UNO
Components	MQ2/MQ6 sensor, MQ135 sensor, LCD 16x2 I2C, servo motor
Required libraries	Wire.h; LiquidCrystal_I2C.h; Servo.h
Sketch file	31_Multi_Safety_Door.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
Servo motor	Signal	2	Control signal from Arduino.
Servo motor	VCC	5V / external 5V	External supply is preferred for reliable operation.
Servo motor	GND	GND	Common ground with Arduino.
MQ135 air-quality sensor	AD switch	A2	Analog output; warm up and calibrate before setting thresholds.
MQ135 air-quality sensor	VCC	5V	MQ heater modules draw significant current.
MQ135 air-quality sensor	GND	GND	Common ground.

Important: Calibrate the MQ sensor in the actual environment and adjust the threshold constants before demonstration.

Project 31: Multi-Safety Door — Circuit Diagram



Project 31: Multi-Safety Door — Arduino Code

☑

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo door;

const byte MQ_AIR_PIN = A2;
const byte SERVO_PIN = 2;

const int AIR_SAFE_LIMIT = 375;

void setup() {
  door.attach(SERVO_PIN);
  door.write(0);

  lcd.init();
  lcd.backlight();
}

void loop() {

  int air = analogRead(MQ_AIR_PIN);

  if (air >= AIR_SAFE_LIMIT) {
    // UNSAFE → turn servo to 90°
    door.write(90);

    lcd.setCursor(0, 0);
    lcd.print("Air: ");
    lcd.print(air);
    lcd.print("       ");

    lcd.setCursor(0, 1);
    lcd.print("UNSAFE - LOCKED ");

  } else {
    // SAFE → return servo to 0°
    door.write(0);

    lcd.setCursor(0, 0);
    lcd.print("Air: ");
    lcd.print(air);
    lcd.print("       ");

    lcd.setCursor(0, 1);
    lcd.print("SAFE - OPEN     ");
  }

  delay(300);
}
```

PROJECT 32

Gas Leakage Alert System

Play a warning audio clip when the MQ sensor detects a gas leak.

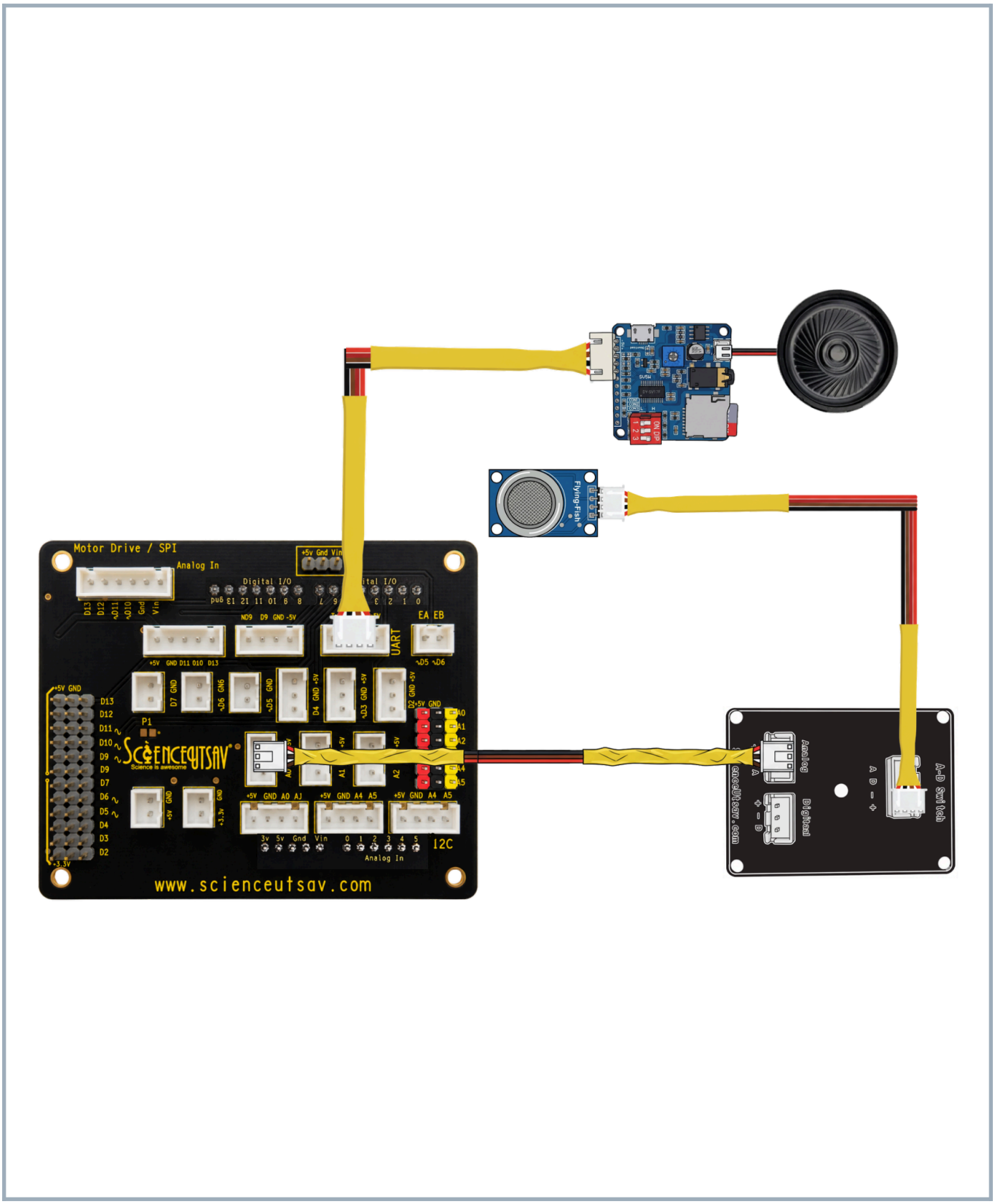
Board	Arduino UNO
Components	MQ sensor, DFPlayer Mini, speaker, microSD card
Required libraries	SoftwareSerial.h; DFRobotDFPlayerMini.h
Sketch file	32_Gas_Leakage_Alert_System.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
DY-SW05 MP3	TX	0	Connect DFPlayer TX to Arduino SoftwareSerial RX.
DY-SW05 MP3	RX	1	Connect through about a 1 kΩ series resistor.
DY-SW05 MP3	VCC	5V	Use a stable supply.
DY-SW05 MP3	GND	GND	Common ground with Arduino.
Speaker	SPK1 / SPK2	DFPlayer SPK1 / SPK2	No Arduino I/O pin is required.
MQ gas/air-quality sensor	AO	A0	Analog output; warm up and calibrate before setting thresholds.
MQ gas/air-quality sensor	VCC	5V	MQ heater modules draw significant current.
MQ gas/air-quality sensor	GND	GND	Common ground.

Important: Calibrate the MQ sensor in the actual environment and adjust the threshold constants before demonstration.

Project 32: Gas Leakage Alert System — Circuit Diagram



Project 32: Gas Leakage Alert System — Arduino Code ☑

Sketch file: 32_Gas_Leakage_Alert_System.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

/*
 DY-SW05 wiring:
 DY-SW05 TX  -> Arduino D8 (SoftwareSerial RX)
 DY-SW05 RX  -> Arduino D9 (SoftwareSerial TX)
 DY-SW05 GND -> Arduino GND
 DY-SW05 5V  -> stable 5V supply
*/

#include <SoftwareSerial.h>
SoftwareSerial mp3Serial(8,9);
const byte MQ_PIN=A0;
const int GAS_THRESHOLD=450;
bool alertPlayed=false;

void sendCommand(byte cmd, byte* data, byte dataLen){
 byte sum=0xAA+cmd+dataLen;
 mp3Serial.write(0xAA);mp3Serial.write(cmd);mp3Serial.write(dataLen);
 for(byte i=0;i<dataLen;i++){mp3Serial.write(data[i]);sum+=data[i];}
 mp3Serial.write(sum);
}
void setVolume(byte level){byte d[1]={level};sendCommand(0x13,d,1);}
void playTrack(byte trackNum){byte d[2]={0,trackNum};sendCommand(0x0F,d,2);}

void setup(){
 mp3Serial.begin(9600);
 delay(500);
 setVolume(25);
}
void loop(){
 int gas=analogRead(MQ_PIN);
 bool leak=gas>=GAS_THRESHOLD;
 if(leak&&!alertPlayed){playTrack(1);alertPlayed=true;}
 if(!leak)alertPlayed=false;
 delay(250);
}
```

PRPROJECT 33

Musical Alarm System

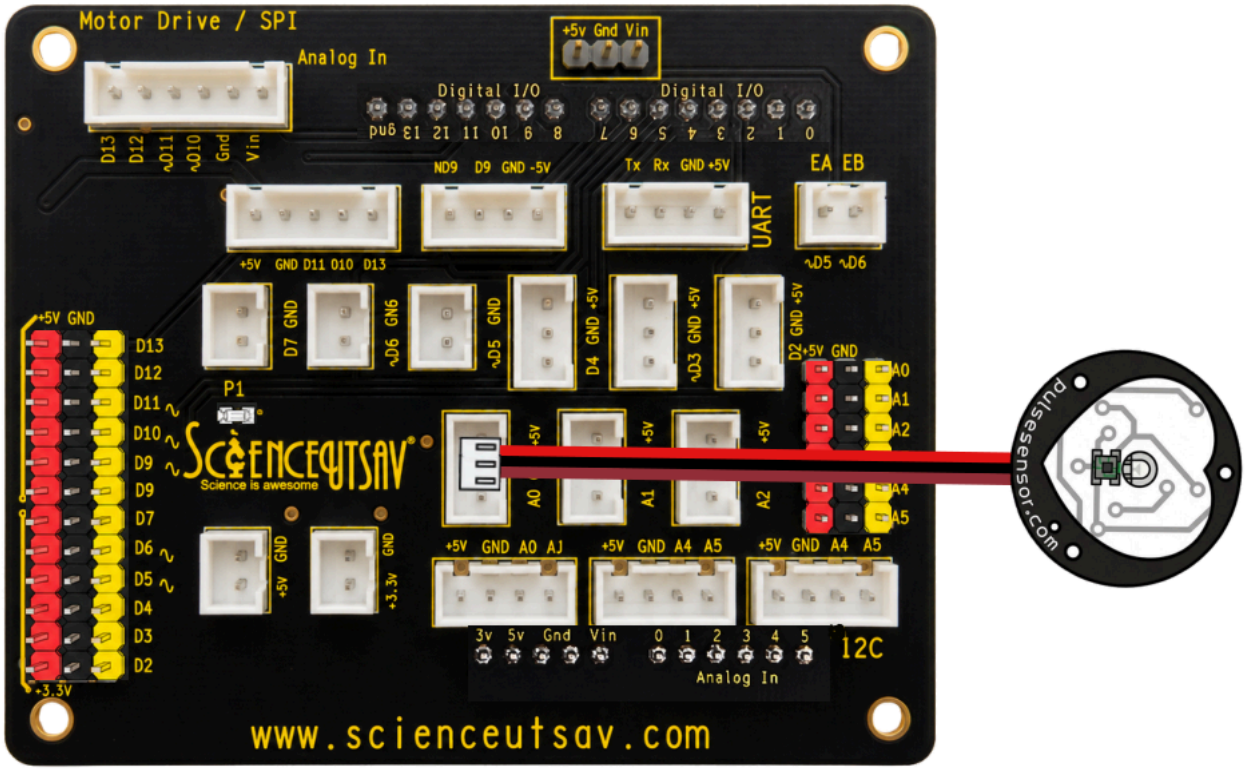
Use the DFPlayer Mini to play a selected song as an RTC alarm.

Board	Arduino UNO
Components	DS3231 RTC, LCD 16x2 I2C, 4x4 keypad, push button, DFPlayer Mini, speaker
Required libraries	Wire.h; LiquidCrystal_I2C.h; RTClib.h; SoftwareSerial.h; DFRobotDFPlayerMini.h; Keypad.h
Sketch file	74_Musical_Alarm_System.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
DS3231/DS1307 RTC	VCC	5V	Use the module supply rating.
DS3231/DS1307 RTC	GND	GND	Common ground.
DS3231/DS1307 RTC	SDA	A4	Shares the I2C bus with the LCD.
DS3231/DS1307 RTC	SCL	A5	Shares the I2C bus with the LCD.
4x4 keypad	R1	2	Keypad row wire.
4x4 keypad	R2	3	Keypad row wire.
4x4 keypad	R3	4	Keypad row wire.
4x4 keypad	R4	5	Keypad row wire.
4x4 keypad	C1	6	Keypad column wire.
4x4 keypad	C2	7	Keypad column wire.
4x4 keypad	C3	A0	Keypad column wire.
4x4 keypad	C4	A1	Keypad column wire.
DFPlayer Mini	TX	8	Connect DFPlayer TX to Arduino SoftwareSerial RX.
DFPlayer Mini	RX	9	Connect through about a 1 kΩ series resistor.
DFPlayer Mini	VCC	5V	Use a stable supply.
DFPlayer Mini	GND	GND	Common ground with Arduino.
Speaker	SPK1 / SPK2	DFPlayer SPK1 / SPK2	No Arduino I/O pin is required.
Arm Button	Signal terminal	10	Other terminal to GND; code uses INPUT_PULLUP unless noted.

Project 33: Musical Alarm System — Circuit Diagram



Project 33: Musical Alarm System — Arduino Code

Sketch file: 74_Musical_Alarm_System.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

/*
 DFPlayer wiring:
 DFPlayer TX -> Arduino D8 (SoftwareSerial RX)
 Arduino D9 (SoftwareSerial TX) -> DFPlayer RX through about 1k resistor
 Speaker -> DFPlayer SPK1/SPK2. Use a stable 5V supply and common ground.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <RTClib.h>
#include <SoftwareSerial.h>
#include <DFRobotDFPlayerMini.h>
#include <Keypad.h>
LiquidCrystal_I2C lcd(0x27,16,2);
RTC_DS3231 rtc;
SoftwareSerial mp3Serial(8,9);
DFRobotDFPlayerMini player;
const byte ARM_BUTTON=10;
byte alarmHour=7,alarmMinute=0,track=1;
bool armed=true,buttonPrev=HIGH;
String input="";
int lastAlarmDay=-1;
const byte ROWS=4,COLS=4;
char keys[ROWS][COLS]={{'1','2','3','A'},{'4','5','6','B'},{'7','8','9','C'},{'*','0','#','D'}};
byte rowPins[ROWS]={2,3,4,5};
byte colPins[COLS]={6,7,A0,A1};
Keypad keypad=Keypad(makeKeymap(keys),rowPins,colPins,ROWS,COLS);

void setup(){
 pinMode(ARM_BUTTON,INPUT_PULLUP);
 lcd.init();lcd.backlight();
 if(!rtc.begin()){lcd.print("RTC error");while(true);}
 mp3Serial.begin(9600);if(!player.begin(mp3Serial)){lcd.print("MP3 error");while(true);}
 player.volume(24);
}
void loop(){
 bool b=digitalRead(ARM_BUTTON);
 if(buttonPrev==HIGH&&b==LOW)armed=!armed;
 buttonPrev=b;

 char key=keypad.getKey();
 if(key=='*')input="";
 else if(key=='#'&&input.length()==4){
```

```
  int hh=input.substring(0,2).toInt(),mm=input.substring(2,4).toInt();
  if(hh<24&&mm<60){alarmHour=hh;alarmMinute=mm;}
  input="";
 }else if(key=='A')track=1;
 else if(key=='B')track=2;
 else if(key=='C')track=3;
 else if(isDigit(key)&&input.length()<4)input+=key;

 DateTime now=rtc.now();
 if(armed&&now.hour()==alarmHour&&now.minute()==alarmMinute&&lastAlarmDay!=now.day()){
  player.play(track);lastAlarmDay=now.day();
 }
 char line1[17],line2[17];
 snprintf(line1,sizeof(line1),"Now %02d:%02d:%02d",now.hour(),now.minute(),now.second());
 snprintf(line2,sizeof(line2),"A %02d:%02d T%d %s",alarmHour,alarmMinute,track,armed?"ON":"OFF");
 lcd.setCursor(0,0);lcd.print(line1);
 lcd.setCursor(0,1);lcd.print(line2);lcd.print(" ");
 delay(80);
}
```

PROJECT 34

Heart Rate Display

Calculate and display the heart rate in BPM on the Serial Monitor.

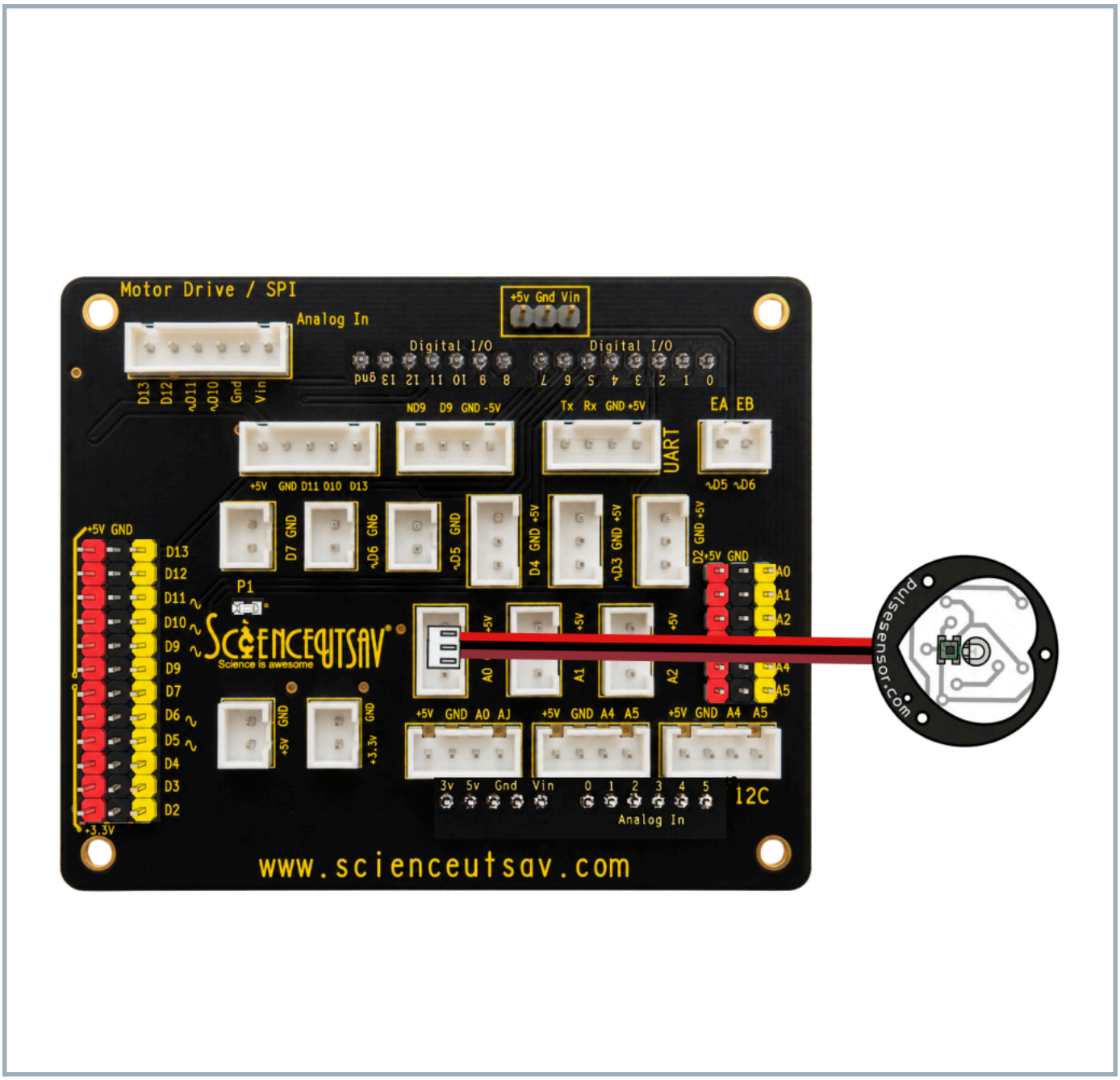
Board	Arduino UNO
Components	Pulse sensor
Required libraries	PulseSensorPlayground.h
Sketch file	34_Heart_Rate_Display.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
Pulse sensor	Signal	A0	Analog signal input; educational use only, not a medical device.
Pulse sensor	VCC	5V	Check the sensor module marking.
Pulse sensor	GND	GND	Common ground.

Important: The pulse-sensor output is for educational demonstration only and must not be used for medical decisions.

Project 34: Heart Rate Display — Circuit Diagram



Project 34: Heart Rate Display — Arduino Code

Sketch file: 34_Heart_Rate_Display.ino

```
int pulsePin = A0;              // Pulse Sensor connected to analog pin A0
int blinkPin = 13;              // Pin to blink LED at each beat

// Volatile variables used in the Interrupt Service Routine
volatile int BPM;               // Calculated Beats Per Minute
volatile int Signal;            // Raw analog signal input
volatile int IBI = 600;         // Time interval between beats (ms)
volatile boolean Pulse = false;     // True when heartbeat is actively detected
volatile boolean QS = false;       // Set to true when Arduino finds a beat

volatile int rate[10];          // Array to hold last 10 IBI values for averaging
volatile unsigned long sampleCounter = 0;
volatile unsigned long lastBeatTime = 0;
volatile int P = 512;           // Peak in pulse wave
volatile int T = 512;           // Trough in pulse wave
volatile int thresh = 525;        // Pulse threshold
volatile int amp = 100;          // Amplitude of pulse wave
volatile boolean firstBeat = true;
volatile boolean secondBeat = false;

void setup() {
 pinMode(blinkPin, OUTPUT);       // Set LED pin as output
 Serial.begin(115200);           // Set Serial Monitor baud rate
 Serial.println("Pulse Sensor Ready. Place finger on sensor...");

 interruptSetup();               // Initialize Timer2 interrupt (500Hz)
}

void loop() {
 // Prints BPM to Serial Monitor only when a full beat cycle is processed
 if (QS == true) {
   Serial.print("BPM: ");
   Serial.println(BPM);

   QS = false;                  // Reset flag for next beat
 }

 delay(20);
}

void interruptSetup() {
 TCCR2A = 0x02;     // CTC mode
 TCCR2B = 0x06;     // 256 Prescaler
 OCR2A = 0x7C;      // Count top = 124 (500Hz sampling rate)
 TIMSK2 = 0x02;     // Enable interrupt on compare match
 sei();           // Enable global interrupts
}

ISR(TIMER2_COMPA_vect) {
 cli();
```

```
Signal = analogRead(pulsePin);
sampleCounter += 2;
int N = sampleCounter - lastBeatTime;

if (Signal < thresh && N > (IBI / 5) * 3) {
  if (Signal < T) {
    T = Signal;
  }
}

if (Signal > thresh && Signal > P) {
  P = Signal;
}

if (N > 250) {
  if ((Signal > thresh) && (Pulse == false) && (N > (IBI / 5) * 3)) {
    Pulse = true;
    digitalWrite(blinkPin, HIGH);
    IBI = sampleCounter - lastBeatTime;
    lastBeatTime = sampleCounter;

    if (secondBeat) {
      secondBeat = false;
      for (int i = 0; i <= 9; i++) {
        rate[i] = IBI;
      }
    }

    if (firstBeat) {
      firstBeat = false;
      secondBeat = true;
      sei();
      return;
    }

    word runningTotal = 0;
    for (int i = 0; i <= 8; i++) {
      rate[i] = rate[i + 1];
      runningTotal += rate[i];
    }

    rate[9] = IBI;
    runningTotal += rate[9];
    runningTotal /= 10;
    BPM = 60000 / runningTotal;
    QS = true;
  }
}

if (Signal < thresh && Pulse == true) {
  digitalWrite(blinkPin, LOW);
  Pulse = false;
  amp = P - T;
  thresh = amp / 2 + T;
  P = thresh;
  T = thresh;
```

```
 }

 if (N > 2500) {
   thresh = 512;
   P = 512;
   T = 512;
   lastBeatTime = sampleCounter;
   firstBeat = true;
   secondBeat = false;
 }

 sei();
}
```

PROJECT 35

Safe Range Check

Display BPM and show Safe or Unsafe based on the configured normal range.

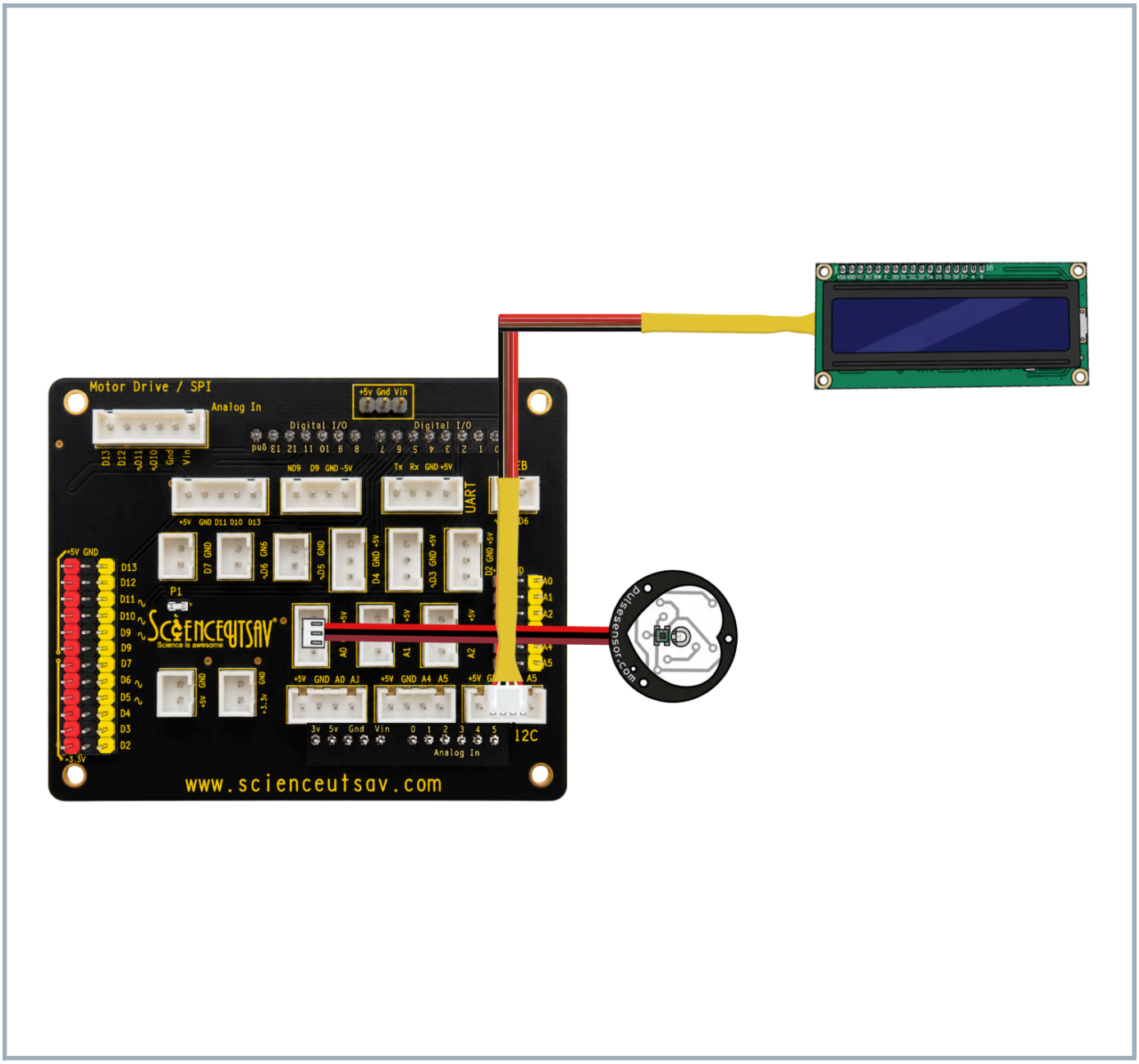
Board	Arduino UNO
Components	Pulse sensor, LCD 16x2 I2C
Required libraries	Wire.h; LiquidCrystal_I2C.h; PulseSensorPlayground.h
Sketch file	35_Safe_Range_Check.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
Pulse sensor	Signal	A0	Analog signal input; educational use only, not a medical device.
Pulse sensor	VCC	5V	Check the sensor module marking.
Pulse sensor	GND	GND	Common ground.

Important: The pulse-sensor output is for educational demonstration only and must not be used for medical decisions.

Project 35: Safe Range Check — Circuit Diagram



Project 35: Safe Range Check — Arduino Code

Sketch file: 35_Safe_Range_Check.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

// Pulse Sensor signal must connect to an ANALOG input. This sketch uses A0.

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <PulseSensorPlayground.h>
LiquidCrystal_I2C lcd(0x27,16,2);
PulseSensorPlayground pulse;
const byte PULSE_PIN=A0;
const int MIN_BPM=60,MAX_BPM=100; // Educational default, not a medical diagnosis.
void setup(){
 lcd.init();lcd.backlight();
 pulse.analogInput(PULSE_PIN);pulse.setThreshold(550);pulse.begin();
}
void loop(){
 if(pulse.sawStartOfBeat()){
  int bpm=pulse.getBeatsPerMinute();
  lcd.clear();lcd.print("Heart: ");lcd.print(bpm);lcd.print(" BPM");
  lcd.setCursor(0,1);
  lcd.print(bpm>=MIN_BPM && bpm<=MAX_BPM?"Range: SAFE":"Range: UNSAFE");
 }
 delay(10);
}
```

PROJECT 36

Tag LED Controller

Change the LED colour pattern and buzzer pattern according to the RFID tag scanned.

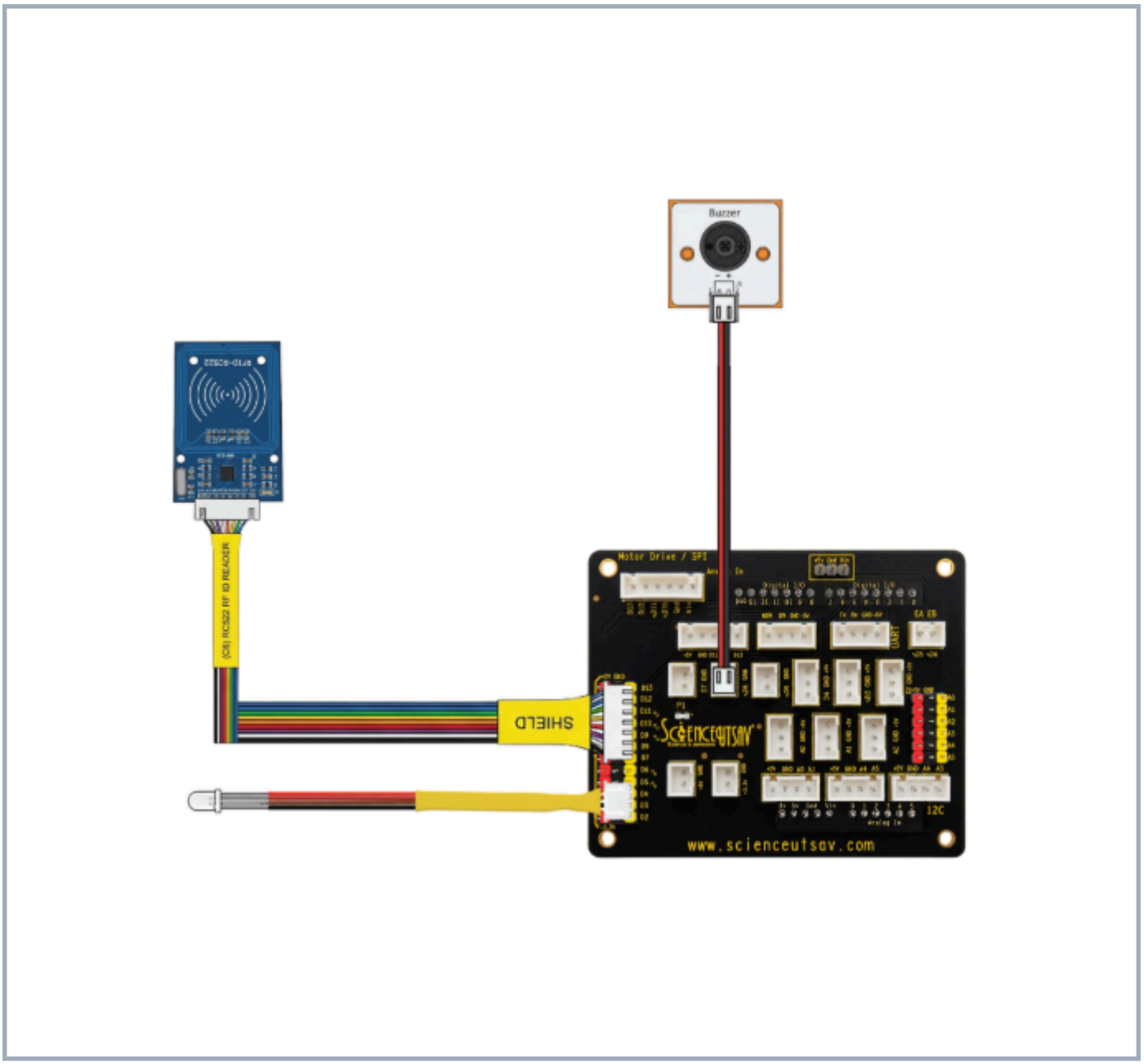
Board	Arduino UNO
Components	RC522 RFID reader, 3 LEDs, buzzer
Required libraries	SPI.h; MFRC522.h
Sketch file	20_Tag_LED_Controller.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
RC522 RFID reader	SDA / SS	D10	SPI chip-select line.
RC522 RFID reader	SCK	D13	SPI clock.
RC522 RFID reader	MOSI	D11	SPI data from UNO to RC522.
RC522 RFID reader	MISO	D12	SPI data from RC522 to UNO.
RC522 RFID reader	RST	D8	Reset pin.
RC522 RFID reader	3.3V	UNO 3.3V	Do not connect directly to a 5 V GPIO output.
RC522 RFID reader	GND	GND	Use the actual ground pin. D9 LOW is only an optional shield arrangement.
RGB LED	Anode	2	Use a 220–330 Ω resistor; cathode to GND.
	Anode	3	Use a 220–330 Ω resistor; cathode to GND.
	Anode	4	Use a 220–330 Ω resistor; cathode to GND.
		5	
Buzzer	Positive / signal	6	Negative terminal to GND.

Important: The shield RGB pins D10, D11 and D13 are occupied by the RC522 SPI bus. This code therefore uses three separate DIP LEDs on D2, D3 and D4.

Project 36: Tag LED Controller — Circuit Diagram



Project 36: Tag LED Controller — Arduino Code

Sketch file: 20_Tag_LED_Controller.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27
 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/
#include <SPI.h>
#include <MFRC522.h>
MFRC522 rfid(10, 8);
const byte RED_PIN = 2;
const byte GREEN_PIN = 3;
const byte BLUE_PIN = 4;
const byte BUZZER = 6;
// IMPORTANT: Connect RC522 VCC to the UNO 3.3V supply.
// Do not connect RC522 VCC directly to a GPIO HIGH; UNO GPIO HIGH is about 5V.
// Set this true only if your shield has a real 3.3V regulator between D7 and RC522 VCC.
const bool SHIELD_HAS_3V3_REGULATOR = false;
const byte RFID_POWER_PIN = 7;
const byte RFID_GROUND_PIN = 9;
void setupRfidPower() {
 if (SHIELD_HAS_3V3_REGULATOR) {
  pinMode(RFID_GROUND_PIN, OUTPUT); digitalWrite(RFID_GROUND_PIN, LOW);
  pinMode(RFID_POWER_PIN, OUTPUT); digitalWrite(RFID_POWER_PIN, HIGH);
  delay(100);
 }
}
String uidText() {
 String s="";
 for(byte i=0;i<rfid.uid.size;i++){ if(rfid.uid.uidByte[i]<16)s+="0"; s+=String(rfid.uid.uidByte[i],HEX); }
 s.toUpperCase(); return s;
}
void setColor(bool r, bool g, bool b) {
 digitalWrite(RED_PIN,r); digitalWrite(GREEN_PIN,g); digitalWrite(BLUE_PIN,b);
}
void setup() {
 pinMode(RED_PIN,OUTPUT); pinMode(GREEN_PIN,OUTPUT); pinMode(BLUE_PIN,OUTPUT);
 pinMode(BUZZER,OUTPUT);
 Serial.begin(9600);
 setupRfidPower(); SPI.begin(); rfid.PCD_Init();
}
void loop() {
 if(!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) return;
 String uid=uidText();
 if(uid=="A1B2C3D4"){ setColor(0,1,0); tone(BUZZER,1800,120); }
 else if(uid=="11223344"){ setColor(0,0,1); tone(BUZZER,1400,250); }
 else { setColor(1,0,0); tone(BUZZER,500,700); }
 Serial.println(uid);
 rfid.PICC_HaltA(); rfid.PCD_StopCrypto1();
 delay(1200); setColor(0,0,0);
}
```

PROJECT 37

Changing Text Display

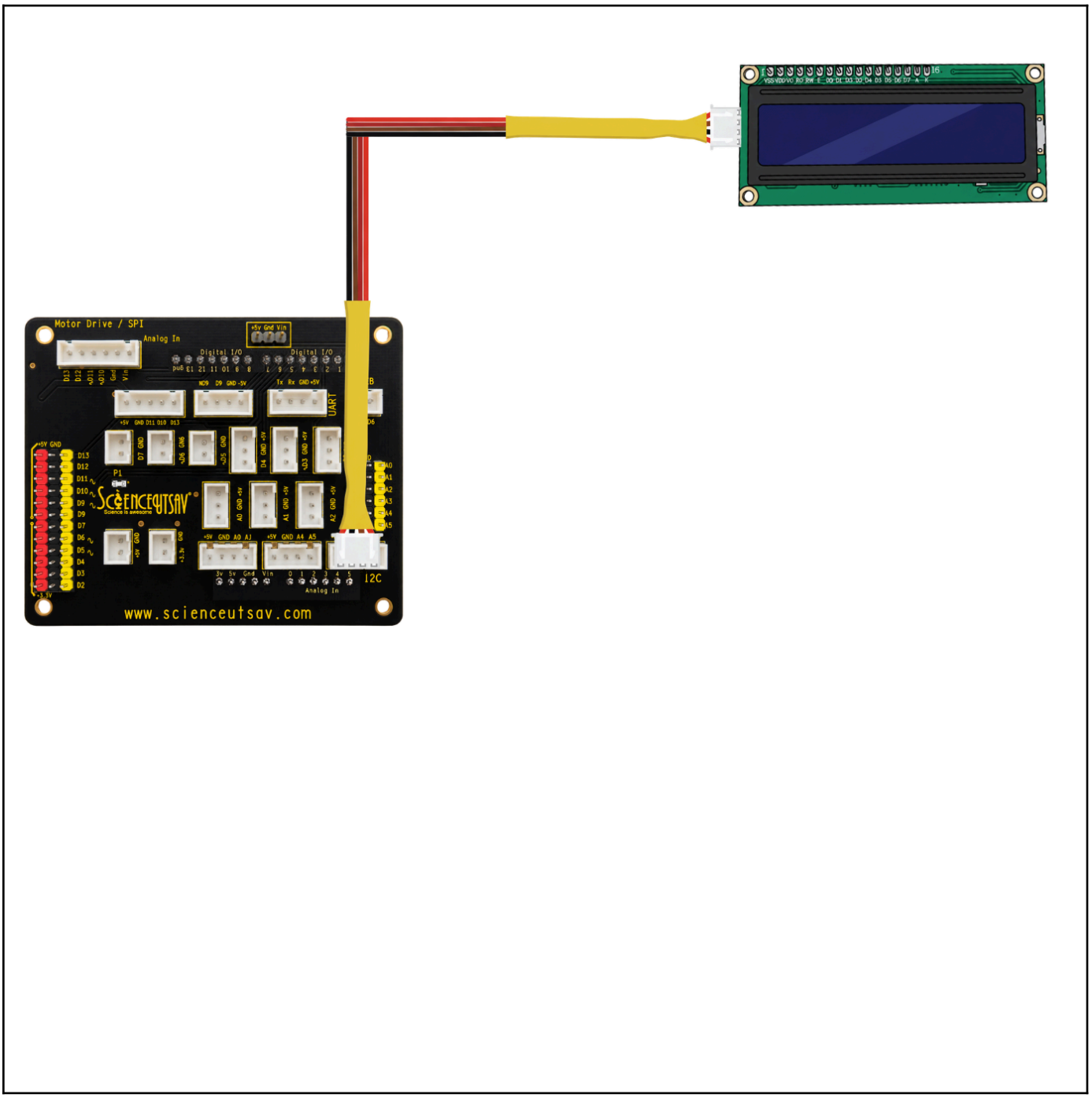
Change the displayed text automatically at regular time intervals.

Board	Arduino UNO
Components	LCD 16x2 I2C
Required libraries	Wire.h; LiquidCrystal_I2C.h
Sketch file	02_Changing_Text_Display.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.

Project 37: Changing Text Display — Circuit Diagram



Project 37: Changing Text Display— Arduino Code

Sketch file: 02_Changing_Text_Display.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);

const char* line1[] = {"Welcome", "Learn Robotics", "Think Safety"};
const char* line2[] = {"RoboScientist", "Build and Test", "Act Responsibly"};
const byte MESSAGE_COUNT = 3;
byte index = 0;
unsigned long lastChange = 0;

void showMessage() {
 lcd.clear();
 lcd.setCursor(0, 0); lcd.print(line1[index]);
 lcd.setCursor(0, 1); lcd.print(line2[index]);
}
void setup() {
 lcd.init(); lcd.backlight();
 showMessage();
}
void loop() {
 if (millis() - lastChange >= 2500) {
   lastChange = millis();
   index = (index + 1) % MESSAGE_COUNT;
   showMessage();
 }
}
```

PROJECT 38

Operator Wellness Alert

Unlock a machinery-room door only when the operator heart rate is in the healthy range.

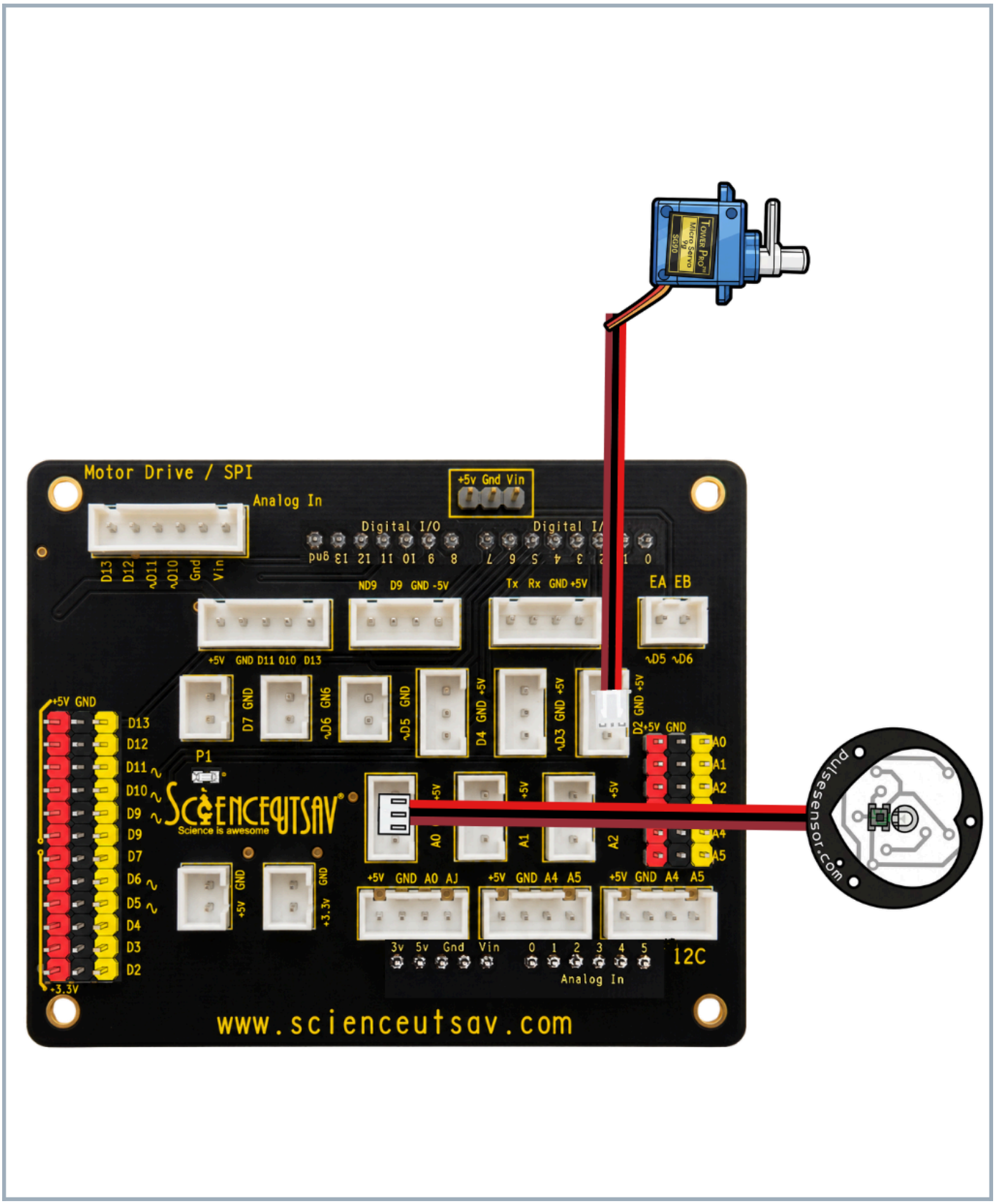
Board	Arduino UNO
Components	Pulse sensor, servo motor, buzzer
Required libraries	Servo.h; PulseSensorPlayground.h
Sketch file	38_Operator_Wellness_Alert.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
Servo motor	Signal	3	Control signal from Arduino.
Servo motor	VCC	5V / external 5V	External supply is preferred for reliable operation.
Servo motor	GND	GND	Common ground with Arduino.
Pulse sensor	Signal	A0	Analog signal input; educational use only, not a medical device.
Pulse sensor	VCC	5V	Check the sensor module marking.
Pulse sensor	GND	GND	Common ground.
Buzzer	Positive / signal	5	Negative terminal to GND.

Important: The pulse-sensor output is for educational demonstration only and must not be used for medical decisions.

Project 38: Operator Wellness Alert — Circuit Diagram



Project 38: Operator Wellness Alert — Arduino Code

Sketch file: 38_Operator_Wellness_Alert.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO

 Pulse Sensor Signal: A0
 Servo Signal: Pin 3
*/

#include <Servo.h>
#include <PulseSensorPlayground.h>

Servo door;
PulseSensorPlayground pulse;

const byte PULSE_PIN = A0;
const byte SERVO_PIN = 2;

const int MIN_BPM = 55;
const int MAX_BPM = 105;

void setup() {
 door.attach(SERVO_PIN);
 door.write(0);

 pulse.analogInput(PULSE_PIN);
 pulse.setThreshold(550);
 pulse.begin();
}

void loop() {
 if (pulse.sawStartOfBeat()) {
  int bpm = pulse.getBeatsPerMinute();

  bool healthy = bpm >= MIN_BPM && bpm <= MAX_BPM;

  if (healthy) {
   door.write(90);  // Open door
  } else {
   door.write(0);   // Close door
  }
 }

 delay(10);
}
```

PROJECT 39

Clock with Heart Rate

Display the RTC clock and live heart rate together on the LCD.

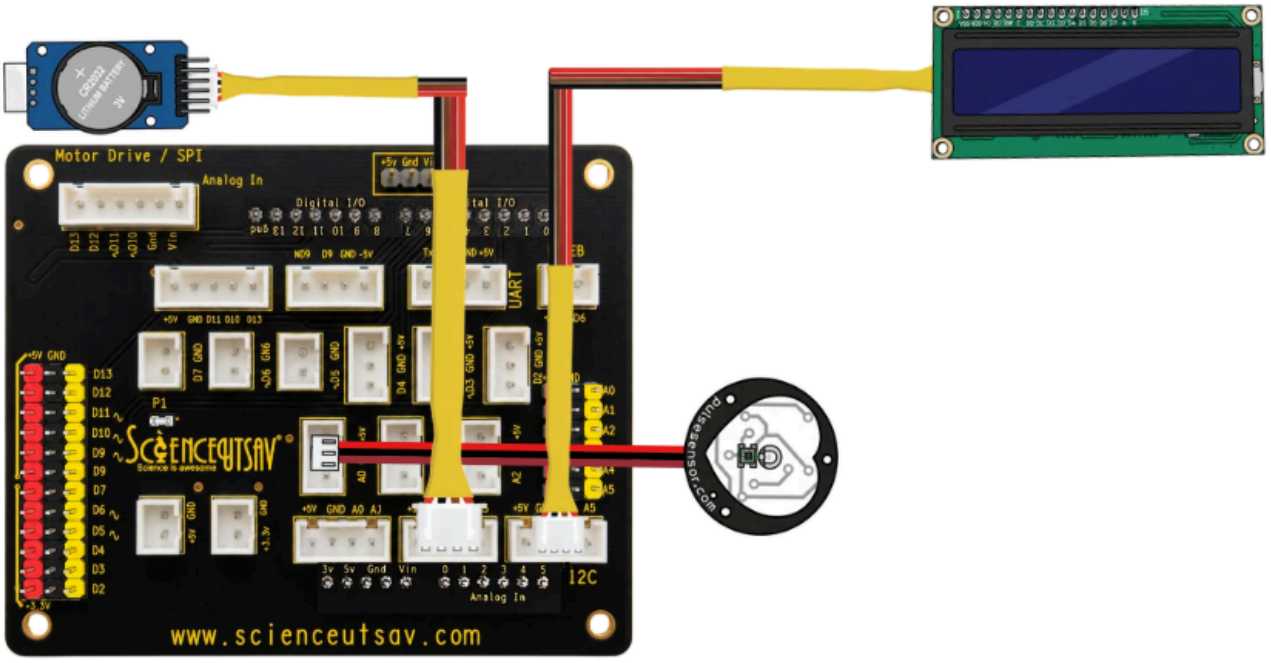
Board	Arduino UNO
Components	DS3231 RTC, pulse sensor, LCD 16x2 I2C
Required libraries	Wire.h; LiquidCrystal_I2C.h; RTClib.h; PulseSensorPlayground.h
Sketch file	39_Clock_with_Heart_Rate.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
DS3231/DS1307 RTC	VCC	5V	Use the module supply rating.
DS3231/DS1307 RTC	GND	GND	Common ground.
DS3231/DS1307 RTC	SDA	A4	Shares the I2C bus with the LCD.
DS3231/DS1307 RTC	SCL	A5	Shares the I2C bus with the LCD.
Pulse sensor	Signal	A0	Analog signal input; educational use only, not a medical device.
Pulse sensor	VCC	5V	Check the sensor module marking.
Pulse sensor	GND	GND	Common ground.

Important: The pulse-sensor output is for educational demonstration only and must not be used for medical decisions.

Project 39: Clock with Heart Rate — Circuit Diagram



Project 39: Clock with Heart Rate — Arduino Code

Sketch file: 39_Clock_with_Heart_Rate.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

// Pulse Sensor signal must connect to an ANALOG input. This sketch uses A0.

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <RTClib.h>
#include <PulseSensorPlayground.h>
LiquidCrystal_I2C lcd(0x27,16,2);
RTC_DS3231 rtc;
PulseSensorPlayground pulse;
const byte PULSE_PIN=A0;
int bpm=0;
unsigned long lastDisplay=0;
void setup(){
 lcd.init();lcd.backlight();
 if(!rtc.begin()){lcd.print("RTC error");while(true);}
 pulse.analogInput(PULSE_PIN);pulse.setThreshold(550);pulse.begin();
}
void loop(){
 if(pulse.sawStartOfBeat())bpm=pulse.getBeatsPerMinute();
 if(millis()-lastDisplay>=250){
  lastDisplay=millis();DateTime now=rtc.now();
  char line1[17];
  snprintf(line1,sizeof(line1),"Time %02d:%02d:%02d",now.hour(),now.minute(),now.second());
  lcd.setCursor(0,0);lcd.print(line1);
  lcd.setCursor(0,1);lcd.print("Heart ");lcd.print(bpm);lcd.print(" BPM    ");
 }
 delay(5);
}
```

PROJECT 40

Patient Reminder System

Remind a patient to take medication and show the live heart rate on the LCD.

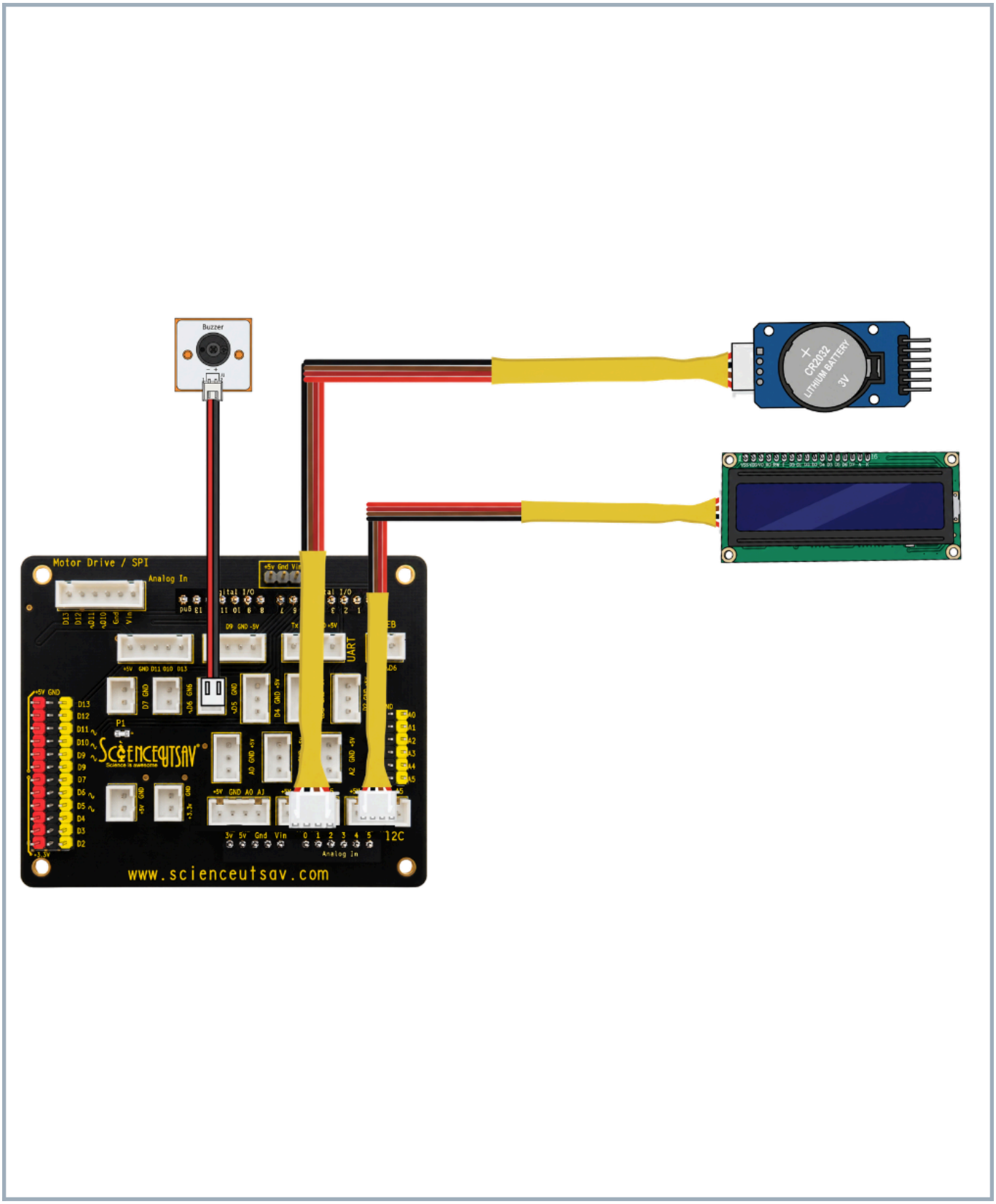
Board	Arduino UNO
Components	DS3231 RTC, pulse sensor, LCD 16x2 I2C, buzzer
Required libraries	Wire.h; LiquidCrystal_I2C.h; RTClib.h; PulseSensorPlayground.h
Sketch file	40_Patient_Reminder_System.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
DS3231/DS1307 RTC	VCC	5V	Use the module supply rating.
DS3231/DS1307 RTC	GND	GND	Common ground.
DS3231/DS1307 RTC	SDA	A4	Shares the I2C bus with the LCD.
DS3231/DS1307 RTC	SCL	A5	Shares the I2C bus with the LCD.
Pulse sensor	Signal	A0	Analog signal input; educational use only, not a medical device.
Pulse sensor	VCC	5V	Check the sensor module marking.
Pulse sensor	GND	GND	Common ground.
Buzzer	Positive / signal	5	Negative terminal to GND.

Important: The pulse-sensor output is for educational demonstration only and must not be used for medical decisions.

Project 40: Patient Reminder System — Circuit Diagram



Project 40: Patient Reminder System — Arduino Code

Sketch file: 40_Patient_Reminder_System.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

// Pulse Sensor signal must connect to an ANALOG input. This sketch uses A0.

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <RTClib.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
RTC_DS3231 rtc;

const byte BUZZER = 5;

// Medicine reminder times
const byte REMINDER_HOURS[] = {8, 14, 20};
const byte REMINDER_COUNT = 3;

int lastReminderDay = -1;
int lastReminderHour = -1;

void setup() {

  pinMode(BUZZER, OUTPUT);

  lcd.init();
  lcd.backlight();

  if (!rtc.begin()) {
    lcd.print("RTC error");
    while (true);
  }

  lcd.print("Medicine");
  lcd.setCursor(0, 1);
  lcd.print("Reminder Ready");

  delay(1500);
}

void loop() {

  DateTime now = rtc.now();

  // Check reminder times
  for (byte i = 0; i < REMINDER_COUNT; i++) {

    if (now.hour() == REMINDER_HOURS[i] &&
        now.minute() == 0 &&
        !(lastReminderDay == now.day() &&
          lastReminderHour == now.hour())) {

      lcd.clear();
      lcd.print("TAKE MEDICINE!");

      // Buzzer beeps 5 times
      for (byte j = 0; j < 5; j++) {
        tone(BUZZER, 1600, 200);
        delay(350);
      }

      lastReminderDay = now.day();
      lastReminderHour = now.hour();
    }
  }

  // Display current time
  char line1[17];

  snprintf(
    line1,
    sizeof(line1),
    "Time %02d:%02d:%02d",
    now.hour(),
    now.minute(),
    now.second()
  );

  lcd.setCursor(0, 0);
  lcd.print(line1);

  lcd.setCursor(0, 1);
  lcd.print("Take Medicine   ");

  delay(100);
```

```
}
```

PROJECT 41

Direction / Speed Control

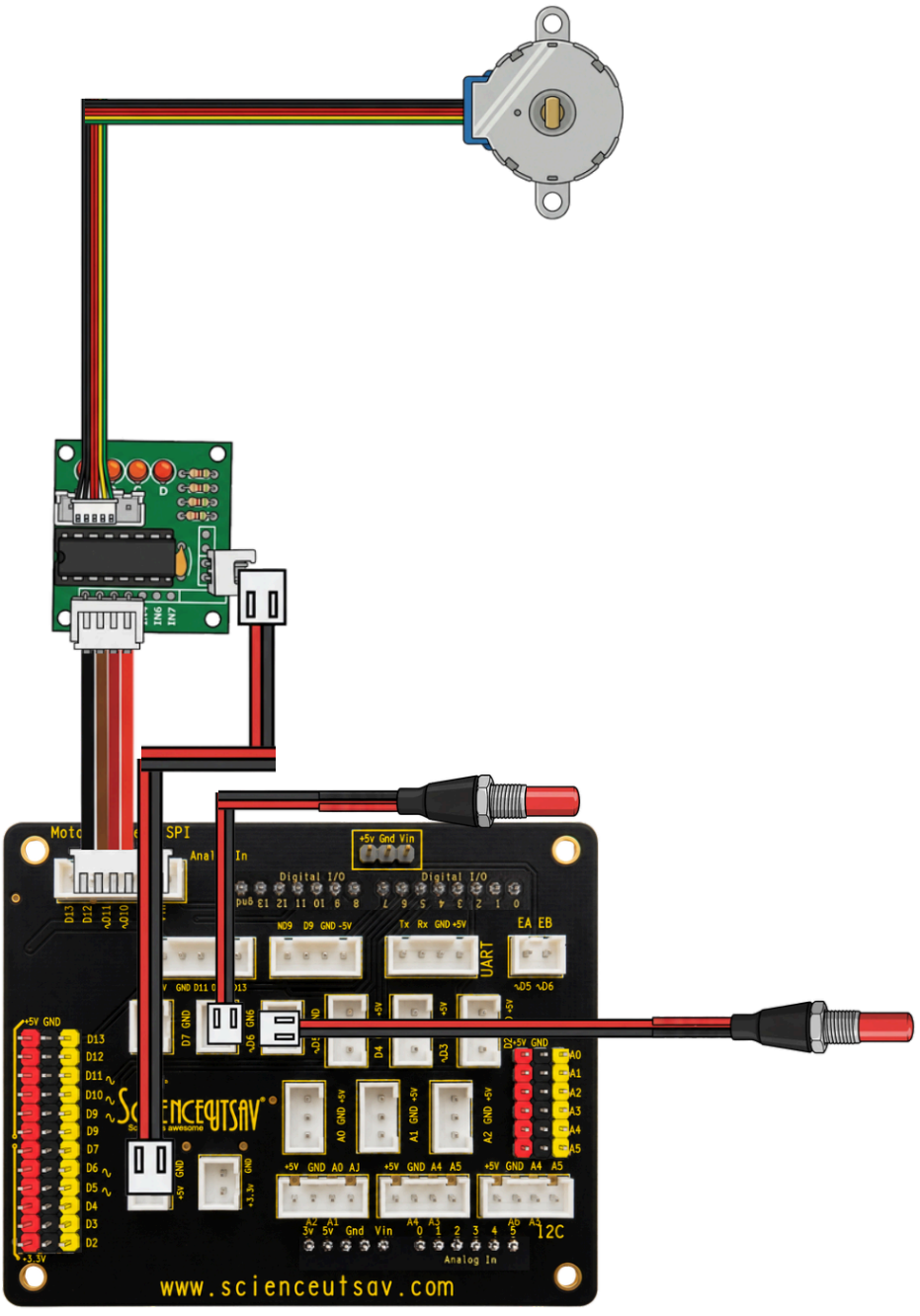
Change the stepper motor direction and speed using two push buttons.

Board	Arduino UNO
Components	Stepper motor with driver, 2 push buttons
Required libraries	Stepper.h
Sketch file	41_Direction_Speed_Control.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
ULN2003 stepper driver	IN1	10	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN2	11	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN3	12	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN4	13	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	GND	GND	Join motor-supply ground and Arduino ground.
Direction Button	Signal terminal	5	Other terminal to GND; code uses INPUT_PULLUP unless noted.
Speed Button	Signal terminal	6	Other terminal to GND; code uses INPUT_PULLUP unless noted.

Project 41: Direction / Speed Control — Circuit Diagram



Project 41: Direction / Speed Control — Arduino Code

Sketch file: 41_Direction_Speed_Control.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Stepper.h>
const int STEPS_PER_REV=2048; // 28BYJ-48 with ULN2003; calibrate for your motor.
// Your shield pins are IN1=D10, IN2=D11, IN3=D12, IN4=D13.
// The Stepper library sequence for 28BYJ-48 is commonly IN1,IN3,IN2,IN4:
Stepper motor(STEPS_PER_REV,10,12,11,13);

const byte DIRECTION_BUTTON=5;
const byte SPEED_BUTTON=6;
int direction=1;
int rpm=8;
bool dirPrev=HIGH,speedPrev=HIGH;
void setup(){
 pinMode(DIRECTION_BUTTON,INPUT_PULLUP);
 pinMode(SPEED_BUTTON,INPUT_PULLUP);
 motor.setSpeed(rpm);
}
void loop(){
 bool d=digitalRead(DIRECTION_BUTTON),s=digitalRead(SPEED_BUTTON);
 if(dirPrev==HIGH && d==LOW)direction=-direction;
 if(speedPrev==HIGH && s==LOW){
  rpm+=4;if(rpm>20)rpm=4;
  motor.setSpeed(rpm);
 }
 dirPrev=d;speedPrev=s;
 motor.step(direction*16);
}
```

PROJECT 42

Motor Positioning

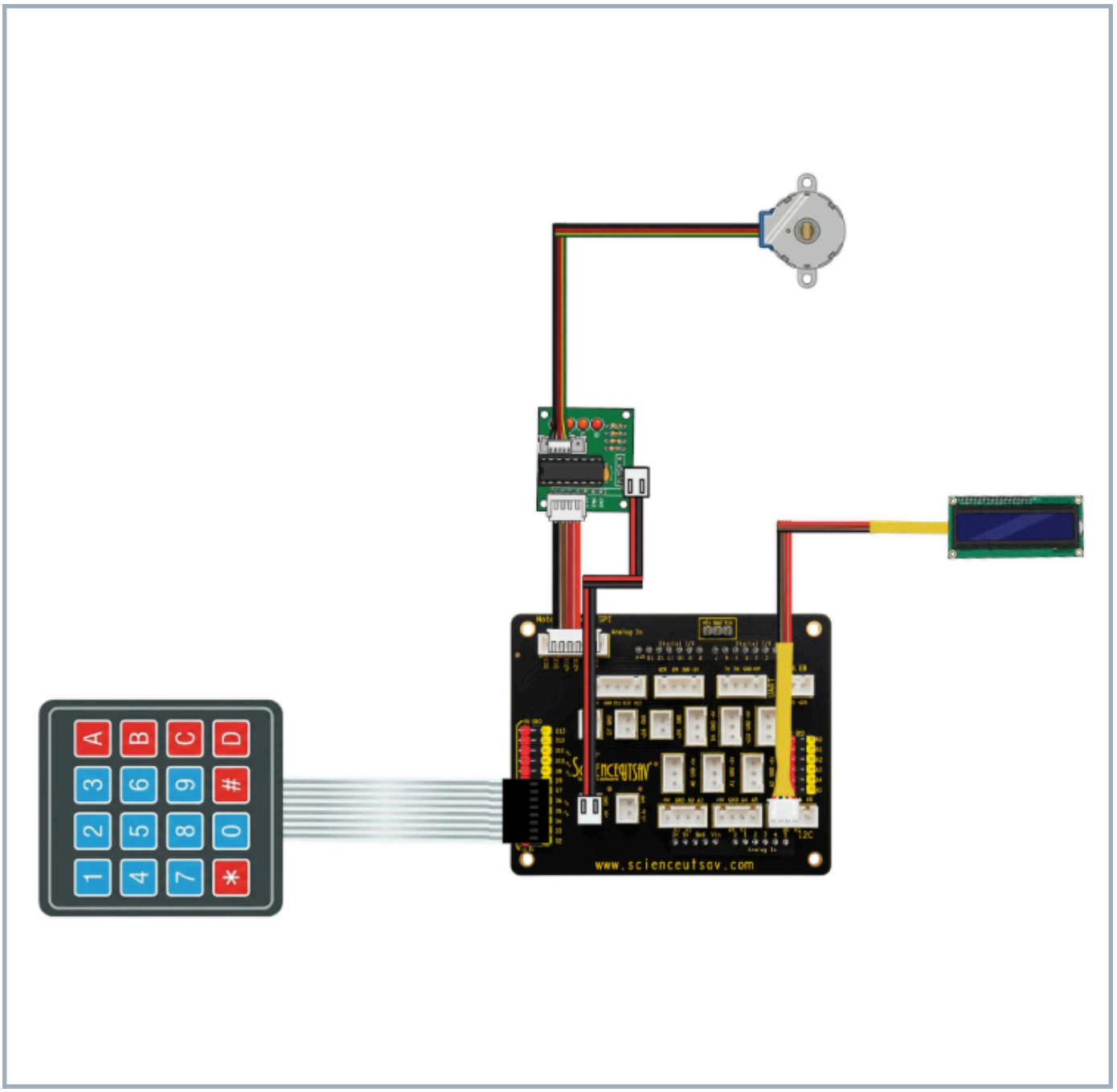
Enter an angle on the keypad and move the stepper motor to the requested position.

Board	Arduino UNO
Components	LCD 16x2 I2C, 4x4 keypad, stepper motor with driver
Required libraries	Wire.h; LiquidCrystal_I2C.h; Keypad.h; Stepper.h
Sketch file	42_Motor_Positioning.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
4x4 keypad	R1	2	Keypad row wire.
4x4 keypad	R2	3	Keypad row wire.
4x4 keypad	R3	4	Keypad row wire.
4x4 keypad	R4	5	Keypad row wire.
4x4 keypad	C1	6	Keypad column wire.
4x4 keypad	C2	7	Keypad column wire.
4x4 keypad	C3	8	Keypad column wire.
4x4 keypad	C4	9	Keypad column wire.
ULN2003 stepper driver	IN1	10	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN2	11	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN3	12	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN4	13	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	GND	GND	Join motor-supply ground and Arduino ground.

Project 42: Motor Positioning — Circuit Diagram



Project 42: Motor Positioning — Arduino Code

Sketch file: 42_Motor_Positioning.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO

 LCD I2C:
 SDA -> A4
 SCL -> A5
 Address -> 0x27

 Keypad:
 A -> Confirm angle
 D -> Clear and start again
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
#include <Stepper.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

const int STEPS_PER_REV = 2048;

// Stepper library pin sequence: IN1, IN3, IN2, IN4
Stepper motor(STEPS_PER_REV, 10, 12, 11, 13);

long currentSteps = 0;
String input = "";

const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
 {'1', '2', '3', 'A'},
 {'4', '5', '6', 'B'},
 {'7', '8', '9', 'C'},
 {'*', '0', '#', 'D'}
};

byte rowPins[ROWS] = {2, 3, 4, 5};
byte colPins[COLS] = {6, 7, 8, 9};

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

void prompt() {
 input = "";

 lcd.clear();
 lcd.print("Enter angle:");
 lcd.setCursor(0, 1);
```

```
}

void moveToAngle(int angle) {
 long targetSteps = (long)angle * STEPS_PER_REV / 360;
 long difference = targetSteps - currentSteps;

 // Select the shortest rotation direction
 difference =
    ((difference % STEPS_PER_REV) + STEPS_PER_REV) % STEPS_PER_REV;

 if (difference > STEPS_PER_REV / 2) {
  difference -= STEPS_PER_REV;
 }

 motor.step(difference);

 currentSteps = (currentSteps + difference) % STEPS_PER_REV;

 if (currentSteps < 0) {
  currentSteps += STEPS_PER_REV;
 }
}

void setup() {
 motor.setSpeed(12);

 lcd.init();
 lcd.backlight();

 prompt();
}

void loop() {
 char key = keypad.getKey();

 if (!key) {
  return;
 }

 // D clears the entered angle and starts again
 if (key == 'D') {
  prompt();
 }

 // A confirms the entered angle
 else if (key == 'A') {
  if (input.length() == 0) {
   lcd.clear();
   lcd.print("Enter an angle");
   delay(1000);
   prompt();
   return;
  }

  int rawAngle = input.toInt();
  int angle = rawAngle % 360;
```

```
  if (angle < 0) {
    angle += 360;
  }

  moveToAngle(angle);

  lcd.clear();
  lcd.print("Position: ");
  lcd.print(angle);
  lcd.print((char)223);

  delay(4000);

  moveToAngle(0);

  lcd.clear();
  lcd.print("Returned to 0");
  lcd.print((char)223);

  delay(800);

  prompt();
 }

 // Accept only three numeric digits
 else if (isDigit(key) && input.length() < 3) {
  input += key;
  lcd.print(key);
 }
}
```

PROJECT 43

Simple Password Door

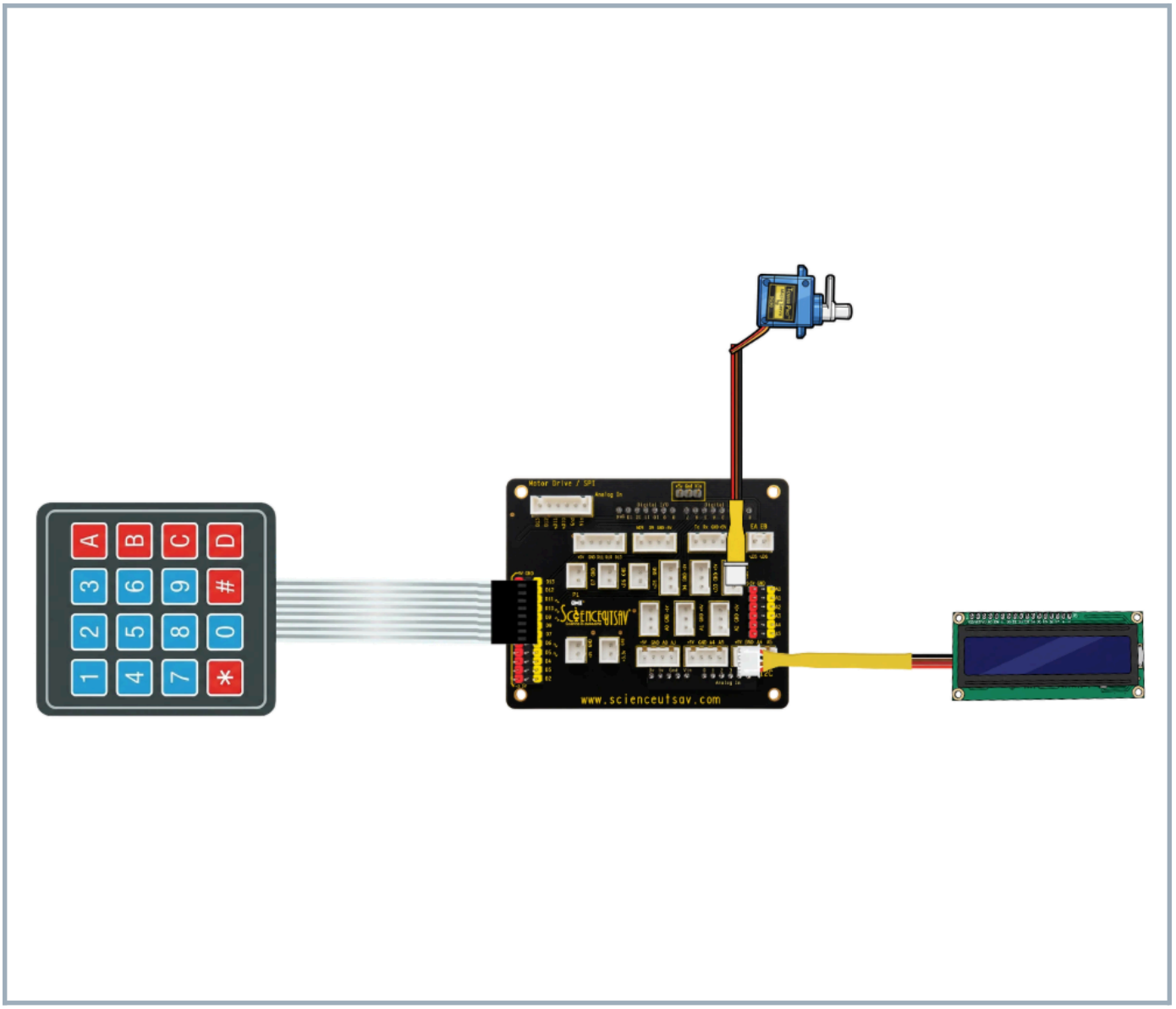
Unlock a model door only after the correct password is entered.

Board	Arduino UNO
Components	4x4 keypad, servo motor
Required libraries	Keypad.h; Servo.h
Sketch file	14_Simple_Password_Door.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
4x4 keypad	R1	6	Keypad row wire.
4x4 keypad	R2	7	Keypad row wire.
4x4 keypad	R3	8	Keypad row wire.
4x4 keypad	R4	9	Keypad row wire.
4x4 keypad	C1	10	Keypad column wire.
4x4 keypad	C2	11	Keypad column wire.
4x4 keypad	C3	12	Keypad column wire.
4x4 keypad	C4	13	Keypad column wire.
Servo motor	Signal	2	Control signal from Arduino.
Servo motor	VCC	5V / external 5V	External supply is preferred for reliable operation.
Servo motor	GND	GND	Common ground with Arduino.

Project 43: Simple Password Door — Circuit Diagram



Project 43: Simple Password Door — Arduino Code

Sketch file: 14_Simple_Password_Door.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Keypad.h>
#include <Servo.h>
Servo door;
const byte SERVO_PIN = 2;
const String PASSWORD = "1234";
String entered = "";
const byte ROWS = 4, COLS = 4;
char keys[ROWS][COLS] = {
 {'1','2','3','A'}, {'4','5','6','B'},
 {'7','8','9','C'}, {'*','0','#','D'}
};
byte rowPins[ROWS] = {6,7,8,9};
byte colPins[COLS] = {10,11,12,13};
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

void setup() {
 door.attach(SERVO_PIN);
 door.write(0);
 Serial.begin(9600);
}
void loop() {
 char key = keypad.getKey();
 if (!key) return;
 if (key == '*') entered = "";
 else if (key == '#') {
  if (entered == PASSWORD) {
   Serial.println("Correct password - door open");
   door.write(90); delay(3000); door.write(0);
  } else Serial.println("Incorrect password");
  entered = "";
 } else if (entered.length() < 12) entered += key;
}
```

PROJECT 44

Vault Dial

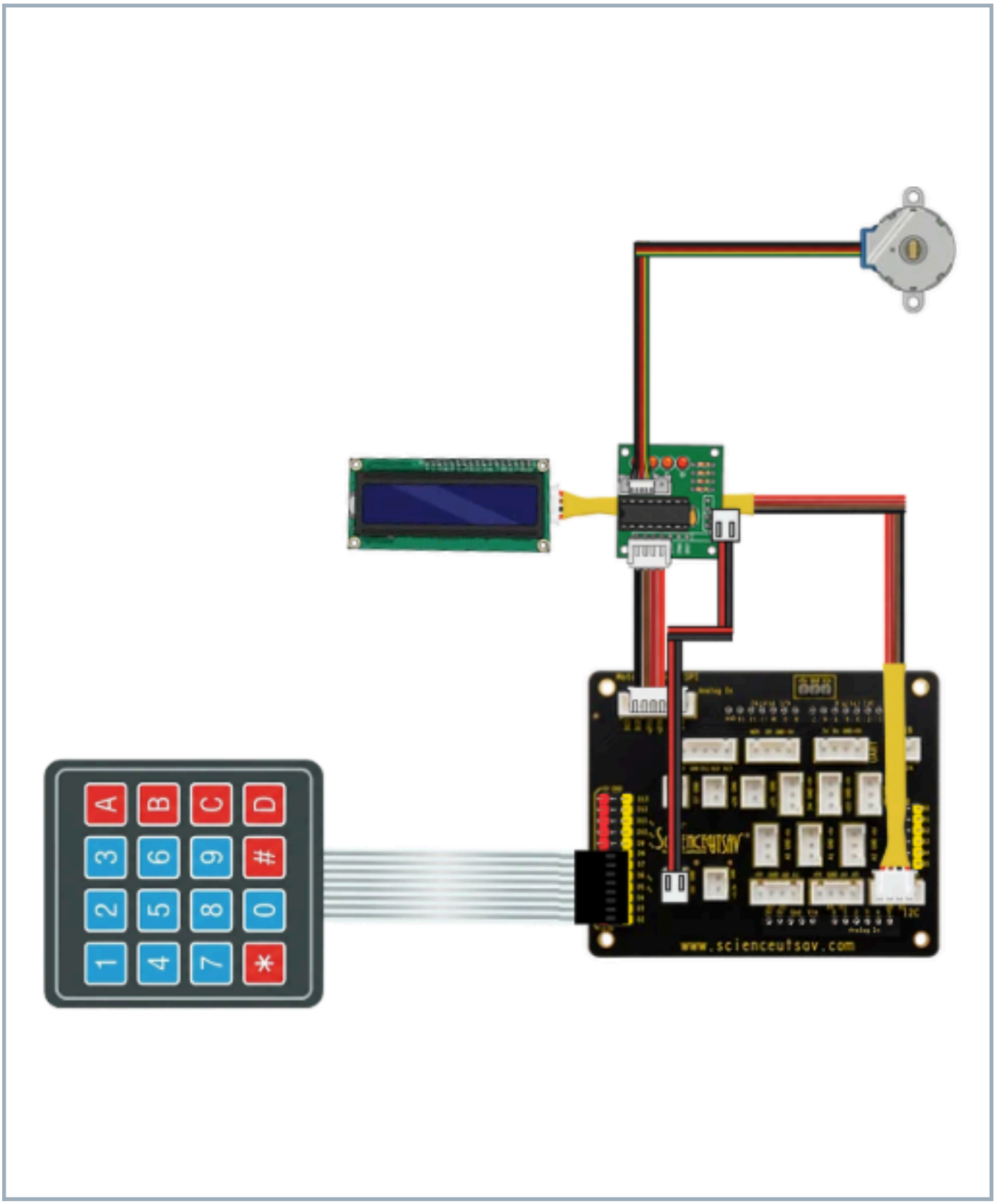
Enter a rotation angle on the keypad and rotate the motor to that position.

Board	Arduino UNO
Components	4x4 keypad, stepper motor with driver
Required libraries	Keypad.h; Stepper.h
Sketch file	44_Vault_Dial.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
4x4 keypad	R1	2	Keypad row wire.
4x4 keypad	R2	3	Keypad row wire.
4x4 keypad	R3	4	Keypad row wire.
4x4 keypad	R4	5	Keypad row wire.
4x4 keypad	C1	6	Keypad column wire.
4x4 keypad	C2	7	Keypad column wire.
4x4 keypad	C3	8	Keypad column wire.
4x4 keypad	C4	9	Keypad column wire.
ULN2003 stepper driver	IN1	10	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN2	11	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN3	12	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN4	13	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	GND	GND	Join motor-supply ground and Arduino ground.

Project 44: Vault Dial — Circuit Diagram



Project 44: Vault Dial — Arduino Code

Sketch file: 44_Vault_Dial.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Keypad.h>
#include <Stepper.h>
const int STEPS_PER_REV=2048;
Stepper dial(STEPS_PER_REV,10,12,11,13);
String input="";
long currentSteps=0;
const byte ROWS=4,COLS=4;
char keys[ROWS][COLS]={{'1','2','3','A'},{'4','5','6','B'},{'7','8','9','C'},{'*','0','#','D'}};
byte rowPins[ROWS]={2,3,4,5};
byte colPins[COLS]={6,7,8,9};
Keypad keypad=Keypad(makeKeymap(keys),rowPins,colPins,ROWS,COLS);

void setup(){Serial.begin(9600);dial.setSpeed(12);Serial.println("Enter angle then #");}
void loop(){
  char key=keypad.getKey();if(!key)return;
  if(key=='*'){input="";Serial.println("Cleared");}
  else if(key=='#'){
    int angle=constrain(input.toInt(),0,360);
    long target=(long)angle*STEPS_PER_REV/360;
    dial.step(target-currentSteps);currentSteps=target;
    Serial.print("Dial moved to ");Serial.println(angle);
    input="";
  }else if(isDigit(key)&&input.length()<3){input+=key;Serial.print(key);}
}
```

PROJECT 45

Time-Based Motor

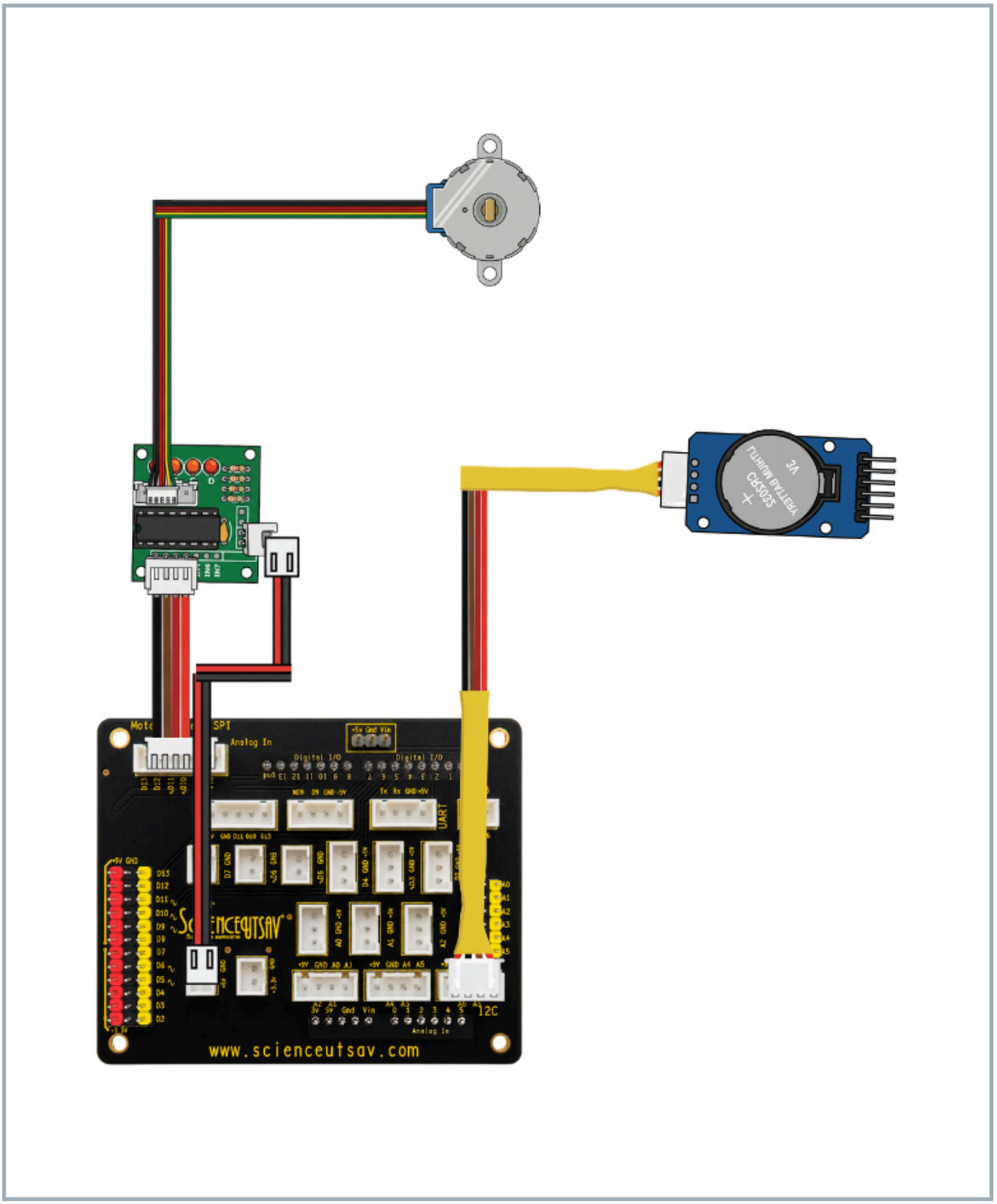
Run the stepper motor automatically at configured times of the day.

Board	Arduino UNO
Components	DS3231 RTC, stepper motor with driver
Required libraries	Wire.h; RTClib.h; Stepper.h
Sketch file	45_Time_Based_Motor.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
DS3231/DS1307 RTC	VCC	5V	Use the module supply rating.
DS3231/DS1307 RTC	GND	GND	Common ground.
DS3231/DS1307 RTC	SDA	A4	Shares the I2C bus with the LCD.
DS3231/DS1307 RTC	SCL	A5	Shares the I2C bus with the LCD.
ULN2003 stepper driver	IN1	10	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN2	11	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN3	12	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN4	13	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	GND	GND	Join motor-supply ground and Arduino ground.

Project 45: Time-Based Motor — Circuit Diagram



Project 45: Time-Based Motor — Arduino Code

Sketch file: 45_Time_Based_Motor.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <RTClib.h>
#include <Stepper.h>
RTC_DS3231 rtc;
const int STEPS_PER_REV=2048;
Stepper motor(STEPS_PER_REV,10,12,11,13);
const byte RUN_HOURS[]={9,13,17};
const byte RUN_COUNT=3;
int lastRunDay=-1,lastRunHour=-1;
void setup(){
 Serial.begin(9600);motor.setSpeed(12);
 if(!rtc.begin()){Serial.println("RTC error");while(true);}
}
void loop(){
 DateTime now=rtc.now();
 for(byte i=0;i<RUN_COUNT;i++){
  if(now.hour()==RUN_HOURS[i]&&now.minute()==0&&
     !(lastRunDay==now.day()&&lastRunHour==now.hour())){
    Serial.println("Scheduled motor rotation");
    motor.step(STEPS_PER_REV/2);
    lastRunDay=now.day();lastRunHour=now.hour();
  }
 }
 delay(500);
}
```

PROJECT 46

RFID Conveyor Belt System

Start or stop a stepper-motor conveyor according to the RFID tag scanned.

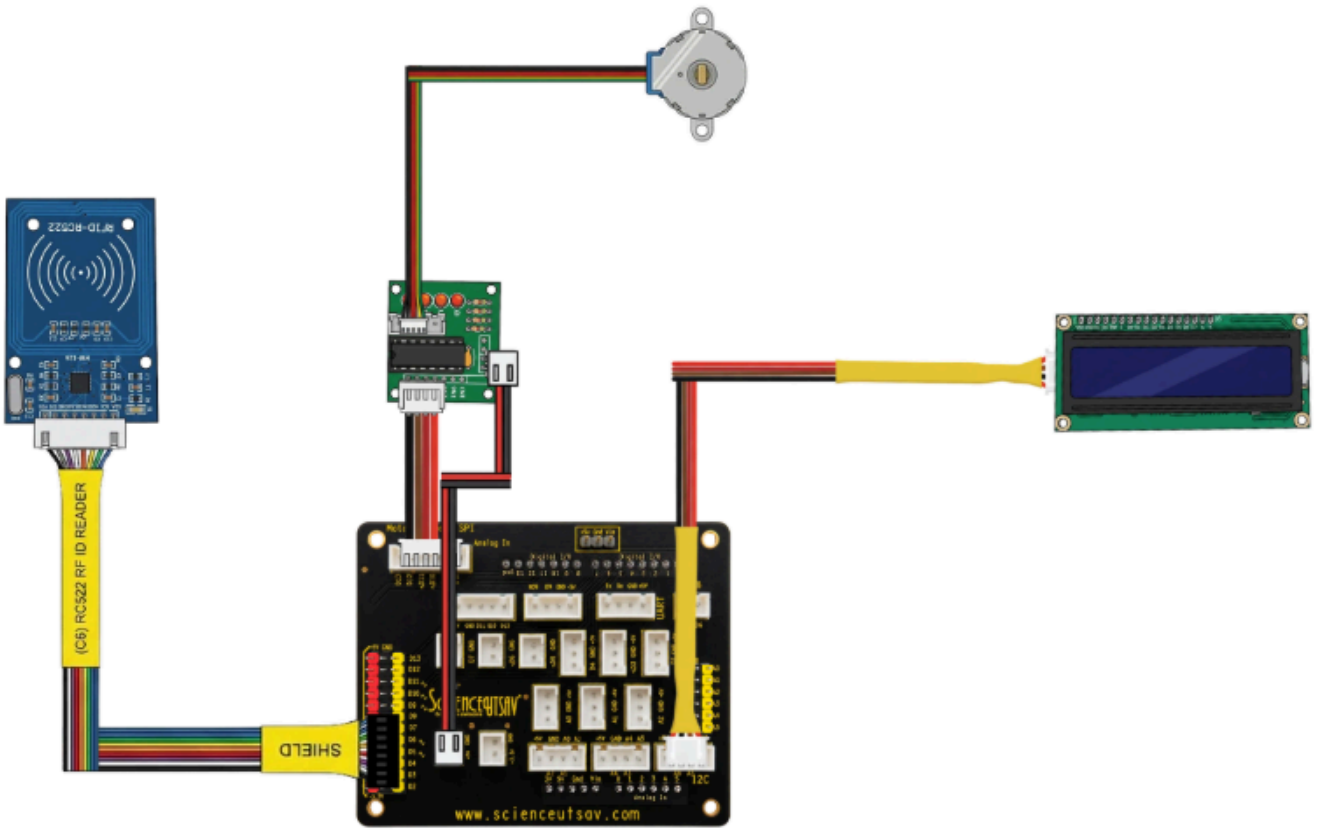
Board	Arduino UNO
Components	RC522 RFID reader, LCD 16x2 I2C, stepper motor with driver
Required libraries	Wire.h; LiquidCrystal_I2C.h; SPI.h; MFRC522.h; Stepper.h
Sketch file	46_RFID_Conveyor_Belt_System.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
RC522 RFID reader	SDA / SS	D10	SPI chip-select line.
RC522 RFID reader	SCK	D13	SPI clock.
RC522 RFID reader	MOSI	D11	SPI data from UNO to RC522.
RC522 RFID reader	MISO	D12	SPI data from RC522 to UNO.
RC522 RFID reader	RST	D8	Reset pin.
RC522 RFID reader	3.3V	UNO 3.3V	Do not connect directly to a 5 V GPIO output.
RC522 RFID reader	GND	GND	Use the actual ground pin. D9 LOW is only an optional shield arrangement.
ULN2003 stepper driver	IN1	2	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN2	3	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN3	4	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN4	5	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	GND	GND	Join motor-supply ground and Arduino ground.

Important: Use Project 18 to read each card UID, then replace the sample UIDs in this sketch.

Project 46: RFID Conveyor Belt System — Circuit Diagram



Project 46: RFID Conveyor Belt System — Arduino Code

Sketch file: 46_RFID_Conveyor_Belt_System.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

/*
  Combined RFID + stepper project:
  RC522 keeps D8-D13. Move ULN2003 IN1-IN4 to D2,D3,D4,D5.
*/
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SPI.h>
#include <MFRC522.h>
#include <Stepper.h>
LiquidCrystal_I2C lcd(0x27,16,2);
MFRC522 rfid(10,8);
const int STEPS_PER_REV=2048;
Stepper conveyor(STEPS_PER_REV,2,4,3,5);
bool running=false;

// IMPORTANT: Connect RC522 VCC to the UNO 3.3V supply.
// Do not connect RC522 VCC directly to a GPIO HIGH; UNO GPIO HIGH is about 5V.
// Set this true only if your shield has a real 3.3V regulator between D7 and RC522 VCC.
const bool SHIELD_HAS_3V3_REGULATOR = false;
const byte RFID_POWER_PIN = 7;
const byte RFID_GROUND_PIN = 9;
void setupRfidPower() {
  if (SHIELD_HAS_3V3_REGULATOR) {
    pinMode(RFID_GROUND_PIN, OUTPUT); digitalWrite(RFID_GROUND_PIN, LOW);
    pinMode(RFID_POWER_PIN, OUTPUT); digitalWrite(RFID_POWER_PIN, HIGH);
    delay(100);
  }
}

String uidText(){
  String s="";
  for(byte i=0;i<rfid.uid.size;i++){if(rfid.uid.uidByte[i]<16)s+="0";s+=String(rfid.uid.uidByte[i],HEX);}
  s.toUpperCase();return s;
}
void setup(){
  Serial.begin(9600);lcd.init();lcd.backlight();
  conveyor.setSpeed(12);
  setupRfidPower();SPI.begin();rfid.PCD_Init();
  lcd.print("Tap START/STOP");
}
void loop(){
  if(rfid.PICC_IsNewCardPresent()&&rfid.PICC_ReadCardSerial()){
    String uid=uidText();
    if(uid=="A1B2C3D4"){
      running=!running;
      lcd.clear();lcd.print(running?"Conveyor RUNNING":"Conveyor STOPPED");
    }else{
      lcd.clear();lcd.print("Unknown tag");
      delay(1000);lcd.clear();lcd.print(running?"Conveyor RUNNING":"Conveyor STOPPED");
    }
    rfid.PICC_HaltA();rfid.PCD_StopCrypto1();delay(500);
  }
  if(running)conveyor.step(8);
}
```

PROJECT 47

Industry Machinery Setup

Start the simulated industrial machine only after a valid RFID tag is scanned.

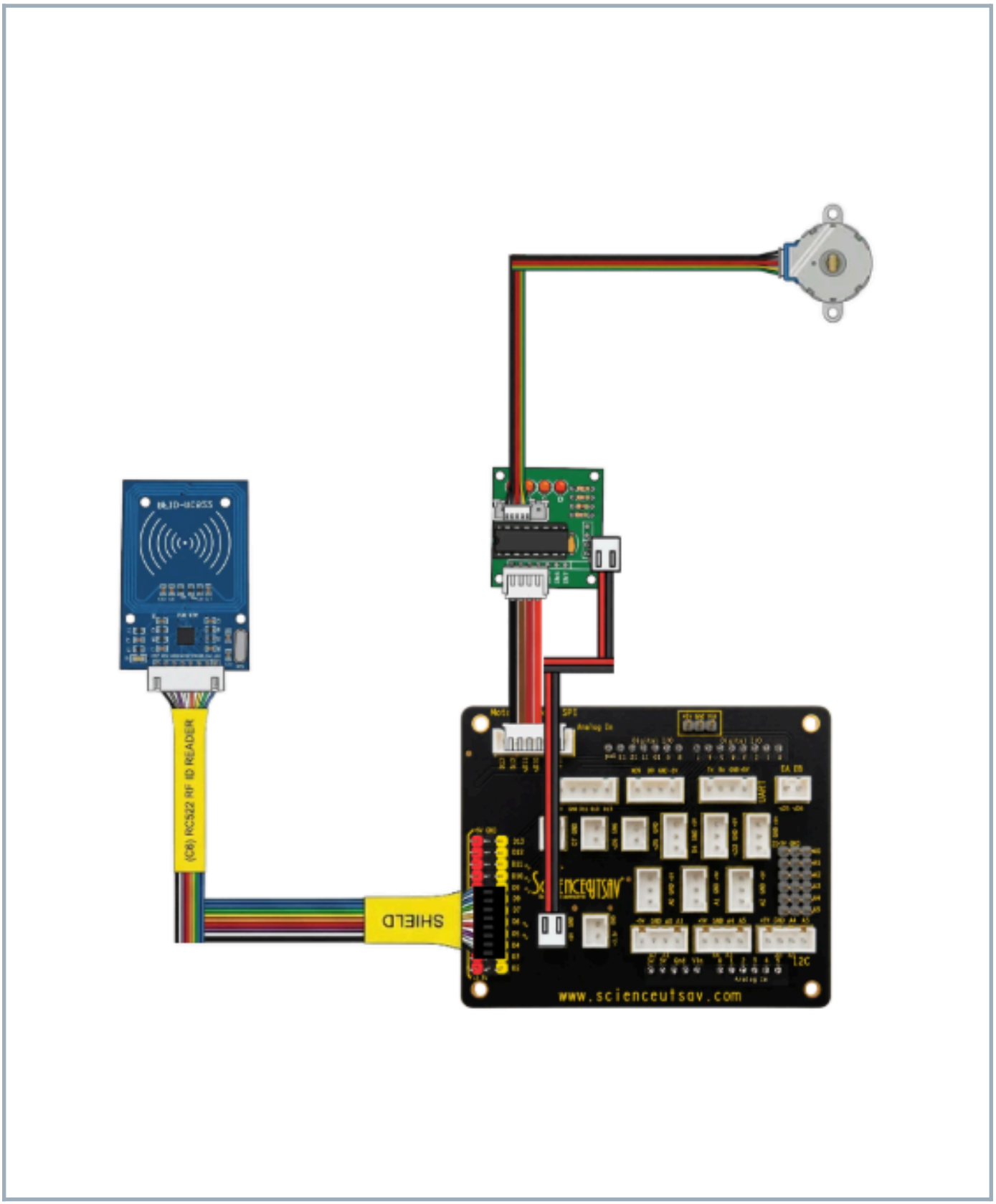
Board	Arduino UNO
Components	RC522 RFID reader, stepper motor with driver
Required libraries	SPI.h; MFRC522.h; Stepper.h
Sketch file	47_Industry_Machinery_Setup.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
RC522 RFID reader	SDA / SS	3	SPI chip-select line.
RC522 RFID reader	SCK	4	SPI clock.
RC522 RFID reader	MOSI	5	SPI data from UNO to RC522.
RC522 RFID reader	MISO	6	SPI data from RC522 to UNO.
RC522 RFID reader	RST	7	Reset pin.
RC522 RFID reader	3.3V	8	Do not connect directly to a 5 V GPIO output.
RC522 RFID reader	GND	9	Use the actual ground pin. D9 LOW is only an optional shield arrangement.
ULN2003 stepper driver	IN1	10	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN2	11	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN3	12	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN4	13	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	GND	GND	Join motor-supply ground and Arduino ground.

Important: Use Project 18 to read each card UID, then replace the sample UIDs in this sketch.

Project 47: Industry Machinery Setup — Circuit Diagram



Project 47: Industry Machinery Setup — Arduino Code

Sketch file: 47_Industry_Machinery_Setup.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

/*
  A valid RFID tag starts a demonstration stepper-machine cycle.
  Combined wiring: RC522 D8-D13; ULN2003 moved to D2-D5.
*/
#include <SPI.h>
#include <MFRC522.h>
#include <Stepper.h>
MFRC522 rfid(10,8);
const int STEPS_PER_REV=2048;
Stepper machine(STEPS_PER_REV,2,4,3,5);

// IMPORTANT: Connect RC522 VCC to the UNO 3.3V supply.
// Do not connect RC522 VCC directly to a GPIO HIGH; UNO GPIO HIGH is about 5V.
// Set this true only if your shield has a real 3.3V regulator between D7 and RC522 VCC.
const bool SHIELD_HAS_3V3_REGULATOR = false;
const byte RFID_POWER_PIN = 7;
const byte RFID_GROUND_PIN = 9;
void setupRfidPower() {
  if (SHIELD_HAS_3V3_REGULATOR) {
    pinMode(RFID_GROUND_PIN, OUTPUT); digitalWrite(RFID_GROUND_PIN, LOW);
    pinMode(RFID_POWER_PIN, OUTPUT); digitalWrite(RFID_POWER_PIN, HIGH);
    delay(100);
  }
}

String uidText(){
  String s="";
  for(byte i=0;i<rfid.uid.size;i++){if(rfid.uid.uidByte[i]<16)s+="0";s+=String(rfid.uid.uidByte[i],HEX);}
  s.toUpperCase();return s;
}
bool validTag(const String& uid){return uid=="A1B2C3D4"||uid=="11223344";}
void runMachine(){
  Serial.println("MACHINE_START");
  machine.step(STEPS_PER_REV);
  delay(1000);
  Serial.println("MACHINE_STOP");
}
void setup(){
  Serial.begin(9600);machine.setSpeed(12);
  setupRfidPower();SPI.begin();rfid.PCD_Init();
}
void loop(){
  if(!rfid.PICC_IsNewCardPresent()||!rfid.PICC_ReadCardSerial())return;
  String uid=uidText();
  if(validTag(uid))runMachine();else Serial.println("ACCESS_DENIED");
  rfid.PICC_HaltA();rfid.PCD_StopCrypto1();delay(700);
}
```

PROJECT 48

Industrial Machinery Basic Secured

Start the motor only after a valid RFID scan and when the room gas level is safe.

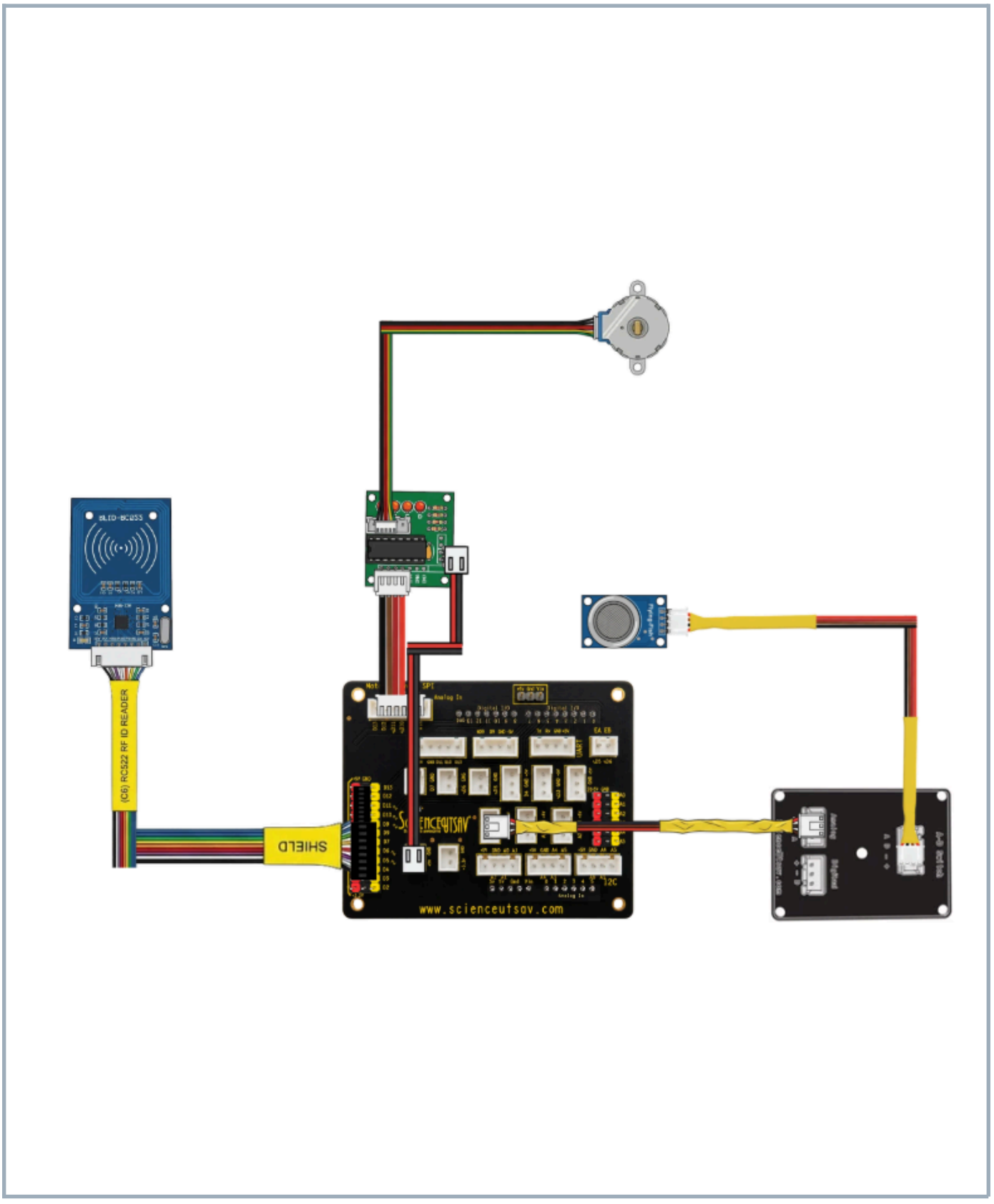
Board	Arduino UNO
Components	RC522 RFID reader, MQ2 sensor, LCD 16x2 I2C, stepper motor with driver
Required libraries	Wire.h; LiquidCrystal_I2C.h; SPI.h; MFRC522.h; Stepper.h
Sketch file	48_Industrial_Machinery_Basic_Secured.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
RC522 RFID reader	SDA / SS	3	SPI chip-select line.
RC522 RFID reader	SCK	4	SPI clock.
RC522 RFID reader	MOSI	5	SPI data from UNO to RC522.
RC522 RFID reader	MISO	6	SPI data from RC522 to UNO.
RC522 RFID reader	RST	7	Reset pin.
RC522 RFID reader	3.3V	8	Do not connect directly to a 5 V GPIO output.
RC522 RFID reader	GND	9	Use the actual ground pin. D9 LOW is only an optional shield arrangement.
ULN2003 stepper driver	IN1	10	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN2	11	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN3	12	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN4	13	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	GND	GND	Join motor-supply ground and Arduino ground.
MQ2 gas sensor	AO	A0	Analog output; warm up and calibrate before setting thresholds.
MQ2 gas sensor	VCC	5V	MQ heater modules draw significant current.
MQ2 gas sensor	GND	GND	Common ground.

Important: Calibrate the MQ sensor in the actual environment and adjust the threshold constants before demonstration.

Project 48: Industrial Machinery Basic Secured — Circuit Diagram



Project 48: Industrial Machinery Basic Secured — Arduino Code

Sketch file: 48_Industrial_Machinery_Basic_Secured.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

/*
  RC522: D8-D13. Stepper ULN2003: D2-D5. MQ2/MQ6 analog: A0.
*/
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SPI.h>
#include <MFRC522.h>
#include <Stepper.h>
LiquidCrystal_I2C lcd(0x27,16,2);
MFRC522 rfid(10,8);
const int STEPS_PER_REV=2048;
Stepper machine(STEPS_PER_REV,2,4,3,5);
const byte MQ2_PIN=A0;
const int GAS_SAFE_LIMIT=430;

// IMPORTANT: Connect RC522 VCC to the UNO 3.3V supply.
// Do not connect RC522 VCC directly to a GPIO HIGH; UNO GPIO HIGH is about 5V.
// Set this true only if your shield has a real 3.3V regulator between D7 and RC522 VCC.
const bool SHIELD_HAS_3V3_REGULATOR = false;
const byte RFID_POWER_PIN = 7;
const byte RFID_GROUND_PIN = 9;
void setupRfidPower() {
  if (SHIELD_HAS_3V3_REGULATOR) {
    pinMode(RFID_GROUND_PIN, OUTPUT); digitalWrite(RFID_GROUND_PIN, LOW);
    pinMode(RFID_POWER_PIN, OUTPUT); digitalWrite(RFID_POWER_PIN, HIGH);
    delay(100);
  }
}

String uidText(){
  String s="";
  for(byte i=0;i<rfid.uid.size;i++){if(rfid.uid.uidByte[i]<16)s+="0";s+=String(rfid.uid.uidByte[i],HEX);}
  s.toUpperCase();return s;
}
bool validTag(const String& uid){return uid=="A1B2C3D4";}
void setup(){
  lcd.init();lcd.backlight();machine.setSpeed(12);
  setupRfidPower();SPI.begin();rfid.PCD_Init();
  lcd.print("Tap operator ID");
}
void loop(){
  if(!rfid.PICC_IsNewCardPresent()||!rfid.PICC_ReadCardSerial())return;
  String uid=uidText();int gas=analogRead(MQ2_PIN);
  lcd.clear();
  if(validTag(uid)&&gas<GAS_SAFE_LIMIT){
    lcd.print("Safe: motor ON");
    machine.step(STEPS_PER_REV);
  }else{
    lcd.print(!validTag(uid)?"Invalid RFID":"Gas unsafe");
    lcd.setCursor(0,1);lcd.print("Motor blocked");
  }
  rfid.PICC_HaltA();rfid.PCD_StopCrypto1();
  delay(1800);lcd.clear();lcd.print("Tap operator ID");
}
```

PROJECT 49

Full Smart Automated Industry Setup

Combine RFID access, air-quality monitoring, heart-rate safety, RTC operating hours, stepper machinery and manual override.

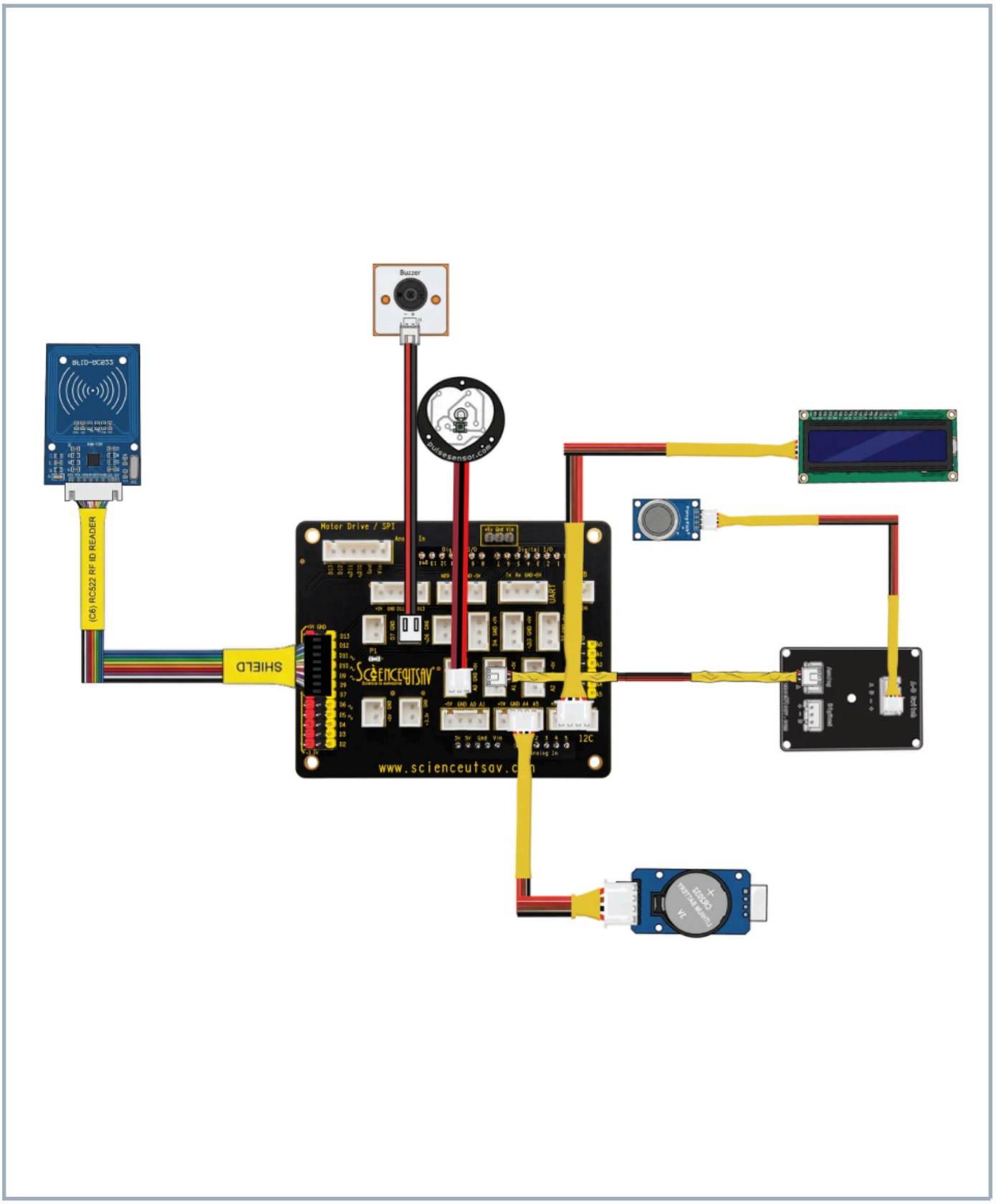
Board	Arduino UNO
Components	RC522 RFID reader, MQ135 sensor, pulse sensor, DS3231 RTC, LCD 16x2 I2C, stepper motor with driver, override button
Required libraries	Wire.h; LiquidCrystal_I2C.h; RTClib.h; SPI.h; MFRC522.h; Stepper.h; PulseSensorPlayground.h
Sketch file	49_Full_Smart_Automated_Industry_Setup_UNO.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
DS3231/DS1307 RTC	VCC	5V	Use the module supply rating.
DS3231/DS1307 RTC	GND	GND	Common ground.
DS3231/DS1307 RTC	SDA	A4	Shares the I2C bus with the LCD.
DS3231/DS1307 RTC	SCL	A5	Shares the I2C bus with the LCD.
RC522 RFID reader	SDA / SS	3	SPI chip-select line.
RC522 RFID reader	SCK	4	SPI clock.
RC522 RFID reader	MOSI	5	SPI data from UNO to RC522.
RC522 RFID reader	MISO	6	SPI data from RC522 to UNO.
RC522 RFID reader	RST	7	Reset pin.
RC522 RFID reader	3.3V	8	Do not connect directly to a 5 V GPIO output.
RC522 RFID reader	GND	9	Use the actual ground pin. D9 LOW is only an optional shield arrangement.
ULN2003 stepper driver	IN1	10	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN2	11	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN3	12	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	IN4	13	Driver input; power the motor from a suitable supply.
ULN2003 stepper driver	GND	GND	Join motor-supply ground and Arduino ground.
Pulse sensor	Signal	A1	Analog signal input; educational use only, not a medical device.
Pulse sensor	VCC	5V	Check the sensor module marking.
Pulse sensor	GND	GND	Common ground.
MQ135 air-quality sensor	AO	A0	Analog output; warm up and calibrate before setting thresholds.
MQ135 air-quality sensor	VCC	5V	MQ heater modules draw significant current.
MQ135 air-quality sensor	GND	GND	Common ground.
Override Button	Signal terminal	6	Other terminal to GND; code uses INPUT_PULLUP unless noted.

Important: Arduino UNO cannot accommodate the full original combination of RC522, four-wire stepper, 4x4 keypad, sensors, RTC and LCD without extra I/O. This UNO sketch uses one override button instead of the keypad. The code package also includes a full Arduino Mega reference version.

Project 49: Full Smart Automated Industry Setup — Circuit Diagram



Project 49: Full Smart Automated Industry Setup — Arduino Code

Sketch file: 49_Full_Smart_Automated_Industry_Setup_UNO.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

/*
  UNO LIMITATION:
  The RC522 uses D7-D13 and the stepper uses D2-D5. LCD+RTC use A4/A5.
  After MQ135 and Pulse Sensor are connected, an 8-wire 4x4 keypad cannot fit.
  This UNO version uses D6 as a single manual override button. For the exact
  full system with a 4x4 keypad, use the included Mega reference sketch or add
  an I2C keypad/GPIO expander.

  Wiring:
  RC522 D8-D13; stepper ULN2003 D2-D5; override D6;
  MQ135 A0; Pulse Sensor A1; LCD+RTC A4/A5.
*/
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <RTClib.h>
#include <SPI.h>
#include <MFRC522.h>
#include <Stepper.h>
#include <PulseSensorPlayground.h>
LiquidCrystal_I2C lcd(0x27,16,2);
RTC_DS3231 rtc;
MFRC522 rfid(10,8);
const int STEPS_PER_REV=2048;
Stepper machine(STEPS_PER_REV,2,4,3,5);
PulseSensorPlayground pulse;
const byte OVERRIDE_BUTTON=6,MQ135_PIN=A0,PULSE_PIN=A1;
const int AIR_LIMIT=450,MIN_BPM=55,MAX_BPM=105;
const byte OPERATING_START=9,OPERATING_END=17;
int bpm=0;
bool overrideEnabled=false,buttonPrev=HIGH;

// IMPORTANT: Connect RC522 VCC to the UNO 3.3V supply.
// Do not connect RC522 VCC directly to a GPIO HIGH; UNO GPIO HIGH is about 5V.
// Set this true only if your shield has a real 3.3V regulator between D7 and RC522 VCC.
const bool SHIELD_HAS_3V3_REGULATOR = false;
const byte RFID_POWER_PIN = 7;
const byte RFID_GROUND_PIN = 9;
void setupRfidPower() {
  if (SHIELD_HAS_3V3_REGULATOR) {
    pinMode(RFID_GROUND_PIN, OUTPUT); digitalWrite(RFID_GROUND_PIN, LOW);
    pinMode(RFID_POWER_PIN, OUTPUT); digitalWrite(RFID_POWER_PIN, HIGH);
    delay(100);
  }
}
```

Project 49: Full Smart Automated Industry Setup — Arduino Code (continued)

```
String uidText(){
  String s="";
  for(byte i=0;i<rfid.uid.size;i++){if(rfid.uid.uidByte[i]<16)s+="0";s+=String(rfid.uid.uidByte[i],HEX);}
  s.toUpperCase();return s;
}
bool validTag(const String& uid){return uid=="A1B2C3D4"||uid=="55667788";}
void setup(){
  pinMode(OVERRIDE_BUTTON,INPUT_PULLUP);
  lcd.init();lcd.backlight();machine.setSpeed(12);
  if(!rtc.begin()){lcd.print("RTC error");while(true);}
  pulse.analogInput(PULSE_PIN);pulse.setThreshold(550);pulse.begin();
  setupRfidPower();SPI.begin();rfid.PCD_Init();
  lcd.print("Industry Ready");
}
void loop(){
  if(pulse.sawStartOfBeat())bpm=pulse.getBeatsPerMinute();

  bool b=digitalRead(OVERRIDE_BUTTON);
  if(buttonPrev==HIGH&&b==LOW)overrideEnabled=!overrideEnabled;
  buttonPrev=b;

  if(rfid.PICC_IsNewCardPresent()&&rfid.PICC_ReadCardSerial()){
    String uid=uidText();DateTime now=rtc.now();
    int air=analogRead(MQ135_PIN);
    bool timeSafe=now.hour()>=OPERATING_START&&now.hour()<OPERATING_END;
    bool pulseSafe=bpm>=MIN_BPM&&bpm<=MAX_BPM;
    bool allSafe=validTag(uid)&&air<AIR_LIMIT&&pulseSafe&&(timeSafe||overrideEnabled);
    lcd.clear();
    if(allSafe){
      lcd.print("START APPROVED");
      lcd.setCursor(0,1);lcd.print("Machine running");
      machine.step(STEPS_PER_REV);
    }else{
      lcd.print("START BLOCKED");
      lcd.setCursor(0,1);
      if(!validTag(uid))lcd.print("Invalid RFID");
      else if(air>=AIR_LIMIT)lcd.print("Air unsafe");
      else if(!pulseSafe)lcd.print("Pulse unsafe");
      else lcd.print("Outside schedule");
    }
    rfid.PICC_HaltA();rfid.PCD_StopCrypto1();
    delay(1800);
  }

  DateTime now=rtc.now();
  lcd.setCursor(0,0);lcd.print("BPM:");lcd.print(bpm);lcd.print(" Air:");lcd.print(analogRead(MQ135_PIN));lcd.print(" ");
  lcd.setCursor(0,1);lcd.print(overrideEnabled?"Override ON     ":"Tap operator ID ");
  delay(40);
}
```

PROJECT 50

Time Display

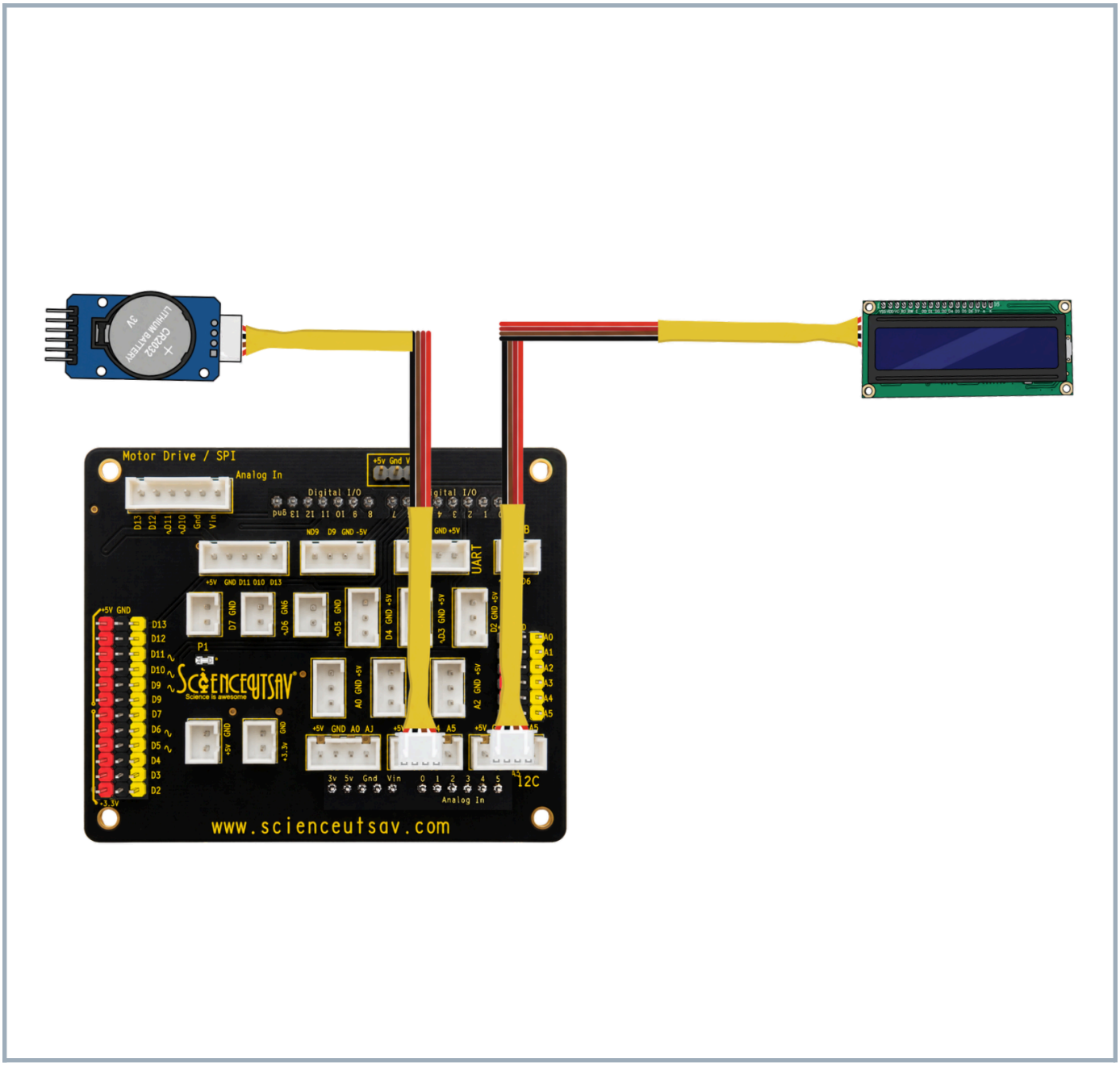
Read and display the current date and time using the RTC module.

Board	Arduino UNO
Components	DS3231/DS1307 RTC, LCD 16x2 I2C
Required libraries	Wire.h; LiquidCrystal_I2C.h; RTClib.h
Sketch file	50_Time_Display.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
DS3231/DS1307 RTC	VCC	5V	Use the module supply rating.
DS3231/DS1307 RTC	GND	GND	Common ground.
DS3231/DS1307 RTC	SDA	A4	Shares the I2C bus with the LCD.
DS3231/DS1307 RTC	SCL	A5	Shares the I2C bus with the LCD.

Project 50: Time Display — Circuit Diagram



Project 50: Time Display — Arduino Code

☑

Sketch file: 50_Time_Display.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <RTClib.h>
LiquidCrystal_I2C lcd(0x27,16,2);
RTC_DS3231 rtc;
void setup(){
 lcd.init();lcd.backlight();
 if(!rtc.begin()){lcd.print("RTC not found");while(true);}
 // Run once if needed, then comment it:
 // rtc.adjust(DateTime(F(__DATE__),F(__TIME__)));
}
void loop(){
 DateTime now=rtc.now();
 char timeText[17],dateText[17];
 snprintf(timeText,sizeof(timeText),"%02d:%02d:%02d",now.hour(),now.minute(),now.second());
 snprintf(dateText,sizeof(dateText),"%02d/%02d/%04d",now.day(),now.month(),now.year());
 lcd.setCursor(0,0);lcd.print(timeText);lcd.print("       ");
 lcd.setCursor(0,1);lcd.print(dateText);lcd.print("      ");
 delay(250);
}
```

PROJECT 51

Stopwatch

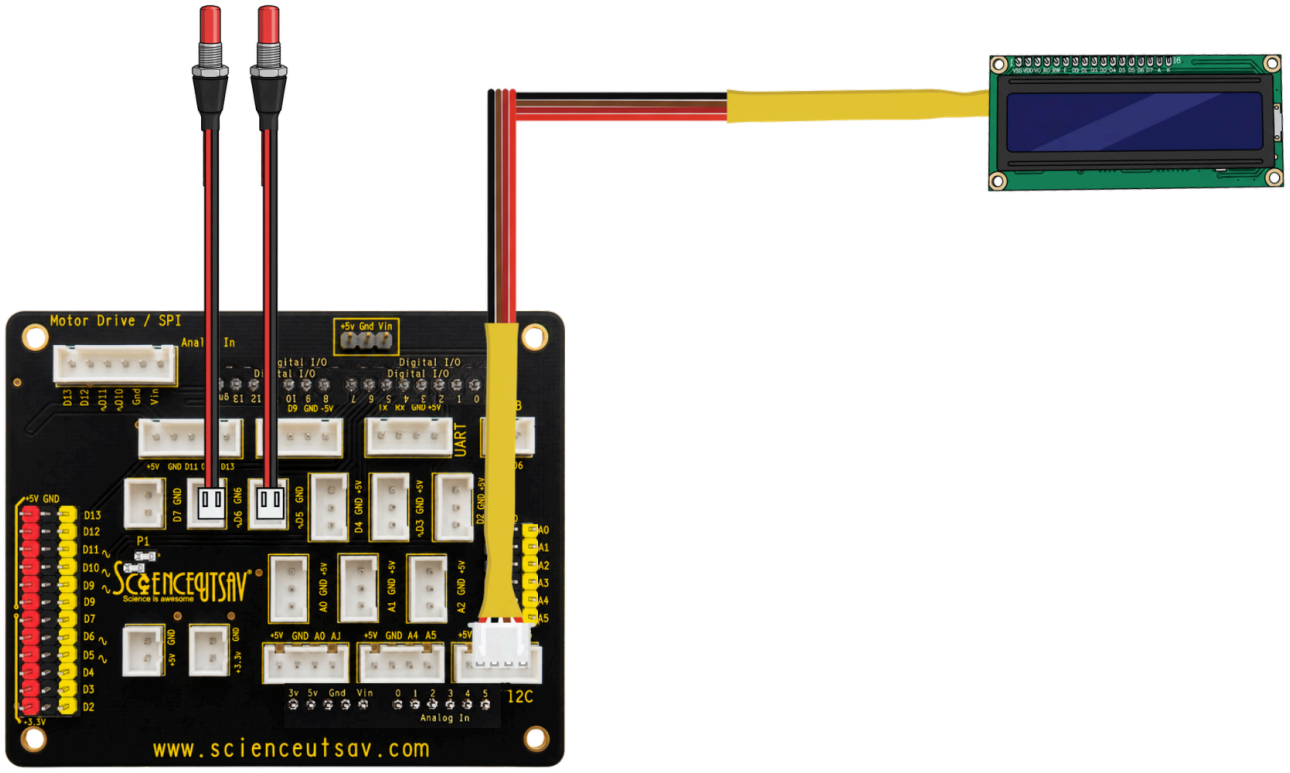
Provide start, stop and reset functions for a stopwatch displayed on the LCD.

Board	Arduino UNO
Components	LCD 16x2 I2C, 2 push buttons
Required libraries	Wire.h; LiquidCrystal_I2C.h
Sketch file	51_Stopwatch.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
Start Stop	Signal terminal	5	Other terminal to GND; code uses INPUT_PULLUP unless noted.
Reset Button	Signal terminal	6	Other terminal to GND; code uses INPUT_PULLUP unless noted.

Project 51: Stopwatch — Circuit Diagram



Project 51: Stopwatch — Arduino Code

Sketch file: 51_Stopwatch.ino

☑

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27,16,2);
const byte START_STOP=5;
const byte RESET_BUTTON=6;
bool running=false;
bool startPrev=HIGH,resetPrev=HIGH;
unsigned long startedAt=0,elapsed=0,lastDisplay=0;

void setup(){
 pinMode(START_STOP,INPUT_PULLUP);
 pinMode(RESET_BUTTON,INPUT_PULLUP);
 lcd.init();lcd.backlight();
}
void loop(){
 bool s=digitalRead(START_STOP),r=digitalRead(RESET_BUTTON);
 if(startPrev==HIGH&&s==LOW){
  if(running){elapsed+=millis()-startedAt;running=false;}
  else{startedAt=millis();running=true;}
 }
 if(resetPrev==HIGH&&r==LOW){running=false;elapsed=0;}
 startPrev=s;resetPrev=r;

 if(millis()-lastDisplay>=50){
  lastDisplay=millis();
  unsigned long total=elapsed+(running?millis()-startedAt:0);
  unsigned long hundredths=(total/10)%100;
  unsigned long seconds=(total/1000)%60;
  unsigned long minutes=(total/60000);
  char text[17];
  snprintf(text,sizeof(text),"%02lu:%02lu.%02lu",minutes,seconds,hundredths);
  lcd.setCursor(0,0);lcd.print("STOPWATCH      ");
  lcd.setCursor(0,1);lcd.print(text);lcd.print(running?" RUN":" STOP");
 }
 delay(1);
}
```

PROJECT 52

Timer Clock

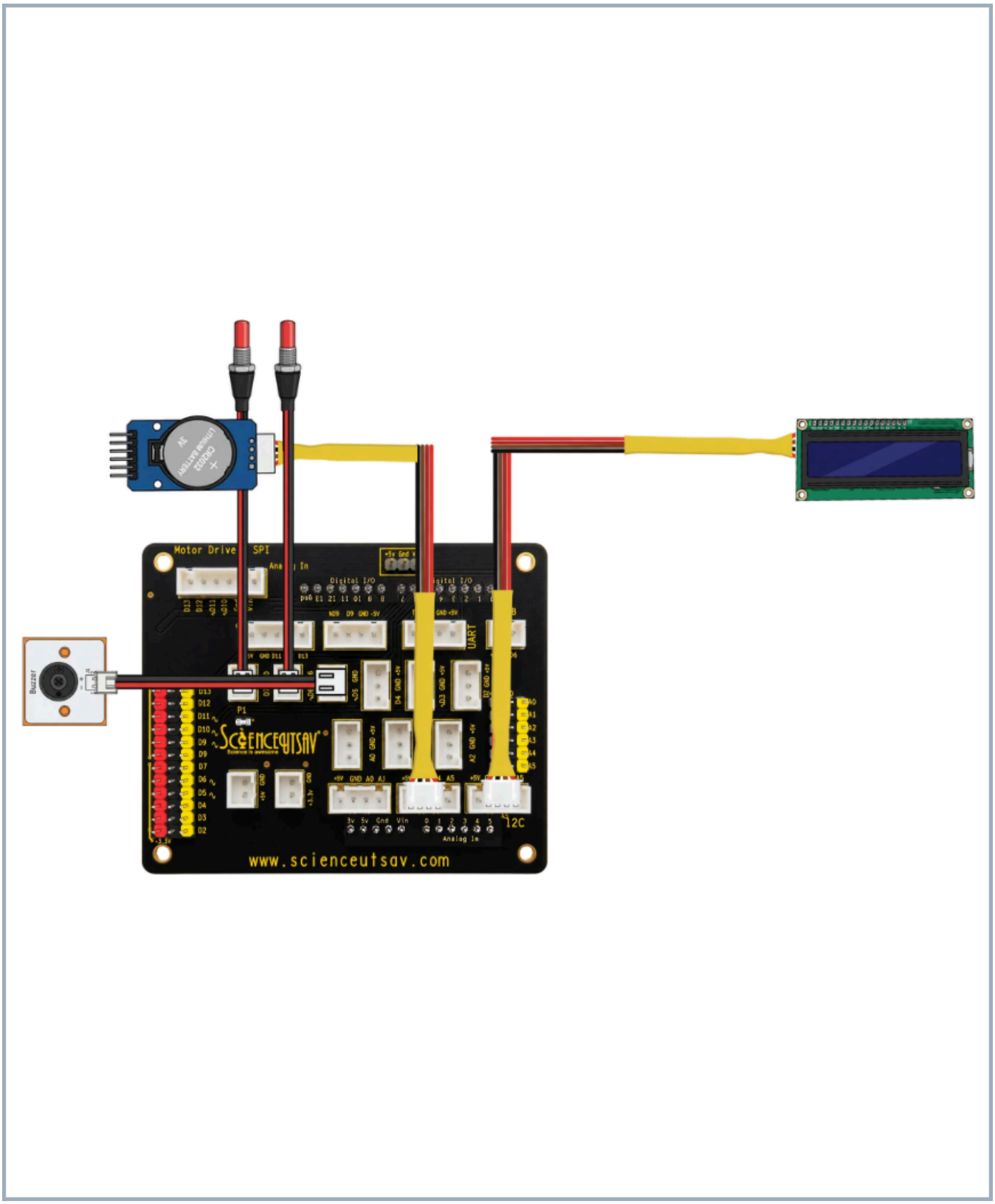
Set an alarm time and sound a buzzer when the RTC reaches it.

Board	Arduino UNO
Components	DS3231 RTC, LCD 16x2 I2C, 3 push buttons, buzzer
Required libraries	Wire.h; LiquidCrystal_I2C.h; RTClib.h
Sketch file	52_Alarm_Clock.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
DS3231/DS1307 RTC	VCC	5V	Use the module supply rating.
DS3231/DS1307 RTC	GND	GND	Common ground.
DS3231/DS1307 RTC	SDA	A4	Shares the I2C bus with the LCD.
DS3231/DS1307 RTC	SCL	A5	Shares the I2C bus with the LCD.
Increase Button	Signal terminal	6	Other terminal to GND; code uses INPUT_PULLUP unless noted.
Change Button	Signal terminal	7	Other terminal to GND; code uses INPUT_PULLUP unless noted.
Buzzer	Positive / signal	5	Negative terminal to GND.

Project 52: Timer Clock — Circuit Diagram



Project 52: Timer Clock — Arduino Code

Sketch file: 52_Timer_Clock.ino

```
/*
 ================= COUNTDOWN TIMER (Arduino Uno) =================
 Wiring:
  LCD (I2C)     -> SDA: A4, SCL: A5, VCC: 5V, GND: GND
  RTC DS3231    -> SDA: A4, SCL: A5 (shares bus with LCD), VCC: 5V, GND: GND
  Buzzer (+)    -> D5   (through a resistor if it's a passive buzzer)
  Button MODE   -> D6   (other leg to GND, uses internal pull-up)
  Button INC    -> D7   (other leg to GND, uses internal pull-up)

 Libraries needed (Sketch > Include Library > Manage Libraries):
  - "LiquidCrystal I2C" (by Frank de Brabander / John Rickman)
  - "RTClib" (by Adafruit)

 How it works:
  1. On power-up you are in SET MINUTES mode. Press INC (D7) to bump minutes.
     Press MODE (D6) to move to SET SECONDS.
  2. In SET SECONDS mode, press INC (D7) to bump seconds. Press MODE (D6)
     to START the countdown.
  3. While RUNNING:
       - MODE (D6) = stop timer and go back to Set Minutes
       - INC  (D7) = pause / resume
  4. When it hits 00:00, the buzzer plays a repeating melody and the LCD
     shows "TIME UP!". Press EITHER button to silence it and return to
     Set Minutes mode.

 Note: The RTC is used to track elapsed time (instead of millis()) so the
 countdown stays accurate even if the LCD/buzzer code takes a little time.
 ================================================================
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <RTClib.h>

// ---------- Pin definitions ----------
#define BUZZER_PIN  5
#define BTN_MODE    6   // set field / start / stop
#define BTN_INC     7   // increment value / pause-resume / silence alarm

// ---------- LCD & RTC ----------
LiquidCrystal_I2C lcd(0x27, 16, 2);   // change to 0x3F if your module needs it
RTC_DS3231 rtc;

// ---------- Melody (original short jingle, loops for as long as alarm is active) ----------
#define NOTE_C4 262
#define NOTE_D4 294
#define NOTE_E4 330
#define NOTE_F4 349
#define NOTE_G4 392
#define NOTE_A4 440
#define NOTE_C5 523
#define NOTE_D5 587
#define NOTE_E5 659
#define NOTE_F5 698
#define NOTE_G5 784
```

```
#define NOTE_A5 880

const int melody[] = {
  NOTE_C5, NOTE_D5, NOTE_E5, NOTE_F5, NOTE_G5, NOTE_F5, NOTE_E5, NOTE_D5,
  NOTE_C5, NOTE_D5, NOTE_E5, NOTE_F5, NOTE_G5, NOTE_A5, NOTE_G5, NOTE_F5,
  NOTE_E5, NOTE_D5, NOTE_C5, NOTE_G4
};
const int noteCount = sizeof(melody) / sizeof(melody[0]);
const int noteDurationMs = 480;   // ~500ms per note * 20 notes ≈ 10 seconds per full pass
const int noteGapMs = 40;         // tiny gap between notes (staccato feel)

int melodyIndex = 0;
unsigned long noteTimer = 0;
bool notePlaying = false;

// ---------- State machine ----------
enum TimerState { SET_MIN, SET_SEC, RUNNING, PAUSED, ALARM };
TimerState state = SET_MIN;

int setMinutes = 0;
int setSeconds = 0;
long targetSeconds = 0;      // total seconds for this run
long remainingAtPause = 0;   // remaining seconds stored when paused
DateTime startStamp;         // RTC time when countdown (re)started

// ---------- Button debouncing ----------
struct Button {
  uint8_t pin;
  bool lastReading;
  bool stableState;
  unsigned long lastChangeTime;
};

Button btnMode = {BTN_MODE, HIGH, HIGH, 0};
Button btnInc  = {BTN_INC,  HIGH, HIGH, 0};

const unsigned long DEBOUNCE_MS = 40;

// Returns true exactly once when the button transitions to "pressed" (LOW)
bool wasPressed(Button &b) {
  bool reading = digitalRead(b.pin);
  if (reading != b.lastReading) {
    b.lastChangeTime = millis();
  }
  bool pressedEdge = false;
  if ((millis() - b.lastChangeTime) > DEBOUNCE_MS) {
    if (reading != b.stableState) {
      b.stableState = reading;
      if (b.stableState == LOW) {
        pressedEdge = true;
      }
    }
  }
  b.lastReading = reading;
  return pressedEdge;
}

// ---------- Display helpers ----------
long lastDisplayedRemaining = -999;
TimerState lastDisplayedState = ALARM; // force first draw

void drawSetScreen() {
```

```
  lcd.setCursor(0, 0);
  lcd.print("Set Timer      ");
  lcd.setCursor(0, 1);
  char buf[17];
  snprintf(buf, sizeof(buf), "%02d:%02d  %s     ", setMinutes, setSeconds,
           (state == SET_MIN) ? "<MIN" : "<SEC");
  lcd.print(buf);
}

void drawRunningScreen(long remaining, bool paused) {
  int m = remaining / 60;
  int s = remaining % 60;
  lcd.setCursor(0, 0);
  lcd.print(paused ? "Paused         " : "Counting down   ");
  lcd.setCursor(0, 1);
  char buf[17];
  snprintf(buf, sizeof(buf), "%02d:%02d          ", m, s);
  lcd.print(buf);
}

void drawAlarmScreen() {
  lcd.setCursor(0, 0);
  lcd.print("   TIME UP!     ");
  lcd.setCursor(0, 1);
  lcd.print(" Press a button ");
}

// ---------- Buzzer melody (non-blocking) ----------
void updateMelody() {
  unsigned long now = millis();
  if (!notePlaying) {
    tone(BUZZER_PIN, melody[melodyIndex]);
    notePlaying = true;
    noteTimer = now;
  } else {
    if (now - noteTimer >= (unsigned long)noteDurationMs) {
      noTone(BUZZER_PIN);
      notePlaying = false;
      noteTimer = now;
      melodyIndex++;
      if (melodyIndex >= noteCount) {
        melodyIndex = 0; // loop the tune
      }
    }
  }
}

void stopMelody() {
  noTone(BUZZER_PIN);
  notePlaying = false;
  melodyIndex = 0;
}

// ---------- Setup ----------
void setup() {
  pinMode(BTN_MODE, INPUT_PULLUP);
  pinMode(BTN_INC, INPUT_PULLUP);
  pinMode(BUZZER_PIN, OUTPUT);

  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
```

```
 lcd.print("Countdown Timer");
 lcd.setCursor(0, 1);
 lcd.print("Initializing...");

 if (!rtc.begin()) {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("RTC not found!");
  while (1) { delay(1000); } // halt - check wiring
 }

 if (rtc.lostPower()) {
  // Set RTC to the compile time if it lost power (only needed once)
  rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
 }

 delay(800);
 lcd.clear();
}

// ---------- Loop ----------
void loop() {
 bool modePressed = wasPressed(btnMode);
 bool incPressed  = wasPressed(btnInc);

 switch (state) {

  case SET_MIN: {
   if (incPressed) {
    setMinutes = (setMinutes + 1) % 60;
   }
   if (modePressed) {
    state = SET_SEC;
   }
   if (state != lastDisplayedState || true) {
    drawSetScreen();
   }
   lastDisplayedState = state;
   break;
  }

  case SET_SEC: {
   if (incPressed) {
    setSeconds = (setSeconds + 1) % 60;
   }
   if (modePressed) {
    targetSeconds = (long)setMinutes * 60 + setSeconds;
    if (targetSeconds <= 0) {
     // nothing to count down, stay in set mode
     state = SET_MIN;
    } else {
     startStamp = rtc.now();
     state = RUNNING;
     lastDisplayedRemaining = -999; // force redraw
    }
   }
   drawSetScreen();
   lastDisplayedState = state;
   break;
  }

  case RUNNING: {
```

```
  if (modePressed) {
    // stop -> back to setting minutes, reset values
    state = SET_MIN;
    setMinutes = 0;
    setSeconds = 0;
    lastDisplayedState = ALARM; // force redraw of set screen
    break;
  }
  if (incPressed) {
    // pause
    DateTime now = rtc.now();
    long elapsed = now.unixtime() - startStamp.unixtime();
    remainingAtPause = targetSeconds - elapsed;
    if (remainingAtPause < 0) remainingAtPause = 0;
    state = PAUSED;
    lastDisplayedRemaining = -999;
    break;
  }

  DateTime now = rtc.now();
  long elapsed = now.unixtime() - startStamp.unixtime();
  long remaining = targetSeconds - elapsed;

  if (remaining <= 0) {
    state = ALARM;
    stopMelody();
    drawAlarmScreen();
    break;
  }

  if (remaining != lastDisplayedRemaining) {
    drawRunningScreen(remaining, false);
    lastDisplayedRemaining = remaining;
  }
  break;
}

case PAUSED: {
  if (modePressed) {
    // stop -> back to setting minutes, reset values
    state = SET_MIN;
    setMinutes = 0;
    setSeconds = 0;
    lastDisplayedState = ALARM; // force redraw of set screen
    break;
  }
  if (incPressed) {
    // resume
    targetSeconds = remainingAtPause;
    startStamp = rtc.now();
    state = RUNNING;
    lastDisplayedRemaining = -999;
    break;
  }
  if (remainingAtPause != lastDisplayedRemaining) {
    drawRunningScreen(remainingAtPause, true);
    lastDisplayedRemaining = remainingAtPause;
  }
  break;
}

case ALARM: {
```

```
    updateMelody();
    if (modePressed || incPressed) {
      stopMelody();
      state = SET_MIN;
      setMinutes = 0;
      setSeconds = 0;
      lastDisplayedState = ALARM; // force redraw
      lastDisplayedRemaining = -999;
    }
    break;
  }
 }

 if (state != RUNNING && state != PAUSED) {
  // small delay only when not time-critical, keeps buttons responsive
  delay(5);
 }
}
```

PROJECT 53

Time-Based Lock

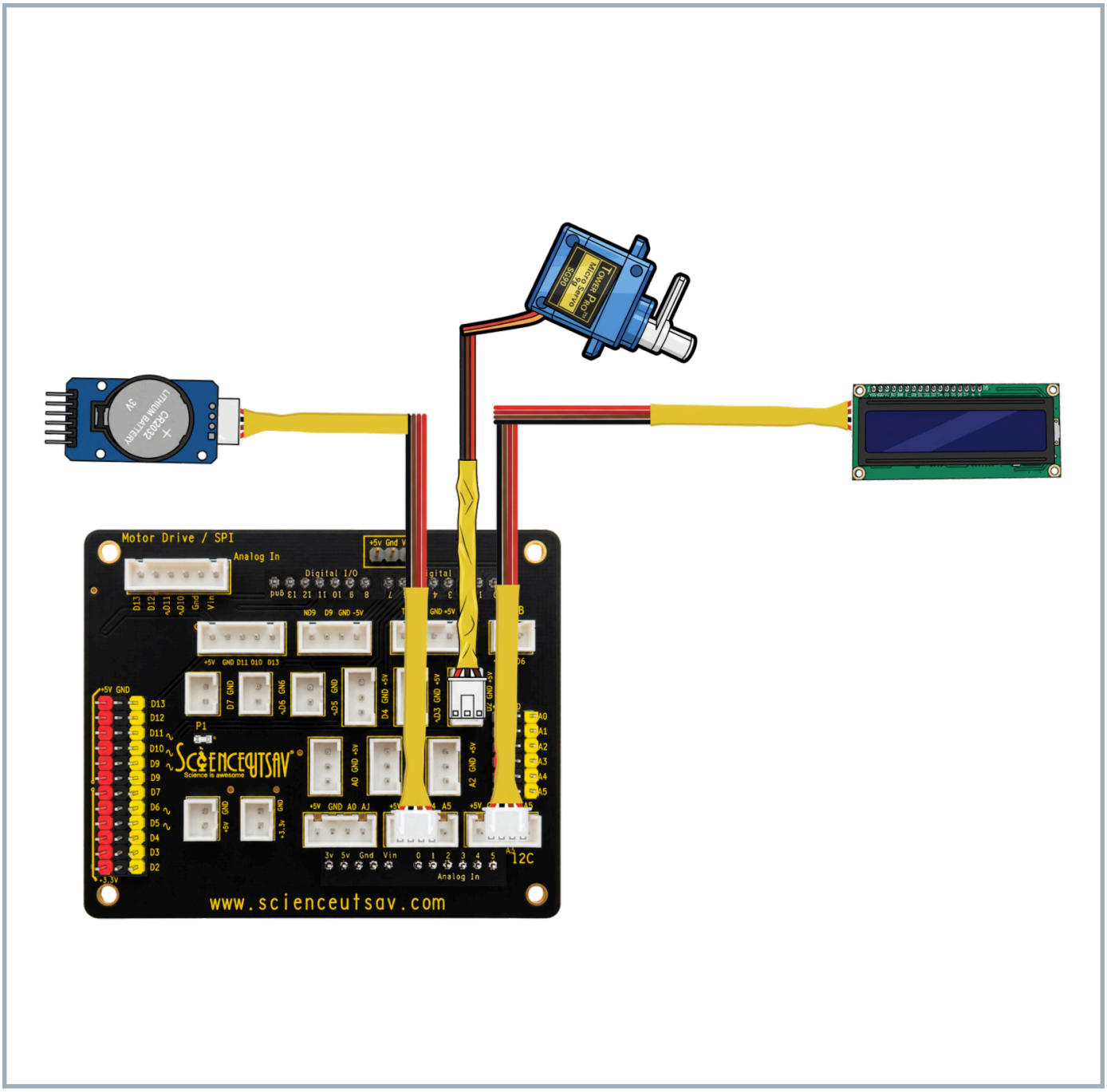
Automatically unlock the door during configured working hours and lock it outside that window.

Board	Arduino UNO
Components	DS3231 RTC, LCD 16x2 I2C, servo motor
Required libraries	Wire.h; LiquidCrystal_I2C.h; RTClib.h; Servo.h
Sketch file	53_Time_Based_Lock.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
LCD 16x2 I2C	VCC	5V	Use the LCD backpack module.
LCD 16x2 I2C	GND	GND	Common ground.
LCD 16x2 I2C	SDA	A4	I2C data line; address 0x27.
LCD 16x2 I2C	SCL	A5	I2C clock line; address 0x27.
DS3231/DS1307 RTC	VCC	5V	Use the module supply rating.
DS3231/DS1307 RTC	GND	GND	Common ground.
DS3231/DS1307 RTC	SDA	A4	Shares the I2C bus with the LCD.
DS3231/DS1307 RTC	SCL	A5	Shares the I2C bus with the LCD.
Servo motor	Signal	3	Control signal from Arduino.
Servo motor	VCC	5V / external 5V	External supply is preferred for reliable operation.
Servo motor	GND	GND	Common ground with Arduino.

Project 53: Time-Based Lock — Circuit Diagram



Project 53: Time-Based Lock — Arduino Code

Sketch file: 53_Time_Based_Lock.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <RTClib.h>
#include <Servo.h>
LiquidCrystal_I2C lcd(0x27,16,2);
RTC_DS3231 rtc;
Servo door;
const byte SERVO_PIN=3;
const byte OPEN_HOUR=9;
const byte CLOSE_HOUR=17;
void setup(){
 door.attach(SERVO_PIN);door.write(0);
 lcd.init();lcd.backlight();
 if(!rtc.begin()){lcd.print("RTC error");while(true);}
}
void loop(){
 DateTime now=rtc.now();
 bool accessTime=now.hour()>=OPEN_HOUR&&now.hour()<CLOSE_HOUR;
 door.write(accessTime?90:0);
 char line1[17];
 snprintf(line1,sizeof(line1),"%02d:%02d:%02d",now.hour(),now.minute(),now.second());
 lcd.setCursor(0,0);lcd.print(line1);lcd.print("       ");
 lcd.setCursor(0,1);lcd.print(accessTime?"ACCESS OPEN    ":"ACCESS CLOSED   ");
 delay(250);
}
```

PROJECT 54

Shuffle Playlist

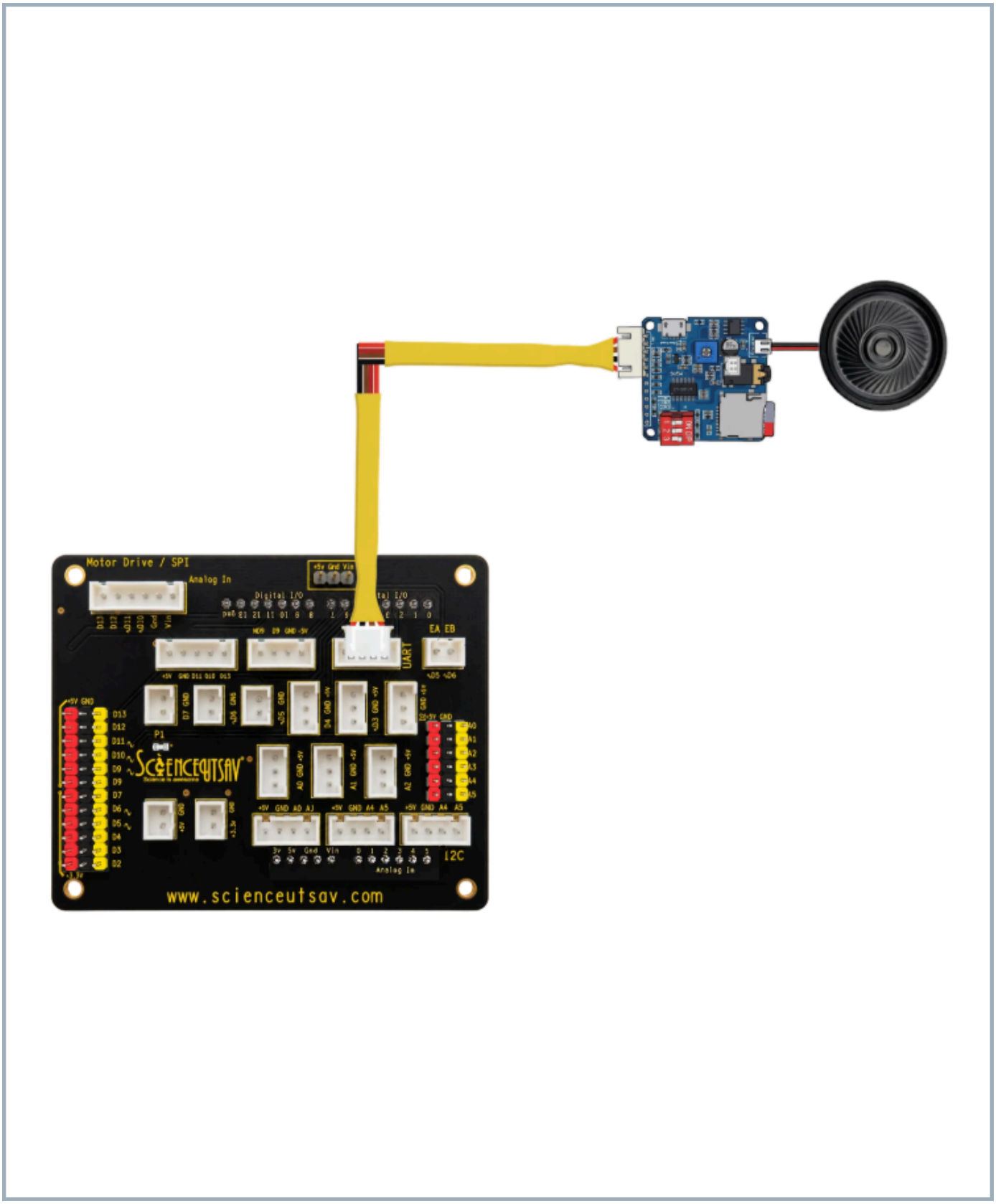
Play tracks from the DFPlayer Mini in random order.

Board	Arduino UNO
Components	DFPlayer Mini, speaker, microSD card
Required libraries	SoftwareSerial.h; DFRobotDFPlayerMini.h
Sketch file	54_Shuffle_Playlist.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
DY-SW05 MP3	TX	0	Connect DFPlayer TX to Arduino SoftwareSerial RX.
DY-SW05 MP3	RX	1	Connect through about a 1 kΩ series resistor.
DY-SW05 MP3	VCC	5V	Use a stable supply.
DY-SW05 MP3	GND	GND	Common ground with Arduino.
Speaker	SPK1 / SPK2	DY-SW05 MP3	No Arduino I/O pin is required.

Project 54: Shuffle Playlist — Circuit Diagram



Project 54: Shuffle Playlist — Arduino Code

Sketch file: 54_Shuffle_Playlist.ino

```
/*
  ScienceUtsav RoboScientist Project
  Board: Arduino UNO
  LCD I2C: SDA=A4, SCL=A5, address 0x27

  Install only the libraries included by this sketch.
  Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

/*
  DY-SW05 wiring:
  DY-SW05 TX  -> Arduino D8 (SoftwareSerial RX)
  DY-SW05 RX  -> Arduino D9 (SoftwareSerial TX)
  DY-SW05 GND -> Arduino GND (common ground)
  DY-SW05 5V  -> stable 5V supply
  Speaker -> DY-SW05 speaker output terminals.
  Verify your module's exact command set against its datasheet before relying on this in the field.
*/

#include <SoftwareSerial.h>
SoftwareSerial mp3Serial(8,9); // RX, TX
const byte TRACK_COUNT=6;
unsigned long nextTrackAt=0;

// DY-SW05 frame format: AA CMD LEN [DATA...] SUM
// SUM = (AA + CMD + LEN + DATA bytes) & 0xFF
void sendCommand(byte cmd, byte* data, byte dataLen){
  byte sum=0xAA+cmd+dataLen;
  mp3Serial.write(0xAA);
  mp3Serial.write(cmd);
  mp3Serial.write(dataLen);
  for(byte i=0;i<dataLen;i++){
    mp3Serial.write(data[i]);
    sum+=data[i];
  }
  mp3Serial.write(sum);
}

void setVolume(byte level){ // 0-30
  byte data[1]={level};
  sendCommand(0x13,data,1);
}

void playTrack(byte trackNum){
  byte data[2]={0,trackNum}; // high byte, low byte of track number
  sendCommand(0x0F,data,2);
}
```

```
void setup(){
 Serial.begin(9600);
 mp3Serial.begin(9600);
 randomSeed(analogRead(A3));
 delay(500); // let the module finish booting
 setVolume(22);
}
void loop(){
 if(millis()>=nextTrackAt){
  byte track=random(1,TRACK_COUNT+1);
  Serial.print("Playing track ");Serial.println(track);
  playTrack(track);
  nextTrackAt=millis()+30000; // Set longer than your typical song or add BUSY-pin sensing.
 }
}
```

PROJECT 55

Song Controller

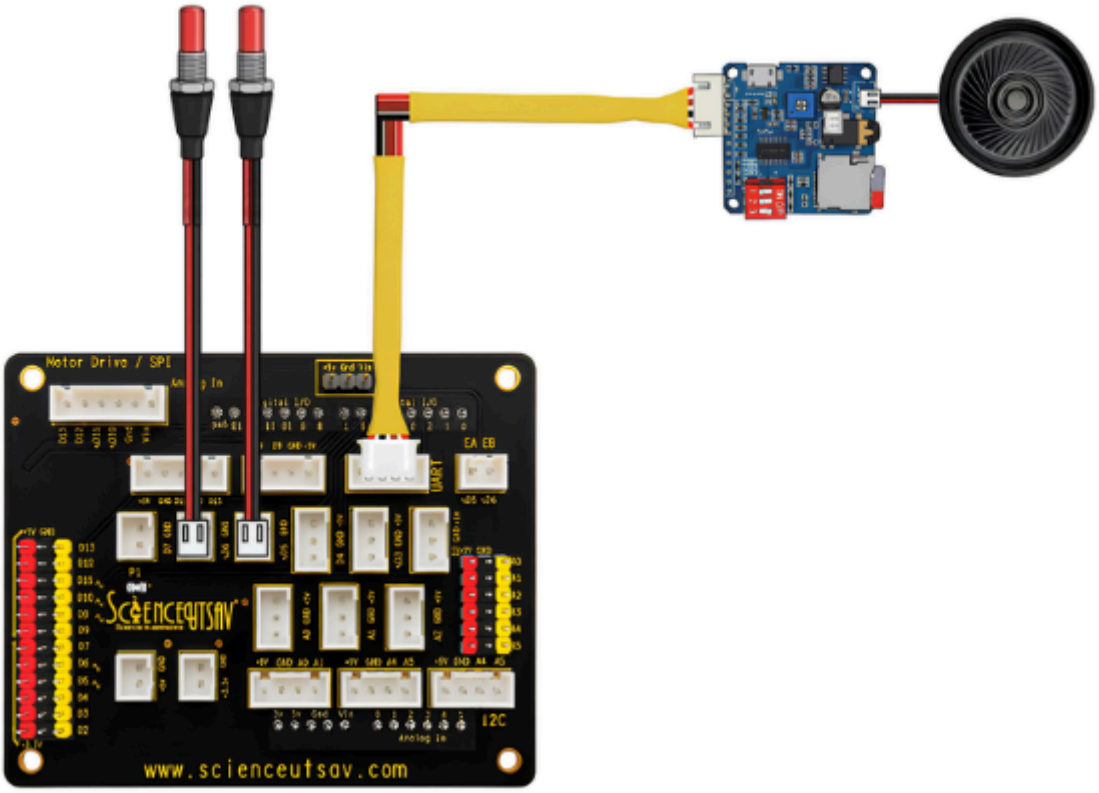
Use previous and next buttons to control DFPlayer track playback.

Board	Arduino UNO
Components	DFPlayer Mini, speaker, microSD card, 2 push buttons
Required libraries	SoftwareSerial.h; DFRobotDFPlayerMini.h
Sketch file	55_Song_Controller.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
DY-SW05 MP3	TX	0	Connect DFPlayer TX to Arduino SoftwareSerial RX.
DY-SW05 MP3	RX	1	Connect through about a 1 kΩ series resistor.
DY-SW05 MP3	VCC	5V	Use a stable supply.
DY-SW05 MP3	GND	GND	Common ground with Arduino.
Speaker	SPK1 / SPK2	DY-SW05 MP3	No Arduino I/O pin is required.
Previous Button	Signal terminal	5	Other terminal to GND; code uses INPUT_PULLUP unless noted.
Next Button	Signal terminal	6	Other terminal to GND; code uses INPUT_PULLUP unless noted.

Project 55: Song Controller — Circuit Diagram



Project 55: Song Controller — Arduino Code

Sketch file: 55_Song_Controller.ino

☑

```
/*
 DY-SV5W MP3 Module Player - 6 Track Loop
 ----------------------------------------
 Wiring:
  Arduino D8  -> Module TX   (Arduino RX, receiving)
  Arduino D9  -> Module RX   (Arduino TX, transmitting)
              (use a 1k resistor or level shifter from D9 to module RX
               if your module is not 5V tolerant)
  Arduino D5  -> Push button -> GND   (Previous track)
  Arduino D6  -> Push button -> GND   (Next track)
  Module VCC  -> 5V, Module GND -> GND

 Behavior:
  - Starts on track 1.
  - Button on D5 = Previous track (wraps 1 -> 6).
  - Button on D6 = Next track (wraps 6 -> 1).
  - Total of 6 songs (track numbers 1 to 6).

 Notes:
  - Uses internal pull-ups, so buttons connect the pin to GND when pressed.
  - Debounced with millis() (no delay() blocking).
  - Command protocol used is the standard DY-SV5W UART frame:
      AA CMD LEN [DATA...] SM
    where SM = (AA + CMD + LEN + DATA bytes) & 0xFF
    CMD 0x07 = play specified track number (2 data bytes: high, low)
  - If your specific module firmware expects a different command set,
    only the sendCommand()/playTrack() functions need to change.
*/

#include <SoftwareSerial.h>

// RX, TX  (Arduino RX = D8 connects to module TX, Arduino TX = D9 connects to module RX)
SoftwareSerial mp3Serial(8, 9);

const int BTN_PREV = 5;
const int BTN_NEXT = 6;

const int TOTAL_TRACKS = 6;
int currentTrack = 1;

// Debounce handling
unsigned long lastPrevPress = 0;
unsigned long lastNextPress = 0;
const unsigned long debounceDelay = 200; // ms

void setup() {
 pinMode(BTN_PREV, INPUT_PULLUP);
 pinMode(BTN_NEXT, INPUT_PULLUP);

 mp3Serial.begin(9600);   // common default baud rate for DY-SV5W
 Serial.begin(9600);      // for debug output on Serial Monitor

 delay(500); // let module boot up
 playTrack(currentTrack);
}
```

```
void loop() {
  // Previous button
  if (digitalRead(BTN_PREV) == LOW) {
    if (millis() - lastPrevPress > debounceDelay) {
      lastPrevPress = millis();
      currentTrack--;
      if (currentTrack < 1) {
        currentTrack = TOTAL_TRACKS; // wrap from 1 -> 6
      }
      playTrack(currentTrack);
    }
  }

  // Next button
  if (digitalRead(BTN_NEXT) == LOW) {
    if (millis() - lastNextPress > debounceDelay) {
      lastNextPress = millis();
      currentTrack++;
      if (currentTrack > TOTAL_TRACKS) {
        currentTrack = 1; // wrap from 6 -> 1
      }
      playTrack(currentTrack);
    }
  }
}

// Sends the "play specified track" command to the DY-SV5W module
void playTrack(int trackNum) {
  byte highByte = (trackNum >> 8) & 0xFF;
  byte lowByte  = trackNum & 0xFF;

  byte cmd[6];
  cmd[0] = 0xAA;       // start byte
  cmd[1] = 0x07;       // command: play specified track
  cmd[2] = 0x02;       // data length
  cmd[3] = highByte;    // track number high byte
  cmd[4] = lowByte;     // track number low byte
  cmd[5] = checksum(cmd, 5);

  mp3Serial.write(cmd, 6);

  Serial.print("Playing track: ");
  Serial.println(trackNum);
}

// Calculates checksum = sum of all previous bytes, truncated to 1 byte
byte checksum(byte *data, int len) {
  int sum = 0;
  for (int i = 0; i < len; i++) {
    sum += data[i];
  }
  return sum & 0xFF;
}
```

PROJECT 56

Keypad Song Control

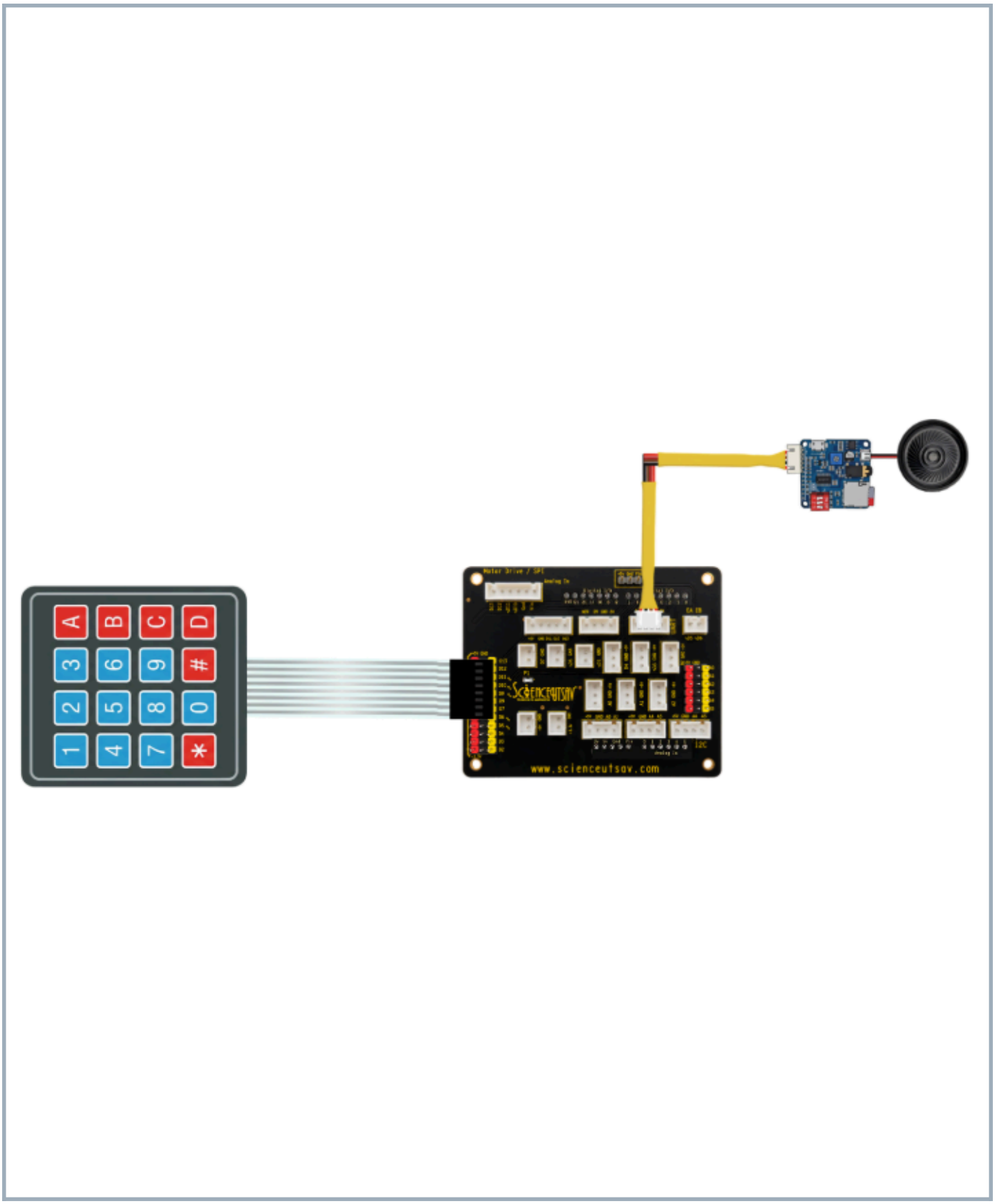
Play the track associated with the number key pressed on the keypad.

Board	Arduino UNO
Components	DFPlayer Mini, speaker, microSD card, 4x4 keypad
Required libraries	SoftwareSerial.h; DFRobotDFPlayerMini.h; Keypad.h
Sketch file	56_Keypad_Song_Control.ino

Connection Details

Component	Module pin / terminal	Arduino / shield pin	Connection note
4x4 keypad	R1	6	Keypad row wire.
4x4 keypad	R2	7	Keypad row wire.
4x4 keypad	R3	8	Keypad row wire.
4x4 keypad	R4	9	Keypad row wire.
4x4 keypad	C1	10	Keypad column wire.
4x4 keypad	C2	11	Keypad column wire.
4x4 keypad	C3	12	Keypad column wire.
4x4 keypad	C4	13	Keypad column wire.
DY-SW05 MP3	TX	0	Connect DFPlayer TX to Arduino SoftwareSerial RX.
DY-SW05 MP3	RX	1	Connect through about a 1 kΩ series resistor.
DY-SW05 MP3	VCC	5V	Use a stable supply.
DY-SW05 MP3	GND	GND	Common ground with Arduino.
Speaker	SPK1 / SPK2	DFPlayer SPK1 / SPK2	No Arduino I/O pin is required.

Project 56: Keypad Song Control — Circuit Diagram



Project 56: Keypad Song Control — Arduino Code

Sketch file: 56_Keypad_Song_Control.ino

```
/*
 ScienceUtsav RoboScientist Project
 Board: Arduino UNO
 LCD I2C: SDA=A4, SCL=A5, address 0x27

 Install only the libraries included by this sketch.
 Adjust thresholds, UIDs, timings and mechanical angles during testing.
*/

/*
 DY-SW05 wiring:
 DY-SW05 TX  -> Arduino D8 (SoftwareSerial RX)
 DY-SW05 RX  -> Arduino D9 (SoftwareSerial TX)
 DY-SW05 GND -> Arduino GND
 DY-SW05 5V  -> stable 5V supply
*/

#include <SoftwareSerial.h>
#include <Keypad.h>
SoftwareSerial mp3Serial(8,9);
const byte ROWS=4,COLS=4;
char keys[ROWS][COLS]={{'1','2','3','A'},{'4','5','6','B'},{'7','8','9','C'},{'*','0','#','D'}};
byte rowPins[ROWS]={6,7,8,9};
byte colPins[COLS]={10,11,12,13};
Keypad keypad=Keypad(makeKeymap(keys),rowPins,colPins,ROWS,COLS);

void sendCommand(byte cmd, byte* data, byte dataLen){
 byte sum=0xAA+cmd+dataLen;
 mp3Serial.write(0xAA);mp3Serial.write(cmd);mp3Serial.write(dataLen);
 for(byte i=0;i<dataLen;i++){mp3Serial.write(data[i]);sum+=data[i];}
 mp3Serial.write(sum);
}
void setVolume(byte level){byte d[1]={level};sendCommand(0x13,d,1);}
void playTrack(byte trackNum){byte d[2]={0,trackNum};sendCommand(0x0F,d,2);}
void volumeUp(){sendCommand(0x14,nullptr,0);}
void volumeDown(){sendCommand(0x15,nullptr,0);}
void pauseTrack(){sendCommand(0x03,nullptr,0);}
void resumeTrack(){sendCommand(0x04,nullptr,0);}

void setup(){
 Serial.begin(9600);mp3Serial.begin(9600);
 delay(500);
 setVolume(22);
}
```

```
void loop(){
  char key=keypad.getKey();
  if(key>='1'&&key<='9'){byte track=key-'0';playTrack(track);Serial.println(track);}
  else if(key=='0')playTrack(10);
  else if(key=='A')volumeUp();
  else if(key=='B')volumeDown();
  else if(key=='C')pauseTrack();
  else if(key=='D')resumeTrack();
}
```